



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Universitat Politècnica
de València

**Departamento de Sistemas Informáticos y
Computación**



Máster Universitario en Ingeniería y Tecnología de Sistemas
Software

Trabajo Fin de Máster

**Integración de Modelos de Lenguaje en el
Modelado BPMN: Extensión de bpmn.io
para generar procesos de negocio e
identificar el valor**

Autor(a): Sergio Muñoz Gómez
Director(a): César Emilio Insfrán Pelozo
Cotutor(a): Denis Silva Da Silveira

Valencia, «mes año»

Este Trabajo Fin de Máster se ha depositado en el Departamento de Sistemas Informàticos y Computación de la Universitat Politècnica de València para su defensa.

Trabajo Fin de Máster

Máster Universitario en Ingeniería y Tecnología de Sistemas Software

Título: Integración de Modelos de Lenguaje en el Modelado BPMN: Extensión de bpmn.io para generar procesos de negocio e identificar el valor

«mes año»

Autor(a): Sergio Muñoz Gómez

Director(a): César Emilio Insfrán Pelozo

Departamento de Sistemas Informàticos y Computación
Universitat Politècnica de València

Resum

Els processos de comerç electrònic estan dissenyats per a aportar valor als clients mitjançant atributs com la qualitat, el preu i elements emocionals i socials. No obstant això, el valor modelat per l'organització no sempre es correspon amb el valor percebut pels usuaris durant la seua interacció real amb el procés. Aquesta desalineació pot afectar directament l'experiència del client i l'eficàcia del procés.

Aquest Treball de Fi de Màster se centra en el desenvolupament d'una ferramenta basada en models de llenguatge (LLM) que permet la generació i l'anàlisi de models BPMN a partir de descripcions en llenguatge natural. La ferramenta s'ha desenvolupat com una extensió sobre l'editor de codi obert bpmn.io, connectada mitjançant un servidor intermediari basat en el protocol *Model Context Protocol* (MCP) als proveïdors de LLM. L'extensió implementa un paradigma de *vibe modeling* guiat i híbrid (*guided and hybrid vibe modeling*): l'usuari construeix i refina el model de manera iterativa mitjançant instruccions en llenguatge natural —tant per text com per veu— seguint una estratègia de *prompting* iteratiu amb validació humana en el bucle (*Iterative prompting with human-in-the-loop validation*), i pot a més modificar directament el diagrama en la interfície de bpmn.io, sent l'XML del model l'estat compartit entre l'usuari i el LLM. La ferramenta permet també identificar els valors associats a la percepció del client segons la classificació PERVAL. Per a avaluar la ferramenta desenvolupada, s'ha dut a terme una enquesta basada en el model d'acceptació de la tecnologia (TAM) amb un grup d'aproximadament quinze usuaris, amb l'objectiu de valorar la utilitat percebuda, la facilitat d'ús percebuda i la intenció d'ús del sistema.

Com a cas d'estudi, s'han utilitzat processos de compra en línia inspirats en escenaris reals de comerç electrònic, sobre els quals s'han realitzat diverses iteracions de modelatge i anàlisi. El resultat principal del projecte és, per tant, doble: (1) el desenvolupament d'una ferramenta basada en models de llenguatge, amb arquitectura MCP, per a la generació, edició conversacional i anàlisi PERVAL de models BPMN, i (2) la validació de l'acceptació d'aquesta ferramenta per part dels usuaris mitjançant el model TAM.

Resumen

Los procesos de comercio electrónico están diseñados para entregar valor al cliente mediante atributos como la calidad, el precio, los elementos emocionales y sociales. Sin embargo, no siempre el valor modelado por la organización corresponde al valor percibido por los usuarios durante la interacción real con el proceso. Este desalineamiento puede impactar directamente la experiencia del cliente y la eficacia del proceso.

Este Trabajo de Fin de Máster se centra en el desarrollo de una herramienta basada en modelos de lenguaje (LLM) que permite generar y analizar modelos BPMN a partir de descripciones en lenguaje natural. La herramienta se ha desarrollado como una extensión sobre el editor de código abierto bpmn.io, conectada mediante un servidor intermedio basado en el protocolo *Model Context Protocol* (MCP) a los proveedores de LLM. La extensión implementa un paradigma de *vibe modeling* guiado e híbrido (*guided and hybrid vibe modeling*): el usuario construye y refina el modelo de forma iterativa mediante instrucciones en lenguaje natural —ya sea por texto o por voz— aplicando una estrategia de prompting iterativo con validación humana en el bucle (*Iterative prompting with human-in-the-loop validation*), y puede además modificar directamente el diagrama en la interfaz de bpmn.io, siendo el XML del modelo el estado compartido entre el usuario y el LLM. La herramienta permite también identificar los valores asociados a la percepción del cliente según la clasificación PERVAL. Para evaluar la herramienta desarrollada, se ha llevado a cabo una encuesta basada en el modelo de aceptación de la tecnología (TAM) con un grupo de aproximadamente quince usuarios, con el objetivo de valorar la utilidad percibida, la facilidad de uso percibida y la intención de uso del sistema.

Como caso de estudio, se han utilizado procesos de compra en línea inspirados en escenarios reales de comercio electrónico, sobre los cuales se han realizado distintas iteraciones de modelado y análisis. El principal resultado del proyecto es, por tanto, doble: (1) el desarrollo de una herramienta basada en modelos de lenguaje, con arquitectura MCP, para la generación, edición conversacional y análisis PERVAL de modelos BPMN, y (2) la validación de la aceptación de dicha herramienta por parte de los usuarios mediante el modelo TAM.

Abstract

E-commerce processes are designed to deliver value to customers through attributes such as quality, price, and emotional and social elements. However, the value modeled by the organization does not always correspond to the value perceived by users during their actual interaction with the process. This misalignment can directly impact the customer experience and the effectiveness of the process.

This Master's Thesis focuses on the development of a tool based on language models (LLMs) that enables the generation and analysis of BPMN models from natural language descriptions. The tool has been developed as an extension of the open-source bpmn.io editor, connected through an intermediate server based on the *Model Context Protocol* (MCP) to LLM providers. The extension implements a guided and hybrid vibe modeling paradigm: the user builds and refines the model iteratively through natural language instructions —either via text or voice— following an Iterative prompting with human-in-the-loop validation strategy, and can also directly edit the BPMN diagram within the bpmn.io interface, with the model's XML acting as a shared state between the user and the LLM. The tool additionally identifies values associated with customer perception according to the PERVAL classification. To evaluate the developed tool, a survey based on the Technology Acceptance Model (TAM) was conducted with a group of approximately fifteen users, aiming to assess perceived usefulness, perceived ease of use, and intention to use the system.

As a case study, online purchasing processes inspired by real e-commerce scenarios were used, on which various modeling and analysis iterations were performed. The main outcome of the project is therefore twofold: (1) the development of a language-model-based tool, with MCP architecture, for the generation, conversational editing, and PERVAL analysis of BPMN models, and (2) the validation of user acceptance of the tool through the TAM model.

Tabla de contenidos

1. Introducción	1
1.1. Motivación	2
1.2. Objetivos	2
1.3. Impacto Esperado	3
1.4. Metodología	3
1.5. Estructura	4
1.6. Convenciones	5
2. Fundamentos	7
2.1. Modelos de lenguaje de gran escala (LLMs)	7
2.2. BPMN: notación y modelado de procesos de negocio	9
2.3. La clasificación PERVAL	11
2.4. El protocolo <i>Model Context Protocol</i> (MCP)	12
3. Estado del arte	15
3.1. Análisis de trabajos relacionados	15
3.1.1. LLMs en la gestión de procesos de negocio	15
3.1.2. Generación automática de modelos BPMN 2.0 desde descripciones textuales	16
3.1.3. IA generativa y modelado conversacional de procesos	17
3.1.4. BPMN-Chatbot: modelado de procesos mediante interfaz conversacional	17
3.2. Síntesis del estado del arte y oportunidades de investigación	18
4. Análisis del problema	21
4.1. Modelado funcional del sistema	21
4.1.1. Diagrama de actores	22
4.1.2. Diagramas de casos de uso	23
4.2. Análisis del marco legal y ético	24
4.2.1. Protección de datos	24
4.2.2. Propiedad intelectual y licencias	25
4.2.3. Consideraciones éticas	25
4.3. Identificación y análisis de soluciones posibles	26
4.3.1. Selección del editor BPMN base	26
4.3.2. Selección del proveedor de LLM	27
4.3.3. Arquitectura de integración: extensión directa frente a servidor MCP	28
4.3.4. Tecnologías para el desarrollo del <i>frontend</i> y del servidor	28

4.4. Solución propuesta	31
4.4.1. Requisitos funcionales	31
4.4.2. Requisitos no funcionales	31
4.4.3. Priorización de requisitos. MoSCoW	32
4.4.4. Historias de usuario	35
4.5. Plan de trabajo	40
5. Diseño de la solución	43
5.1. Arquitectura del sistema	43
5.1.1. Justificación de la arquitectura adoptada	44
5.1.2. Arquitectura detallada por componente	45
5.1.2.1. Extensión sobre bpmn.io (cliente MCP)	45
5.1.2.2. Servidor MCP	46
5.1.2.2.1. Herramienta de análisis PERVAL (analyze_perval).	47
5.2. Diseño detallado	48
5.2.1. Modelo de datos	48
5.2.2. Flujo de comunicación	49
5.2.2.1. Generación y refinamiento iterativo de modelos BPMN	49
5.2.2.2. Análisis PERVAL	50
5.3. Tecnología utilizada	51
5.3.1. Diseño	51
5.3.2. Programación	52
5.3.2.1. Lenguajes y tecnologías utilizados en cada componente	52
5.3.2.1.1. Extensión sobre bpmn.io.	52
5.3.2.1.2. Servidor MCP.	52
5.3.2.2. Frameworks y bibliotecas	53
5.3.2.3. Entornos de desarrollo	54
5.3.3. Control de versiones	54
5.3.4. Despliegue	54
6. Desarrollo e implementación	55
6.1. Evolución del proyecto por <i>sprints</i>	55
6.1.1. <i>Sprint</i> 0 (26/05/2026 – 01/06/2026)	55
6.1.2. <i>Sprint</i> 1 (02/06/2026 – 08/06/2026)	56
6.1.3. <i>Sprint</i> 2 (09/06/2026 – 15/06/2026)	56
6.1.4. <i>Sprint</i> 3 (16/06/2026 – 22/06/2026)	57
6.1.5. <i>Sprint</i> 4 (23/06/2026 – 29/06/2026)	57
6.1.6. <i>Sprint</i> 5 (30/06/2026 – 06/07/2026)	58
6.1.7. <i>Sprint</i> 6 (07/07/2026 – 13/07/2026)	58
6.1.8. <i>Sprint</i> 7 (14/07/2026 – 20/07/2026)	59
6.2. Implementación de los componentes técnicos clave	59
6.2.1. La función <code>callOpenAI</code> y el mecanismo de <i>fallback</i>	60
6.2.2. Prompts del flujo de generación iterativa	61
6.2.2.1. <code>buildStructureSystemPrompt</code> — herramienta <code>identify_structure</code>	61
6.2.2.2. <code>buildModifyStructurePrompt</code> — herramienta <code>modify_structure</code>	62
6.2.2.3. <code>buildFlowStartPrompt</code> — herramienta <code>suggest_flow_start</code>	62
6.2.2.4. <code>buildNextStepPrompt</code> — herramienta <code>suggest_next_step</code>	63
6.2.2.5. <code>buildGatewayBranchesPrompt</code> — herramienta <code>suggest_gateway_branches</code>	64
6.2.2.6. <code>buildBranchStepPrompt</code> — herramienta <code>suggest_branch_step</code>	66
6.2.2.7. <code>bpmnJsonExpertPrompt</code> — <i>prompt</i> de sistema auxiliar	67

TABLA DE CONTENIDOS

6.2.3. Renderizado programático del XML BPMN	67
6.2.4. Prompts de refinamiento conversacional	72
6.2.4.1. buildRefinementSystemPrompt	73
6.2.4.2. buildRefinementMessage	73
6.2.5. Prompt de análisis PERVAL	74
6.2.5.1. buildPervalSystemPrompt	74
6.2.5.2. buildPervalUserMessage	75
6.2.6. Robustez ante respuestas imprecisas del LLM	75
6.2.6.1. parseLLMJson	75
6.2.6.2. cleanXML	76
6.2.7. Funciones auxiliares	77
6.2.7.1. effectiveLanes	77
6.2.7.2. elementLabel	77
7. Ejecución y funcionamiento del sistema	79
7.1. Interfaz general del sistema	79
7.2. Generación iterativa de modelos BPMN	79
7.2.1. Paso 1 — Descripción del proceso	80
7.2.2. Paso 2 — Identificación de participantes y estructura	80
7.2.3. Paso 3 — Identificación del inicio del proceso	81
7.2.4. Paso 4 — Definición del flujo paso a paso	81
7.2.5. Paso 5 — Diagrama BPMN generado	82
7.2.6. Alternativa: generación completa del diagrama	82
7.3. Edición conversacional del diagrama	84
7.4. Análisis de valor percibido PERVAL	84
7.4.1. Configuración del análisis	84
7.4.2. Resultados del análisis	84
7.5. Entrada por voz	86
7.6. Soporte multilingüe	86
7.7. Despliegue en producción	88
Bibliografía	92
Anexo I	93
Anexo II	95

Capítulo 1

Introducción

En el contexto actual de la transformación digital, el modelado de procesos de negocio se ha convertido en una herramienta esencial para las organizaciones que desean comprender, documentar y mejorar la forma en que entregan valor a sus clientes. El estándar *Business Process Model and Notation* (BPMN) es ampliamente utilizado para representar estos procesos de forma gráfica y estructurada. Sin embargo, la creación manual de modelos BPMN requiere conocimientos especializados y supone un esfuerzo considerable, especialmente cuando se trata de procesos complejos o cuando se desea incorporar un análisis del valor percibido por el cliente.

En este contexto, los modelos de lenguaje de gran escala (*Large Language Models*, LLMs) ofrecen nuevas posibilidades para automatizar la generación y el análisis de modelos de procesos. Su capacidad para interpretar descripciones en lenguaje natural y generar artefactos estructurados los convierte en aliados prometedores para asistir a analistas y profesionales en la construcción de modelos BPMN.

No obstante, su aplicación al modelado de procesos de negocio mediante BPMN continúa siendo un área emergente de investigación. Estudios recientes han mostrado resultados alentadores, pero también evidencian desafíos relacionados con la calidad semántica, la validez de los modelos generados y la necesidad de evaluaciones empíricas sistemáticas que permitan comparar el desempeño de los LLMs con el de analistas humanos expertos [1, 2].

Este Trabajo de Fin de Máster (TFM) aborda el desarrollo de una extensión del editor de código abierto bpmn.io que, mediante un servidor intermedio basado en el protocolo Model Context Protocol (MCP), integra un LLM para la generación y análisis automático de modelos BPMN a partir de descripciones textuales o conversacionales. La herramienta implementa un paradigma de *vibe modeling* guiado e híbrido (*guided and hybrid vibe modeling*), en el que el usuario construye y refina el modelo de forma iterativa mediante instrucciones en lenguaje natural aplicando una estrategia de prompting iterativo con validación humana en el bucle (*Iterative prompting with human-in-the-loop validation*) y puede además modificar directamente el diagrama en la interfaz de bpmn.io; el XML del modelo actúa como estado compartido entre el usuario y el LLM. La herramienta permite también identificar los valores asociados a la percepción del cliente según la clasificación PERVAL. Para validar su aceptación por parte de los usuarios, se ha llevado a cabo una encuesta basada en el modelo de aceptación de la tecnología (TAM) con un grupo de aproximadamente quince

usuarios.

1.1. Motivación

La motivación de este trabajo surge de la observación de un doble desafío en el ámbito del análisis de procesos de negocio orientados al valor. Por un lado, la construcción de modelos BPMN de calidad es una tarea técnicamente exigente que requiere conocimiento del estándar, comprensión del dominio y experiencia en análisis de procesos. Por otro lado, los procesos de comercio electrónico están diseñados para entregar valor al cliente mediante atributos como la calidad, el precio y los elementos emocionales y sociales, pero no siempre el valor modelado por la organización corresponde al valor percibido por los usuarios durante la interacción real con el proceso.

Este desalineamiento entre el valor diseñado y el valor percibido puede tener un impacto directo en la experiencia del cliente y en la eficacia del proceso. Sin embargo, su identificación y corrección requieren de un análisis sistemático que, en la práctica, suele ser manual y costoso en tiempo y recursos.

Los avances recientes en LLMs ofrecen una oportunidad para abordar estos desafíos de forma más eficiente. Su capacidad para comprender lenguaje natural y generar artefactos estructurados los convierte en una alternativa prometedora para asistir en la generación de modelos BPMN y en la identificación del valor para el cliente.

A pesar del creciente interés por la aplicación de los LLMs en tareas de Ingeniería del Software, son todavía escasos los estudios que analizan su utilidad en el modelado de procesos orientado al valor. Este vacío en la literatura justifica la relevancia académica de esta investigación y la necesidad de estudios, aunque sea de carácter preliminar, que aporten evidencia sobre el desempeño de estos modelos en dicho contexto.

La realización de este TFM ha permitido aplicar de forma práctica los conocimientos adquiridos durante el máster universitario, en especial en las áreas de ingeniería del software, modelado de procesos y tecnologías de inteligencia artificial.

1.2. Objetivos

El objetivo principal de este trabajo es diseñar, implementar y evaluar una herramienta basada en LLMs capaz de generar y analizar modelos BPMN de forma automática a partir de descripciones textuales o conversacionales, integrada como extensión sobre el editor bpmn.io mediante una arquitectura basada en el protocolo MCP, y evaluar la aceptación de la herramienta por parte de los usuarios mediante una encuesta basada en el modelo de aceptación de la tecnología (TAM).

A partir de este objetivo principal, se definen los siguientes objetivos secundarios:

- diseñar e implementar una extensión funcional sobre bpmn.io que permita la generación iterativa de modelos BPMN mediante interacción en lenguaje natural con un LLM,
- incorporar en la herramienta la capacidad de identificar y clasificar los valores asociados al cliente según la clasificación PERVAL,

- diseñar y llevar a cabo una encuesta basada en el modelo TAM con un grupo de aproximadamente quince usuarios para evaluar la utilidad percibida, la facilidad de uso percibida y la intención de uso del sistema,
- analizar los resultados de la encuesta TAM y evaluar el grado de aceptación de la herramienta por parte de los usuarios, y
- extraer conclusiones sobre la viabilidad y las limitaciones del uso de LLMs para el modelado de procesos de negocio orientado al valor.

1.3. Impacto Esperado

Este trabajo aspira a generar las siguientes contribuciones:

- **Contribución científica:** proporcionar evidencia sobre la aceptación y utilidad percibida de los LLMs aplicados al modelado de procesos orientado al valor, evaluada mediante el modelo TAM.
- **Contribución tecnológica:** desarrollar una implementación funcional integrada en bpmn.io.

Este proyecto se alinea con los siguientes Objetivos de Desarrollo Sostenible (ODS):

- **ODS 9: industria, innovación e infraestructura.** El uso de LLMs para la automatización del modelado de procesos contribuye al avance de soluciones tecnológicas innovadoras aplicadas a la ingeniería del software y a la mejora de las herramientas de análisis empresarial.
- **ODS 12: producción y consumo responsables.** Al facilitar la identificación del valor percibido por el cliente en los procesos de comercio electrónico, la herramienta desarrollada contribuye a un diseño de procesos más eficiente y ajustado a las necesidades reales del usuario, reduciendo el desperdicio de recursos en actividades que no generan valor.

1.4. Metodología

Desde una perspectiva metodológica, el trabajo combina una estrategia de desarrollo de software basada en Scrum con una evaluación de la aceptación del sistema por parte de los usuarios mediante el modelo TAM (Technology Acceptance Model). La investigación puede caracterizarse como un estudio de diseño y evaluación tecnológica, en el que se desarrolla un artefacto software y posteriormente se evalúa su aceptación mediante una encuesta TAM realizada con un grupo de aproximadamente quince usuarios.

La metodología seguida en este trabajo se basa en enfoques ágiles de desarrollo de software, concretamente en el marco de trabajo *Scrum*, con el objetivo de aplicar una planificación iterativa e incremental. Esta metodología permite adaptarse de forma continua a los requisitos funcionales y técnicos durante el desarrollo, resultando especialmente adecuada para un proyecto que integra un LLM en una herramienta de modelado y que requiere ciclos frecuentes de experimentación y mejora continua.

Scrum es una de las metodologías ágiles más utilizadas en el desarrollo de software. Se basa en un proceso iterativo e incremental organizado en ciclos de trabajo deno-

minados *sprints*, en los que el equipo desarrolla y entrega versiones funcionales del producto a partir de requisitos priorizados. Este enfoque permite una adaptación continua a los cambios y una mayor alineación con las necesidades del cliente. Además, Scrum favorece la entrega progresiva de funcionalidades, la reducción de riesgos, la mejora de la productividad y la calidad del producto, así como una coordinación más eficiente entre el equipo de desarrollo y el cliente [3].

Durante la fase inicial del proyecto se llevó a cabo una etapa de diseño conceptual centrada en el modelado del sistema antes de iniciar el desarrollo. Para ello, se definieron los principales actores implicados (el usuario, la extensión sobre bpmn.io y el proveedor de LLM) y se elaboraron diagramas de casos de uso e Historias de Usuario (*User Stories*, US), utilizados para identificar los requisitos funcionales y organizar su implementación. Además, se diseñaron diagramas de interacción entre los distintos componentes del sistema, como el panel conversacional, el servidor MCP y el módulo de análisis PERVAL, los cuales fueron refinados progresivamente durante el desarrollo para ajustarse a la arquitectura final del sistema.

El trabajo se organizó en tareas distribuidas en *sprints* cortos, siguiendo una metodología ágil e incremental. Este enfoque permitió avanzar de forma progresiva en el diseño, desarrollo e integración de las distintas funcionalidades del sistema, facilitando además la adaptación continua a nuevos requisitos y mejoras detectadas durante el proyecto. Finalmente, se llevó a cabo una validación de la propuesta mediante una encuesta basada en el modelo TAM, en la que un grupo de usuarios evaluó la utilidad percibida, la facilidad de uso y la intención de uso del sistema. representativo.

1.5. Estructura

La estructura de este TFM ha sido diseñada para guiar al lector a lo largo del proceso de investigación y desarrollo llevado a cabo. A continuación, se describe el contenido de cada capítulo y los anexos.

- **Introducción:** en este capítulo se contextualiza el proyecto, explicando la motivación que lo origina, los objetivos planteados y el impacto esperado del trabajo. También se describe la metodología de investigación seguida, la estructura del documento y las convenciones tipográficas adoptadas.
- **Fundamentos:** en este capítulo se introducen los conceptos teóricos y tecnológicos necesarios para comprender el resto del trabajo, incluyendo los modelos de lenguaje de gran escala (LLMs) y su funcionamiento, los paradigmas de prompting iterativo con validación humana en el bucle (*Iterative prompting with human-in-the-loop validation*) y vibe modeling guiado e híbrido, aplicados al modelado de procesos, el estándar BPMN y su notación básica, la clasificación PERVAL para el análisis del valor percibido por el cliente, y el protocolo MCP como mecanismo de integración entre la extensión y los proveedores de LLM.
- **Estado del arte:** en este capítulo se revisa la literatura sobre generación automática de modelos de procesos mediante IA, especialmente enfoques conversacionales y basados en LLMs, identificando las lagunas existentes y las oportunidades de investigación que motivan la propuesta de este TFM.
- **Análisis del problema:** en este capítulo se analizan los retos de la generación automática de modelos BPMN mediante LLMs y del análisis del valor del cliente

con PERVAL. Se identifican los requisitos funcionales y no funcionales del sistema, se presentan bocetos de la interacción prevista entre el usuario, la extensión sobre bpmn.io y el LLM, se justifican las decisiones tecnológicas adoptadas y se presenta la planificación del trabajo por sprints, considerando además los aspectos legales y éticos del uso de modelos de lenguaje.

- **Diseño de la solución:** en este capítulo se presenta la arquitectura general de la extensión sobre bpmn.io y la organización de sus componentes principales: el módulo de generación de modelos BPMN mediante el LLM y el módulo de análisis de valor PERVAL. Se describen el modelo de datos empleado, los mecanismos de comunicación entre la interfaz, el plugin y la API del LLM, y se analizan los *frameworks*, lenguajes y herramientas utilizados en el desarrollo, evaluando sus ventajas y limitaciones.
- **Desarrollo e implementación:** en este capítulo se presenta la evolución del proyecto siguiendo una metodología ágil basada en *sprints*. Se describen las principales tareas, avances y funcionalidades implementadas en cada iteración, desde la selección tecnológica inicial y la configuración del entorno de desarrollo sobre bpmn.io hasta la integración del LLM, la implementación del módulo de análisis PERVAL y la puesta en marcha completa de la extensión.
- **Ejecución y funcionamiento del sistema:** en este capítulo se explica la instalación, configuración y funcionamiento de la extensión desarrollada sobre bpmn.io. Se describe el flujo desde la entrada en el lenguaje natural hasta la generación y renderizado del modelo BPMN y se presenta un caso práctico para validar la integración del sistema.
- **Validación de la propuesta:** en este capítulo se presenta el diseño y la ejecución del estudio llevado a cabo para evaluar la herramienta, basado en un proceso de compra en línea. Se describe el diseño del cuestionario, el perfil de los participantes y el procedimiento seguido para evaluar la utilidad percibida, la facilidad de uso percibida y la intención de uso del sistema.
- **Resultados:** en este capítulo se presentan y analizan los resultados del estudio empírico, evaluando la aceptación general de la herramienta y extrayendo conclusiones sobre su utilidad y facilidad de uso según la percepción de los usuarios.
- **Conclusiones y trabajo futuro:** en este capítulo se valora el grado de consecución de los objetivos, se discuten las limitaciones de la herramienta y se proponen líneas de trabajo futuro.
- **Anexos:** contienen información complementaria al cuerpo principal del documento, incluyendo material de apoyo al estudio empírico, datos adicionales de la evaluación y otros recursos que facilitan la comprensión y la reproducibilidad del trabajo realizado.

1.6. Convenciones

En este TFM se adoptará una convención específica para el marcado de términos en lenguas extranjeras. La primera vez que aparezca un término técnico o una palabra que no pertenezca al idioma principal del documento, se destacará en cursiva.

Posteriormente, podrá utilizarse sin cursiva si ya ha sido introducido previamente.

Capítulo 2

Fundamentos

En este capítulo se introducen los conceptos teóricos y tecnológicos necesarios para comprender el resto del trabajo: los modelos de lenguaje de gran escala (LLMs) y su funcionamiento, los paradigmas de prompting iterativo con validación humana y vibe modeling aplicados al modelado de procesos, el estándar BPMN y su notación básica, la clasificación PERVAL para el análisis del valor percibido por el cliente y el protocolo MCP como mecanismo de integración entre la extensión desarrollada y los proveedores LLM.

2.1. Modelos de lenguaje de gran escala (LLMs)

Los modelos de lenguaje de gran escala (*Large Language Models*, LLMs) constituyen una categoría de modelos fundacionales (*foundation models*) basados en aprendizaje profundo y entrenados sobre grandes volúmenes de datos textuales. Su objetivo es aprender representaciones estadísticas del lenguaje que permitan predecir y generar secuencias de texto coherentes, así como realizar diversas tareas de procesamiento del lenguaje natural a partir de instrucciones expresadas en lenguaje natural [16].

La mayoría de los LLMs actuales se basan en la arquitectura *transformer*, propuesta por Vaswani et al. [8], que sustituye las redes neuronales recurrentes empleadas tradicionalmente en el procesamiento del lenguaje natural por un mecanismo de *self-attention* (auto-atención). Este mecanismo permite al modelo ponderar, para cada palabra o fragmento de texto (*token*) de una secuencia, la relevancia del resto de los elementos de dicha secuencia, facilitando así la capturar eficiente y paralelizable de dependencias de largo alcance entre palabras.

El entrenamiento de estos modelos se organiza habitualmente en dos fases. En primer lugar, una fase de *pre-training* (preentrenamiento), en la que el modelo aprende a predecir el siguiente *token* de una secuencia a partir de grandes corpus de texto no etiquetado. Mediante este proceso, el modelo desarrolla representaciones internas que codifican regularidades estadísticas del lenguaje, hechos y patrones de razonamiento presentes en los datos de entrenamiento. En segundo lugar, opcionalmente, puede llevarse a cabo una fase de *fine-tuning* (ajuste fino), en la que el modelo se especializa en tareas concretas mediante conjuntos de datos más reducidos y específicos.

2.1. Modelos de lenguaje de gran escala (LLMs)

Diversos estudios han mostrado que, a medida que aumenta la escala de estos modelos, emergen capacidades que no se manifiestan de forma evidente en modelos más pequeños, tales como la resolución de nuevas tareas a partir de unos pocos ejemplos (*few-shot learning*), la adaptación a dominios específicos y la capacidad de realizar razonamientos complejos utilizando únicamente la información proporcionada en el contexto de entrada (*in-context learning*) [9, 17]. En particular, Brown et al. [9] demostraron que modelos de gran escala pueden resolver tareas para las que no han sido entrenados explícitamente, utilizando únicamente ejemplos e instrucciones incluidos en el propio *prompt*.

La forma habitual de interactuar con un LLM consiste en proporcionarle una instrucción en lenguaje natural, denominada *prompt*, que describe la tarea a realizar y, opcionalmente, el formato esperado de la respuesta. La calidad y precisión de la salida generada depende en gran medida de cómo se formula dicho *prompt*, así como del contexto y de las instrucciones adicionales proporcionadas al modelo, dando lugar a la disciplina conocida como *prompt engineering*.

A pesar de sus capacidades, los LLMs también presentan limitaciones importantes. Entre ellas destacan la generación de información incorrecta o no fundamentada (*hallucinations*), la sensibilidad a pequeñas variaciones en la formulación de las instrucciones y la dificultad para garantizar la corrección semántica de los artefactos generados [18, 19]. Estas limitaciones resultan especialmente relevantes cuando los modelos se utilizan en tareas de ingeniería del software y modelado de procesos de negocio, en las que la precisión y la consistencia de los artefactos producidos constituyen requisitos fundamentales.

Estas capacidades y limitaciones son especialmente relevantes para el presente TFM. Por una parte, los LLMs ofrecen la posibilidad de interpretar descripciones de procesos de negocio expresadas en lenguaje natural y genere, a partir de ellas, representaciones estructuradas, como modelos BPMN. Por otra parte, su naturaleza conversacional permite mantener un diálogo iterativo con el usuario para refinar progresivamente dichos modelos a partir de nuevas instrucciones y restricciones. La aplicación de estas capacidades al ámbito de la gestión de procesos de negocio ha sido objeto de creciente atención en investigaciones recientes [1, 2, 4, 6, 5, 7], cuyo principales avances y limitaciones se analizan con mayor detalle en el capítulo dedicado al Estado del Arte.

Una estrategia relevante en el contexto de este TFM es la **estrategia de prompting iterativo con validación humana en el bucle** (*Iterative prompting with human-in-the-loop validation*). Este enfoque consiste en emplear el LLM no como un generador de respuestas únicas y definitivas, sino como un colaborador dentro de un ciclo iterativo de generación y refinamiento. En cada iteración, el usuario evalúa el artefacto generado, identifica posibles errores o carencias y proporciona instrucciones adicionales para guiar la mejora progresiva del resultado. Este ciclo se repite hasta que el artefacto satisface los criterios de calidad y corrección esperados, convirtiendo al usuario en un validador activo del proceso de generación.

Esta estrategia se adecúa para tareas de modelado de procesos, en las que la complejidad del dominio y la ambigüedad del lenguaje natural dificultan la obtención de resultados correctos y completos en una única iteración. Además, el prompting iterativo soluciona parcialmente las limitaciones propias de los LLMs, como la generación de información incorrecta o semánticamente incoherente, ya que la intervención del

usuario actúa como mecanismo de corrección continua durante el proceso de generación.

Muy relacionado con el prompting iterativo, el concepto de *vibe modeling* surge por analogía con el término *vibe coding*, acuñado por Andrej Karpathy en 2025 para describir un paradigma de desarrollo de software en el que el programador delega la generación del código en un LLM mediante instrucciones en lenguaje natural, limitando su intervención a la guía de alto nivel y la validación del resultado. De manera análoga, el *vibe modeling* hace referencia a la práctica emergente de construir modelos de procesos de negocio mediante un diálogo iterativo con un LLM, en el que el usuario no diseña el modelo elemento a elemento, sino que expresa su intención a través de descripciones en lenguaje natural y permite que el modelo genere, ajuste y refine el artefacto resultante. El usuario adopta, en este paradigma, el rol de director o validador del proceso de modelado, mientras que el LLM actúa como modelador automático.

El *vibe modeling* puede adoptar distintos niveles de participación humana. En su forma más básica, el usuario describe su proceso y el LLM genera el modelo de forma autónoma. En su forma **guiada**, el LLM conduce al usuario a través de preguntas y refinamientos sucesivos, manteniéndole dentro del ciclo de decisión y aplicando así la estrategia de prompting iterativo con validación humana en el bucle. La herramienta desarrollada en este TFM implementa una variante más avanzada: el **vibe modeling guiado e híbrido** (*guided and hybrid vibe modeling*). Además de los mecanismos de instrucción y validación propios del *vibe modeling* guiado, la herramienta permite al usuario modificar directamente el diagrama BPMN en la interfaz de bpmn.io, sin necesidad de expresar dichas modificaciones como instrucciones en lenguaje natural. Dado que tanto el LLM como el usuario acceden y modifican el mismo artefacto XML subyacente —que actúa como estado compartido del modelo—, el proceso de modelado integra de forma bidireccional la edición humana directa y la generación automática asistida por el LLM. Esta característica diferencia la propuesta de la mayoría de los trabajos existentes en la literatura, que contemplan únicamente la interacción a través de instrucciones textuales, sin posibilidad de edición directa del artefacto por parte del usuario.

Este enfoque reduce la barrera de acceso al modelado de procesos, ya que no exige que el usuario domine en profundidad la notación BPMN ni las herramientas de modelado convencionales. Al mismo tiempo, plantea preguntas sobre la calidad, la corrección y la trazabilidad de los modelos generados, así como sobre el grado de supervisión humana necesario para garantizar que los resultados sean adecuados para los fines perseguidos. La herramienta desarrollada en este TFM se inscribe en este paradigma, al permitir al usuario construir y refinar de forma iterativa modelos BPMN a partir de descripciones en lenguaje natural.

2.2. BPMN: notación y modelado de procesos de negocio

La Gestión de Procesos de Negocio (*Business Process Management, BPM*) se ha consolidado como una disciplina fundamental para las organizaciones que buscan mejorar su eficiencia operativa, incrementar la calidad de sus servicios y apoyar iniciativas de transformación digital. En este contexto, la modelización de procesos constituye una actividad esencial para comprender, documentar, analizar y rediseñar la forma en que

2.2. BPMN: notación y modelado de procesos de negocio

las organizaciones crean y entregan valor [11, 20].

BPMN proporciona una notación común y estandarizada que facilita la comunicación entre analistas de negocio, expertos del dominio, desarrolladores de sistemas y demás partes interesadas involucradas en el diseño y ejecución de los procesos [10, 11]. Su objetivo principal es ofrecer una notación gráfica fácilmente entendible por todos los usuarios con distintos perfiles técnicos y de negocio, al tiempo que proporciona un nivel de expresividad suficiente para modelar procesos de diferente complejidad [11].

A pesar de su amplia adopción en entornos académicos e industriales, la construcción manual de modelos BPMN continúa siendo una actividad cognitivamente exigente. La elaboración de modelos de calidad requiere conocimientos especializados sobre la notación, comprensión del dominio del problema y experiencia en modelado de procesos. Asimismo, la complejidad de los modelos puede aumentar considerablemente cuando intervienen múltiples participantes, excepciones, rutas alternativas de ejecución o elevados niveles de detalle [11, 21].

Un diagrama BPMN (*Business Process Diagram*, BPD) se construye a partir de cuatro categorías básicas de elementos:

- **Objetos de flujo** (*flow objects*): constituyen los elementos principales del proceso e incluyen los *events* (eventos), representados mediante círculos y utilizados para indicar algo que sucede durante la ejecución del proceso (eventos de inicio, intermedios o de fin); las *activities* (actividades), representadas mediante rectángulos de esquinas redondeadas y utilizadas para representar el trabajo realizado; y los *gateways* (compuertas), representados mediante rombos y encargadas de controlar la divergencia y convergencia del flujo de ejecución, por ejemplo mediante compuertas exclusivas, paralelas o rutas inclusivas.
- **Objetos de conexión** (*connecting objects*): incluyen los *sequence flows* (flujos de secuencia), que indican el orden de ejecución de las actividades; los *message flows* (flujos de mensaje), que representan el intercambio de información entre participantes; y las *associations* (asociaciones), que vinculan artefactos a otros elementos del diagrama.
- **Carriles** (*swimlanes*): comprenden los *pools* y *lanes*, que permiten organizar gráficamente las actividades de acuerdo con el participante, la organización o el rol responsable de su ejecución.
- **Artefactos** (*artifacts*): incluyen elementos adicionales, como los *data objects* (objetos de datos), los *groups* (grupos) y las *annotations* (anotaciones), que aportan información complementaria sin afectar el comportamiento del flujo de proceso.

Además de su representación gráfica, la especificación BPMN 2.0 define un formato de intercambio basado en XML que permite representar, almacenar e intercambiar modelos de proceso entre distintas herramientas de modelado [10]. Esta característica es fundamental para el presente TFM, ya que el LLM integrado en la extensión genera y modifica los modelos de proceso directamente en este formato XML, que posteriormente es interpretado y renderizado por bpmn.io para su visualización y edición dentro del entorno de modelado.

Esta característica resulta particularmente relevante para el presente TFM. La solución desarrollada utiliza una instancia local de bpmn.io como entorno de modelado y aprovecha la representación XML de BPMN como mecanismo de intercambio entre

el LLM y el editor gráfico. En particular, el modelo de lenguaje genera propuestas de modelos de proceso expresadas en XML BPMN, las cuales son posteriormente interpretadas y renderizadas por bpmn.io para su visualización, edición y refinamiento iterativo dentro del entorno de modelado.

2.3. La clasificación PERVAL

El concepto de valor percibido ha sido ampliamente estudiado en las áreas de marketing, comportamiento del consumidor y gestión de servicios, debido a su importancia para comprender cómo los clientes evalúan productos, servicios y experiencias de consumo. En términos generales, el valor percibido puede entenderse como la valoración global que realiza un cliente sobre la utilidad de un producto o servicio, considerando el equilibrio entre los beneficios obtenidos y los sacrificios realizados para acceder a él, tales como costes económicos, esfuerzo, tiempo o riesgos percibidos [22].

Entre las distintas aproximaciones propuestas en la literatura, la clasificación PERVAL (*Perceived Value*) fue propuesta por Sweeney y Soutar [12] como una escala multidimensional destinada a medir el valor percibido por el cliente en relación con un producto y servicio. A diferencia de enfoques unidimensionales centrados exclusivamente en aspectos económicos o funcionales, PERVAL asume que la percepción de valor constituye un fenómeno complejo y multidimensional, en el que intervienen simultáneamente componentes funcionales, emocionales, sociales y económicos. Esta perspectiva ha favorecido su amplia utilización en investigaciones relacionadas con el comportamiento del consumidor, la evaluación de servicios y el análisis de experiencias de usuario.

La escala PERVAL identifica cuatro dimensiones principales del valor percibido:

- **Valor emocional** (*emotional value*): utilidad derivada de los sentimientos o estados afectivos que el producto o servicio genera en el cliente, tales como satisfacción, tranquilidad, diversión o confianza.
- **Valor social** (*social value*): utilidad derivada de la capacidad del producto o servicio para mejorar la autoimagen, el estatus o el reconocimiento social percibido por el cliente.
- **Valor de calidad/rendimiento** (*quality/performance value*): utilidad derivada de la calidad percibida, la fiabilidad y el rendimiento esperado del producto o servicio en relación con las necesidades y expectativas del usuario.
- **Valor del precio** (*price/value for money*): utilidad derivada de la percepción de que los beneficios obtenidos justifican el coste económico asumido, es decir, la sensación de obtener un retorno adecuado por el dinero invertido.

La naturaleza multidimensional de PERVAL resulta especialmente adecuada para el análisis de procesos de negocio orientados al cliente. En muchos contextos organizacionales, los procesos no solo buscan alcanzar objetivos operativos de eficiencia y productividad, sino también generar experiencias que aporten valor a los usuarios finales. Sin embargo, el valor diseñado por una organización no siempre coincide con el valor efectivamente percibido durante la interacción real con el proceso.

En el contexto de este TFM, la clasificación PERVAL se emplea como marco conceptual para analizar las actividades de un proceso de negocio modelado en BPMN e

2.4. El protocolo *Model Context Protocol* (MCP)

identificar de qué manera dichas actividades contribuyen a la generación de las distintas dimensiones de valor percibido desde la perspectiva del cliente. Este análisis permite detectar posibles desalineamientos entre el valor que la organización pretende ofrecer y el valor que efectivamente percibe el usuario durante su interacción con el proceso.

La incorporación de PERVAL en el presente trabajo adquiere una relevancia particular, ya que permite complementar la generación automática de modelos BPMN mediante LLMs con una perspectiva explícitamente orientada al cliente. De esta manera, el análisis no se limita únicamente a la representación estructural del proceso, sino que también considera las implicaciones que las diferentes actividades y decisiones del proceso pueden tener sobre la experiencia de valor del usuario final.

2.4. El protocolo *Model Context Protocol* (MCP)

El *Model Context Protocol* (MCP) es un protocolo abierto propuesto por Anthropic en 2024 con el objetivo de estandarizar la comunicación entre aplicaciones de usuario y modelos de lenguaje de gran escala, facilitando la integración de herramientas externas, fuentes de datos y lógica de aplicación en el flujo de trabajo del LLM [14]. Su adopción por parte de la industria ha sido progresiva desde su publicación, constituyendo hoy una referencia en el diseño de arquitecturas de integración de LLMs. La figura 2.1 ilustra la diferencia entre la integración directa del LLM con cada herramienta y la integración estandarizada a través de este protocolo.

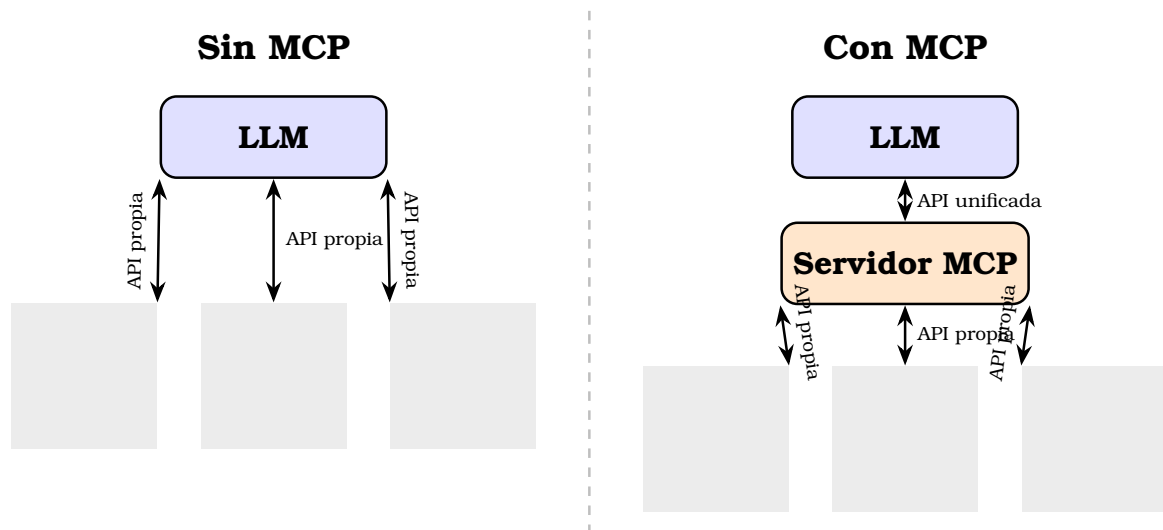


Figura 2.1: Comparación entre la integración directa del LLM con herramientas externas mediante APIs propias (izquierda) y la integración estandarizada a través del protocolo MCP (derecha).

La arquitectura MCP se organiza en torno a tres componentes principales:

- **Host o cliente MCP:** la aplicación que desea hacer uso de las capacidades del LLM. En el presente TFM, el cliente es la extensión desarrollada sobre bpmn.io, responsable de enviar las instrucciones del usuario al servidor MCP e integrar en la interfaz de modelado las respuestas generadas por el modelo.

- **Servidor MCP:** el componente intermedio que actúa como puente entre el cliente y el proveedor del LLM. El servidor recibe las solicitudes del cliente, construye el contexto adecuado para el modelo, gestiona las llamadas a la API del LLM y devuelve los resultados al cliente. En este TFM, el servidor MCP está implementado como un servicio Node.js desplegado en Render.com.
- **Herramientas (tools):** funciones con esquemas de entrada y salida definidos de forma estructurada (habitualmente mediante JSON Schema o Zod) que el LLM puede invocar durante el procesamiento de una solicitud. Cada herramienta tiene un nombre, una descripción y un conjunto de parámetros que permiten al modelo determinar cuándo y cómo utilizarla.

El flujo de comunicación básico del protocolo funciona del siguiente modo: el cliente envía una solicitud al servidor MCP con el mensaje del usuario; el servidor transmite la solicitud al proveedor de LLM junto con la lista de herramientas disponibles; el LLM genera una respuesta, que puede incluir la invocación de una o varias herramientas; el servidor ejecuta las herramientas indicadas, obtiene los resultados y los incorpora al contexto del modelo para que elabore la respuesta definitiva; finalmente, el resultado se devuelve al cliente.

Esta arquitectura ofrece varias ventajas para el presente TFM. En primer lugar, desacopla la interfaz de usuario de los detalles de la API del LLM, lo que facilita el cambio de proveedor de modelo sin modificar la lógica del cliente. En segundo lugar, centraliza en el servidor la gestión de herramientas, el manejo de errores y la seguridad de las credenciales, evitando que estas queden expuestas en el código del cliente. En tercer lugar, la estandarización del protocolo facilita la interoperabilidad entre distintas herramientas y proveedores de LLM, contribuyendo a la portabilidad y extensibilidad de la solución desarrollada.

En la solución implementada en este TFM, el servidor MCP expone herramientas específicas para la generación y edición de modelos BPMN en formato XML y para el análisis PERVAL de los procesos modelados. La extensión sobre bpmn.io actúa como cliente MCP, transmitiendo las instrucciones del usuario al servidor y recibiendo los modelos generados por el LLM para su renderización y visualización en el entorno de modelado.

Capítulo 3

Estado del arte

En los siguientes subapartados se lleva a cabo un análisis de los trabajos de investigación más relevantes en el ámbito de la generación automática de modelos de procesos de negocio mediante inteligencia artificial, con especial atención a los enfoques basados en modelos de lenguaje de gran escala (*Large Language Models*, LLMs). Se examinan cuatro contribuciones representativas que abordan, desde distintas perspectivas, el problema de construir modelos BPMN a partir de lenguaje natural, incluyendo enfoques conversacionales e interactivos. A partir de este análisis, se identifica el conjunto de lagunas que el presente TFM se propone cubrir y se expone la propuesta que lo diferencia de los trabajos previos.

3.1. Análisis de trabajos relacionados

La revisión de la literatura en torno a la generación automatizada de modelos BPMN mediante LLMs pone de manifiesto un campo en rápida evolución, en el que los avances más significativos se han producido a partir de 2023, coincidiendo con la generalización del acceso a modelos como GPT-3.5 y GPT-4. A continuación se presentan los cuatro trabajos más directamente relacionados con el presente TFM.

3.1.1. LLMs en la gestión de procesos de negocio

Grohs et al. [4] presentaron uno de los primeros trabajos que evalúan de forma sistemática el uso de los LLMs en tareas propias de *Business Process Management* (BPM). Los autores analizaron la capacidad de GPT-4 para realizar distintas actividades relacionadas con la gestión de procesos de negocio a partir de información textual, entre ellas la generación de modelos BPMN, la extracción de modelos declarativos basados en restricciones y la clasificación de tareas según su idoneidad para la automatización mediante *Robotic Process Automation* (RPA). Para ello, compraron el rendimiento del modelo con enfoques específicos desarrollados previamente para cada una de las tareas.

Los resultados mostraron que GPT-4 es capaz de interpretar descripciones textuales de procesos y generar representaciones estructuradas con un nivel de calidad comparable e incluso superior al de algunas soluciones especializadas. Además, el estudio pone de manifiesto que un único modelo de propósito general puede aplicarse a diferentes problemas de BPM mediante el uso de instrucciones adecuadas, evitando la

necesidad de desarrollar herramientas independientes para cada tarea. Este trabajo supone un avance relevante dentro de la investigación en BPM, ya que demuestra que los LLMs pueden actuar como herramientas versátiles para el análisis y modelado de procesos de negocio, sentando las bases para el desarrollo de nuevas aplicaciones apoyadas en IA generativa.

Sin embargo, el trabajo se centra fundamentalmente en la evaluación de las capacidades de los LLMs de forma aislada, sin proporcionar una herramienta integrada ni una interfaz que permita al usuario interactuar con el modelo de manera iterativa y conversacional. Tampoco contempla la integración de los artefactos generados en entornos de modelado estándar como bpmn.io, ni aborda el análisis del valor percibido por el cliente dentro de los procesos modelados.

3.1.2. Generación automática de modelos BPMN 2.0 desde descripciones textuales

Hörner et al. [5] presentan BPMNGen, un marco conversacional basado en LLMs para la generación automática de modelos BPMN 2.0 a partir de descripciones en lenguaje natural. La propuesta combina una arquitectura cliente-servidor con un asistente basado en GPT, capaz de interpretar descripciones textuales de procesos y transformarlas en representaciones estructuradas que posteriormente se convierten en modelos BPMN válidos. Además de la generación inicial de diagramas, el sistema permite modificar y refinar los modelos mediante nuevas instrucciones en lenguaje natural, facilitando un proceso de modelado más accesible para usuarios con distintos niveles de experiencia. El trabajo incluye una evaluación empírica centrada en la calidad semántica de los modelos generados, así como en su comprensión y carga cognitiva percibida por los usuarios, comparando los resultados obtenidos con modelos creados por expertos.

No obstante, aunque BPMNGen incorpora mecanismos para la modificación iterativa de modelos mediante instrucciones en lenguaje natural, la evaluación realizada por los autores se centra principalmente en escenarios de generación automática basados en una única descripción textual. Asimismo, la propuesta está orientada a la generación y evaluación de modelos BPMN desde una perspectiva de calidad semántica y comprensión por parte de los usuarios, sin profundizar en aspectos relacionados con el análisis del valor aportado por las actividades del proceso o la identificación de actividades con y sin valor añadido. Estas limitaciones abren oportunidades para desarrollar herramientas que amplíen las capacidades de modelado incorporando nuevas perspectivas de análisis y soporte a la toma de decisiones.

Este trabajo es especialmente relevante para el presente TFM porque aborda de forma explícita los retos de la generación de BPMN válido desde lenguaje natural, y proporciona un vocabulario y unas métricas de evaluación que resultan de utilidad para valorar la calidad de los modelos generados. La identificación de los desafíos abiertos en este campo sirve de motivación directa para las decisiones de diseño adoptadas en la extensión desarrollada, tales como el uso de un esquema de validación XML y la estrategia de regeneración iterativa ante errores sintácticos.

3.1.3. IA generativa y modelado conversacional de procesos

Klievtsova et al. [6] investigaron el potencial de los LLMs para automatizar distintas tareas relacionadas con el modelado de procesos de negocio a partir de descripciones textuales. En su trabajo presentan ConverMod, una propuesta orientada a la extracción automática de actividades y relaciones de control de flujo desde textos descriptivos, así como a la generación de modelos BPMN utilizando modelos como GPT-3 y GPT-4. También se evalúan diferentes estrategias de *prompting* y representaciones intermedias con el objetivo de determinar en qué medida los LLMs son capaces de capturar la información contenida en una descripción de proceso y transformarla en un modelo estructurado.

Los resultados obtenidos muestran que GPT-4 alcanza los mejores niveles de precisión, exhaustividad y similitud respecto a modelos de referencia contruidos por expertos, evidenciando así el potencial de los LLMs como apoyo a las tareas de modelado de procesos.

No obstante, este trabajo se centra en la evaluación experimental de la capacidad de los LLMs para generar modelos de proceso a partir de descripciones textuales, sin integrarse directamente en una herramienta de modelado BPMN ampliamente utilizada. Aunque los modelos generados pueden visualizarse mediante representaciones gráficas derivadas, el trabajo no contempla la interacción continua entre usuario y modelo dentro de un entorno de modelado real. Esta limitación abre la puerta al desarrollo de soluciones que integren capacidades conversacionales directamente en editores BPMN, permitiendo no solo generar, si no también refinarlos.

Este trabajo presenta una estrecha relación con la motivación del presente TFM, ya que demuestra que los modelos de lenguaje pueden reducir la complejidad asociada a la creación de modelos BPMN a partir de especificaciones expresadas en lenguaje natural. Asimismo, pone de manifiesto que las capacidades de comprensión semántica de los LLMs permiten identificar actividades, relaciones y estructuras de control relevantes para la construcción automática de modelos de proceso. Estas conclusiones refuerzan la hipótesis de que la inteligencia artificial generativa puede facilitar el acceso al modelado de procesos a usuarios con conocimientos limitados sobre los formalismos propios de BPMN.

3.1.4. BPMN-Chatbot: modelado de procesos mediante interfaz conversacional

Köpke y Safan [7] presentaron en el marco de la conferencia BPM 2024 el *BPMN-Chatbot*, un sistema que combina una interfaz de mensajería instantánea con la capacidad de generar y visualizar diagramas BPMN como resultado de la conversación. El sistema utiliza un LLM como motor de generación de modelos y muestra el diagrama resultante directamente en la interfaz de chat, de forma que el usuario puede inspeccionar el proceso modelado sin abandonar el entorno conversacional. La propuesta persigue reducir la barrera de entrada al modelado formal de procesos, permitiendo que analistas con distintos niveles de experiencia puedan construir diagramas BPMN de forma rápida e intuitiva.

Este trabajo es el más cercano al enfoque del presente TFM en cuanto a la combinación de una interfaz conversacional con la generación de diagramas BPMN. La arquitectura del BPMN-Chatbot ilustra cómo los LLMs pueden integrarse en un flujo

3.2. Síntesis del estado del arte y oportunidades de investigación

de trabajo de modelado orientado al usuario final y constituye un precedente directo de la línea de investigación en la que se enmarca este trabajo. De hecho, el hecho de que este sistema fuese presentado en BPM 2024, la conferencia internacional de referencia en gestión de procesos de negocio, confirma la relevancia y oportunidad de este enfoque en la comunidad investigadora.

No obstante, el BPMN-Chatbot presenta diferencias significativas respecto al sistema desarrollado en este TFM. En primer lugar, no está integrado ningún editor BPMN existente: los diagramas se visualizan dentro de la interfaz conversacional mediante un componente gráfico específico, pero no pueden editarse ni manipularse directamente en un entorno de modelado estándar. En segundo lugar, el sistema no incorpora ningún mecanismo de análisis del valor percibido por el cliente, que es uno de los objetivos centrales del presente trabajo a través de la clasificación PERVAL.

3.2. Síntesis del estado del arte y oportunidades de investigación

Del análisis de los trabajos presentados en la sección anterior se desprende que, si bien la aplicación de LLMs al modelado de procesos de negocio es un campo en rápido crecimiento, existen lagunas relevantes que el presente TFM se propone cubrir.

Los trabajos revisados sugieren que los LLMs son capaces de generar modelos de proceso de calidad razonable a partir de descripciones en lenguaje natural [4, 5] y que la interacción conversacional constituye una vía prometedora para la participación de usuarios no expertos en el modelado [6, 7]. Sin embargo, ninguno de los trabajos revisados en este estudio propone una integración directa del LLM en un editor BPMN estándar de código abierto, como bpmn.io, que permita al usuario no solo generar el modelo sino también visualizarlo, modificarlo y exportarlo dentro de un entorno profesional de modelado.

Asimismo, la dimensión del valor percibido por el cliente en los procesos de negocio no ha sido abordada en ninguno de los trabajos revisados. La clasificación PERVAL ofrece un marco conceptual para identificar y analizar los atributos de valor que un proceso orientado al cliente genera sobre el usuario, pero su aplicación en el contexto de la generación automática de modelos BPMN mediante LLMs representa, según la revisión bibliográfica realizada, una contribución novedosa del presente TFM.

En síntesis, la revisión realizada pone de manifiesto dos lagunas principales en la literatura: (i) la ausencia de soluciones que integren de forma nativa capacidades conversacionales basadas en LLMs dentro de entornos BPMN ampliamente utilizados por la comunidad profesional, y (ii) la falta de mecanismos que permitan identificar y analizar sistemáticamente el valor percibido por el cliente durante el proceso de modelado.

En consecuencia, el sistema desarrollado en este trabajo se distingue de los trabajos previos en tres aspectos fundamentales: en primer lugar, la integración de un LLM en una instancia local de bpmn.io mediante un panel conversacional, que permite la generación iterativa de modelos BPMN 2.0 dentro de un entorno de modelado estándar; en segundo lugar, la incorporación de un módulo de análisis de valor PERVAL que identifica y clasifica los valores percibidos por el cliente en el proceso modelado; y en tercer lugar, la validación de la propuesta mediante un estudio empírico comparativo

Estado del arte

con un analista experto, con el objetivo de evaluar tanto la calidad de los modelos generados como la consistencia de los valores identificados.

Capítulo 4

Análisis del problema

En este capítulo se aborda el problema definido en el presente Trabajo Fin de Máster, centrado en la integración de LLM en el modelado de procesos de negocio mediante *bpmn.io* y en la incorporación de un módulo de análisis del valor percibido por el cliente basado en la clasificación PERVAL.

Con el propósito de comprender el problema y fundamentar el diseño de la solución propuesta, se realiza, en primer lugar, un modelado funcional del sistema mediante diagramas de actores, de contexto y de casos de uso. Estos modelos permiten delimitar el alcance del sistema, identificar sus principales componentes y especificar las interacciones previstas entre el usuario, la extensión sobre *bpmn.io* y los proveedores de LLM.

Posteriormente, se analiza el marco legal y ético asociado al uso de modelos de lenguaje y al tratamiento de los datos introducidos por los usuarios. Asimismo, se identifican y comparan las principales alternativas tecnológicas consideradas para abordar el problema planteado, justificando las decisiones de diseño finalmente adoptadas.

Finalmente, a partir de dicho análisis realizado, se concreta la solución propuesta mediante la especificación de los requisitos funcionales y no funcionales del sistema, la definición de un conjunto de historias de usuario y la planificación del trabajo realizado, organizada en *sprints* semanales siguiendo una metodología ágil basada en Scrum.

4.1. Modelado funcional del sistema

Con el objetivo de delimitar de forma precisa el alcance funcional de la solución propuesta y comprender las interacciones entre sus diferentes componentes, se ha realizado un modelado funcional del sistema utilizando basado en diagramas UML simplificados. El modelado funcional constituye una actividad fundamental durante el análisis de sistemas, ya que permite identificar los actores involucrados, especificar las responsabilidades de cada componente y describir las funcionalidades que el sistema debe proporcionar para satisfacer las necesidades de los usuarios.

En el contexto del presente TFM, el modelado funcional resulta especialmente relevante debido a la naturaleza híbrida de la solución desarrollada, que integra una extensión sobre *bpmn.io*, un servidor intermedio basado en Model Context Protocol

(MCP) y uno o varios proveedores externos de modelos de LLMs. La interacción coordinada entre estos componentes requiere definir de manera explícita las fronteras del sistema, las responsabilidades de cada subsistema y los flujos de información intercambiados entre ellos.

Para ello, el análisis se estructura en tres niveles complementarios. En primer lugar, se identifican los actores que participan en el sistema y las relaciones existentes entre ellos mediante un diagrama de actores. En segundo lugar, se presentan los diagramas de contexto, cuyo objetivo es delimitar el alcance de cada subsistema y representar las interacciones establecidas con los actores identificados. Finalmente, se describen los diagramas de casos de uso, que permiten especificar las funcionalidades ofrecidas por cada subsistema y la forma en que dichas funcionalidades son utilizadas para satisfacer los objetivos del usuario.

4.1.1. Diagrama de actores

En el sistema propuesto se identifican dos actores principales, representados en la figura 4.1:

- **Usuario:** persona que utiliza la extensión sobre *bpmn.io* para describir, generar, editar y analizar modelos de procesos de negocio en notación BPMN. Puede ser un analista de procesos, un consultor o cualquier perfil que necesite modelar procesos sin un conocimiento profundo de la notación BPMN.
- **LLM:** servicio externo de modelos de lenguaje (*GitHub Models*) utilizado por el sistema para interpretar las descripciones en lenguaje natural del usuario y generar o modificar los modelos BPMN, así como para realizar el análisis de valor PERVAL.

Los componentes internos de la solución son la extensión sobre *bpmn.io* y el servidor MCP y se representan dentro de la frontera del sistema.

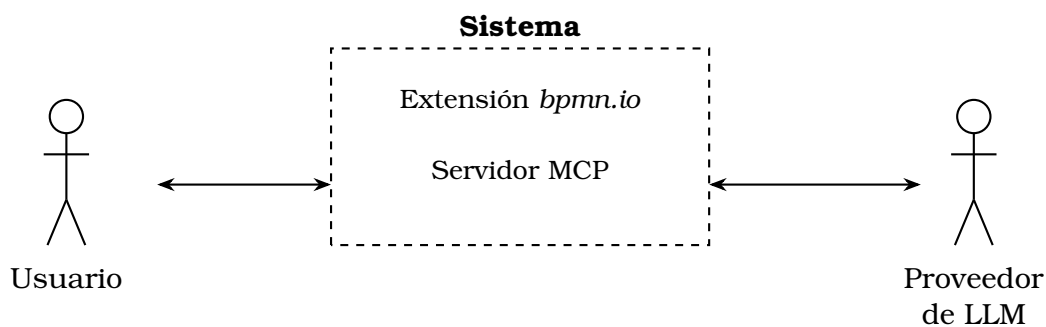


Figura 4.1: Diagrama de actores del sistema.

Para delimitar con claridad el alcance de cada subsistema y sus relaciones con los actores identificados en la sección anterior, se presentan a continuación dos diagramas de contexto: el de la extensión cliente sobre *bpmn.io* y el del servidor MCP que actúa de *backend*.

La figura 4.2 muestra el contexto de la **extensión sobre bpmn.io**, que constituye la interfaz con la que interactúa el **Usuario**. Esta extensión se comunica con el **Servidor**

Análisis del problema

MCP (componente interno del sistema) para delegar en él la generación y edición de modelos BPMN, así como el análisis PERVAL.

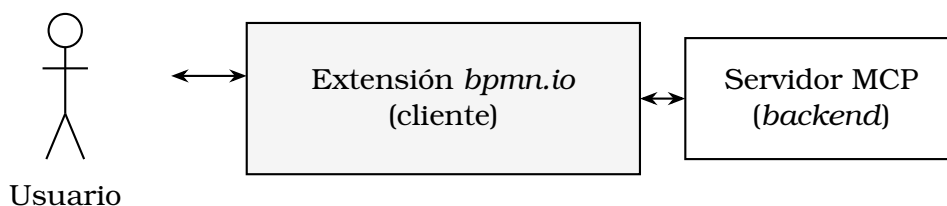


Figura 4.2: Diagrama de contexto de la extensión sobre *bpmn.io* (cliente).

La figura 4.3 muestra el contexto del **servidor MCP**, que actúa como puente entre la extensión cliente y los proveedores de **LLM**. Este componente recibe las peticiones de generación o edición de modelos BPMN procedentes de la extensión, construye los *prompts* adecuados, invoca al LLM seleccionado y devuelve el XML BPMN resultante, validado y, en caso necesario, reparado.



Figura 4.3: Diagrama de contexto del servidor MCP (*backend*).

4.1.2. Diagramas de casos de uso

Una vez delimitado el contexto de cada subsistema, se detallan a continuación los casos de uso correspondientes. La figura 4.4 muestra los casos de uso de la **extensión sobre bpmn.io**, en la que el actor **Usuario** interactúa con las funcionalidades de descripción y edición conversacional del modelo, visualización y exportación del diagrama, configuración del análisis PERVAL y selección de idioma. Tanto la descripción inicial del proceso como la solicitud de modificaciones incluyen («*include*») el caso de uso interno “Renderizar diagrama y actualizar historial”, que el propio sistema ejecuta de forma automática. Si dicho proceso falla, se extiende («*extend*») con el caso de uso “Mostrar estado de carga o error”.

La figura 4.5 muestra los casos de uso del **servidor MCP**. La **extensión bpmn.io** (en su rol de cliente MCP) invoca dos casos de uso principales: “Generar o editar modelo BPMN” y “Analizar valor PERVAL”. El primero incluye la construcción del *prompt* de generación y la invocación al LLM, y se extiende con la limpieza y reparación del XML cuando la respuesta del modelo es incompleta o malformada (función `cleanXML`). De forma análoga, “Analizar valor PERVAL” incluye la construcción de un *prompt* específico y la invocación al LLM para obtener la clasificación de cada tarea o entregable según las dimensiones de valor. En ambos casos, la invocación al LLM puede extenderse con la selección de un modelo de respaldo (*fallback*) cuando el modelo principal no está disponible o devuelve un error.

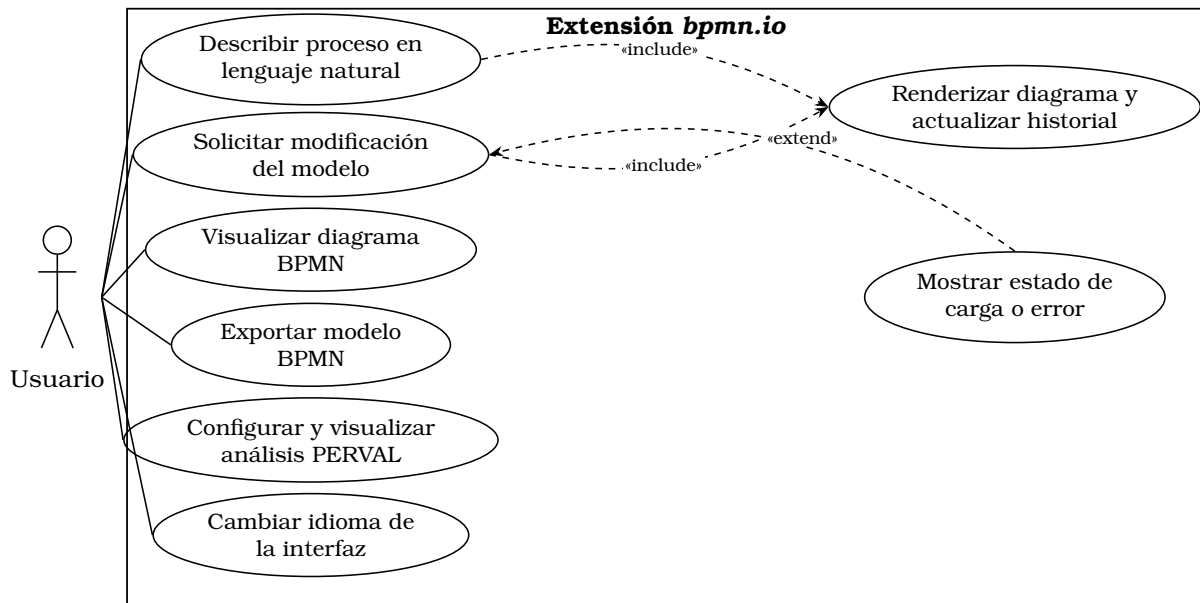


Figura 4.4: Diagrama de casos de uso de la extensión sobre *bpmn.io*.

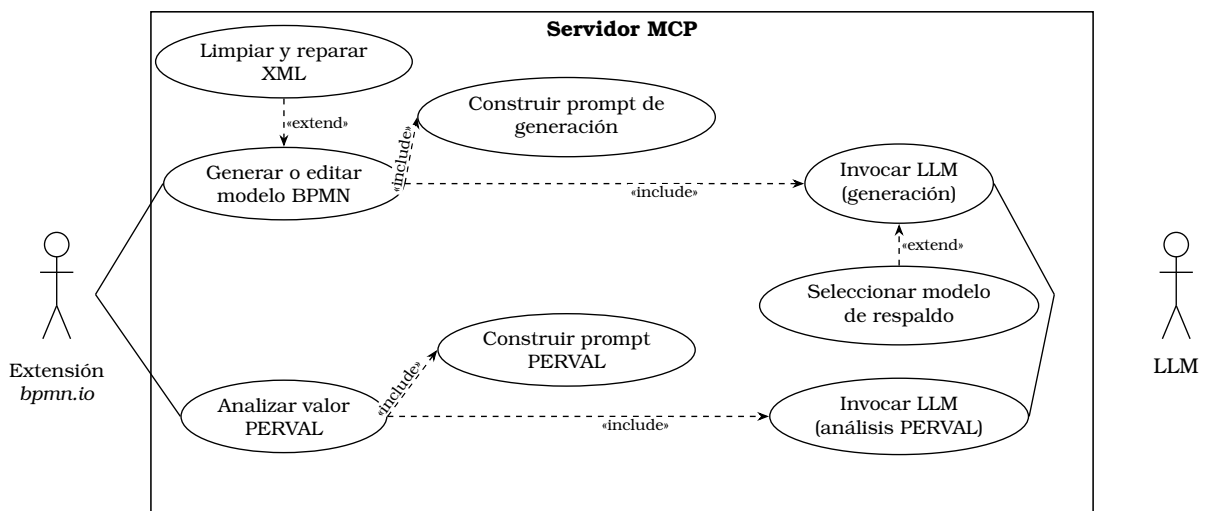


Figura 4.5: Diagrama de casos de uso del servidor MCP

4.2. Análisis del marco legal y ético

El uso de LLMs para generar y modificar modelos de procesos de negocio, así como para analizar el valor percibido por el cliente a partir de información introducida por el usuario, plantea una serie de consideraciones legales y éticas que deben analizarse antes de definir la solución técnica. En esta sección se analizan tres aspectos: la protección de datos personales, la propiedad intelectual y las licencias del software empleado, y las consideraciones éticas derivadas del uso de LLM.

4.2.1. Protección de datos

El sistema desarrollado permite al usuario describir, en lenguaje natural, procesos de negocio que pueden incluir referencias a roles, departamentos o, en casos puntuales,

nombres de personas u organizaciones. Estas descripciones se envían, junto con el contexto de la conversación, al proveedor de LLM seleccionado (*GitHub Models*) para su procesamiento. Por tanto, resulta necesario analizar dicho tratamiento a la luz del Reglamento (UE) 2016/679, de Protección de Datos (RGPD) [13].

En primer lugar, cabe señalar que el sistema no requiere registro ni autenticación de usuarios, por lo que no se almacena de forma persistente ningún dato personal identificativo (nombre, correo electrónico, etc.) en los servidores de la aplicación. El servidor MCP actúa como un componente *stateless*: recibe una petición, la reenvía al proveedor de LLM correspondiente, y devuelve la respuesta sin persistirla en una base de datos propia. El historial de la conversación se mantiene únicamente en el lado del cliente (en el navegador del usuario), por lo que su gestión depende del propio usuario.

No obstante, los textos introducidos por el usuario sí se transmiten a los proveedores externos de LLM, que actúan como encargados del tratamiento conforme a sus respectivas políticas de privacidad. A efectos del RGPD [13], la herramienta informa al usuario de que las descripciones introducidas serán enviadas a un proveedor externo de LLM para su procesamiento, de modo que el uso del sistema presupone la aceptación de estas condiciones de funcionamiento. Esta aproximación resulta más prudente que invocar un consentimiento implícito, ya que el RGPD (artículo 4.11) exige que el consentimiento sea libre, específico, informado e inequívoco, requisitos cuya acreditación formal excede el alcance de este trabajo.

4.2.2. Propiedad intelectual y licencias

El sistema desarrollado se apoya en distintas librerías y servicios de terceros, cuyas licencias deben ser compatibles con el uso académico y, potencialmente, con una futura distribución del código como software de libre acceso. La extensión cliente se construye sobre *bpmn.io* (*bpmn-js* y *diagram-js*), distribuido bajo licencia *bpmn.io license* (basada en MIT con restricciones sobre la eliminación del logotipo), mientras que el servidor MCP se implementa con *Express* y el SDK oficial de *Model Context Protocol* [14], ambos de licencia *MIT*. El uso de los servicios de *GitHub Models* está sujeto a su respectivo términos de servicio y políticas de uso de API, que deben respetarse en cuanto a límites de uso, atribución y restricciones sobre el contenido generado.

Por otra parte, los modelos BPMN generados a partir de las descripciones del usuario son obra derivada de la entrada proporcionada por este, por lo que la propiedad intelectual de dichos modelos corresponde al usuario que los genera, sin que el sistema reclame ningún derecho sobre los resultados producidos.

4.2.3. Consideraciones éticas

El uso de LLM para la generación automática de modelos de procesos de negocio y para el análisis del valor percibido por el cliente conlleva una serie de riesgos éticos que deben tenerse en cuenta en el diseño del sistema:

- **Fiabilidad y supervisión humana:** los LLM pueden generar modelos BPMN incorrectos, incompletos o que no representen fielmente la intención del usuario (fenómeno conocido como *alucinación*). Por ello, el sistema se diseña como una **herramienta de apoyo** que propone modelos editables por el usuario a través

4.3. Identificación y análisis de soluciones posibles

del propio editor visual de *bpmn.io*, en ningún caso como un sistema de decisión autónoma. El usuario mantiene en todo momento el control final sobre el modelo resultante.

- **Transparencia:** el sistema informa al usuario de qué proveedor de LLM se está utilizando en cada momento, de modo que el usuario es consciente de que las respuestas provienen de un sistema de inteligencia artificial generativa y no de un proceso determinista.
- **Sesgos en el análisis PERVAL:** la clasificación del valor percibido (dimensiones emocional, social, de calidad/rendimiento y de precio, según Sweeney y Soutar [12]) realizada por el LLM puede verse afectada por los sesgos presentes en los datos de entrenamiento del modelo. Por este motivo, los resultados del análisis PERVAL se presentan como una **orientación** para el usuario, acompañados de una justificación textual generada por el propio LLM, de modo que el usuario pueda valorar críticamente dicha clasificación.
- **Uso responsable de recursos:** cada interacción con el sistema implica una o varias llamadas a un proveedor externo de LLM, lo que tiene un coste computacional y económico asociado. El diseño del sistema (validación previa de las entradas, mecanismos de *fallback* entre modelos y reparación local del XML generado, en lugar de reintentos completos) busca minimizar el número de llamadas innecesarias al LLM.

4.3. Identificación y análisis de soluciones posibles

Una vez delimitado el alcance funcional del sistema y analizado su marco legal y ético, en esta sección se identifican y comparan las principales alternativas tecnológicas consideradas para abordar el problema planteado. Concretamente, se analizan cuatro decisiones: la selección del editor BPMN sobre el que construir la extensión, la selección del proveedor o proveedores de LLM, la arquitectura de integración entre la extensión y los LLM, y las tecnologías empleadas en el desarrollo del *frontend* y del servidor.

4.3.1. Selección del editor BPMN base

El primer aspecto a decidir es sobre qué herramienta de modelado BPMN construir la extensión que permita la generación y edición de modelos mediante lenguaje natural. Se consideraron las siguientes alternativas:

- **Desarrollo de un plugin para draw.io:** *draw.io* (también conocido como *diagrams.net*, integrado en *Google Drive*) es una herramienta de diagramación ampliamente utilizada que admite la edición de diagramas BPMN. Sin embargo, tras estudiar sus mecanismos de extensión se descartó esta alternativa, ya que la plataforma no permite incorporar *plugins* propios desarrollados por terceros: las únicas extensiones disponibles son las ofrecidas oficialmente por la propia herramienta, sin posibilidad de añadir un panel conversacional ni de manipular el modelo mediante código propio.
- **bpmn.io (bpmn-js):** conjunto de librerías *JavaScript* de código abierto, mantenidas por *Camunda*, que implementan un editor BPMN 2.0 completo ejecutable

en el navegador, junto con la librería de bajo nivel *diagram-js* sobre la que se construye. Ofrece un repositorio de ejemplos (*bpmn-js-examples*) que sirven como punto de partida para extensiones personalizadas, una API completa para manipular el modelo de forma programática (importar/exportar XML, añadir y modificar elementos, gestionar el *canvas*) y un sistema de módulos y extensiones (*additional modules*) que permite añadir funcionalidades sin modificar el núcleo de la librería.

Se optó por ***bpmn-js***, concretamente partiendo del ejemplo *modeler* del repositorio *bpmn-js-examples*, al ser una herramienta de código abierto que sí permite extender el editor con un panel conversacional propio y manipular el modelo BPMN mediante código, por lo que no se plantearon más alternativas. Además, al ser una librería web basada en *JavaScript*, resulta directamente compatible con el resto de tecnologías *frontend* empleadas y con el despliegue de la extensión como sitio estático.

4.3.2. Selección del proveedor de LLM

La generación de modelos BPMN a partir de lenguaje natural y el análisis PERVAL requieren el uso de un LLM capaz de seguir instrucciones complejas y devolver respuestas estructuradas (XML BPMN o JSON). Se consideraron las siguientes alternativas:

- **Acceso directo a la API de OpenAI (ChatGPT) o de Google (Gemini):** ambos proveedores ofrecen modelos de gran capacidad, adecuados tanto para la generación de XML BPMN como para el análisis PERVAL, como muestran trabajos previos sobre generación de modelos BPMN con LLM [1, 5]. Sin embargo, el acceso a sus respectivas API mediante clave propia conlleva un coste económico asociado al volumen de peticiones realizadas, lo que resulta poco adecuado para un proyecto académico que requiere un número elevado de llamadas durante el desarrollo y las pruebas.
- **GitHub Models:** servicio de Microsoft/GitHub que ofrece acceso gratuito (dentro de unos límites de uso) a una amplia variedad de modelos de distintos proveedores (*gpt-4.1*, *gpt-4o*, *gpt-4o-mini*, *gpt-4.1-mini*, *gpt-4.1-nano*, *Phi-4*, entre otros) a través de una API compatible con el formato de *OpenAI*.

Se optó por **GitHub Models** como proveedor principal de LLM, al ofrecer un plan gratuito que permite realizar un número elevado de peticiones durante el desarrollo y las pruebas sin coste económico, además de dar acceso a varios modelos de distintos proveedores a través de una única API compatible con el formato de *OpenAI*. Dentro de los modelos disponibles, se seleccionó ***gpt-4.1*** como modelo principal, por ser el más capaz de la familia GPT disponible en *GitHub Models* en el momento del desarrollo. Para mejorar la resiliencia del sistema, se implementa además un mecanismo de **selección automática de modelo de respaldo** (*fallback*) que, ante errores o límites de uso del modelo activo, prueba secuencialmente con los siguientes modelos de la lista (*gpt-4o*, *gpt-4o-mini*, *gpt-4.1-mini*, *gpt-4.1-nano* y *Phi-4*), de más a menos capaz, lanzando un error únicamente si fallan todos.

4.3.3. Arquitectura de integración: extensión directa frente a servidor MCP

Otra decisión relevante es cómo conectar la extensión sobre *bpmn.io*, que se ejecuta en el navegador del usuario, con los proveedores de LLM. Se consideraron dos alternativas principales:

- **Llamadas directas desde el cliente al proveedor de LLM:** la extensión, ejecutándose enteramente en el navegador, invocaría directamente la API del proveedor de LLM. Esta alternativa es la más sencilla de implementar, pero presenta un problema crítico de seguridad: para realizar dichas llamadas sería necesario incluir las claves de API (*tokens*) directamente en el código *JavaScript* del cliente, quedando expuestas a cualquier usuario que inspeccione el código fuente de la página. Además, dificultaría la incorporación de lógica adicional (validación, reparación de XML, selección de modelo de respaldo) sin duplicarla en el cliente.
- **Servidor intermedio mediante el protocolo MCP (*Model Context Protocol*):** se introduce un servidor *backend* (Node.js/Express) que expone un conjunto de herramientas (*tools*) siguiendo la especificación de *Model Context Protocol* [14], encargado de recibir las peticiones de la extensión, construir los *prompts*, invocar al LLM correspondiente (con su clave de API almacenada de forma segura como variable de entorno del servidor) y devolver el resultado procesado (XML BPMN validado y, en su caso, reparado, o el resultado del análisis PERVAL).

Se optó por la segunda alternativa: un **servidor MCP** como intermediario entre la extensión cliente y los proveedores de LLM. Esta decisión se justifica por varios motivos:

1. **Seguridad:** las claves de API de los proveedores de LLM (`GITHUB_TOKEN`) se almacenan exclusivamente como variables de entorno del servidor, sin exponerse en ningún momento al cliente.
2. **Centralización de la lógica:** la construcción de *prompts*, el *parsing* robusto de las respuestas del LLM, la reparación de XML BPMN truncado o malformado y el mecanismo de selección de modelo de respaldo (con *fallback* entre los modelos de `LLM_MODELS`) se implementan una única vez en el servidor, en lugar de duplicarse en el cliente.
3. **Extensibilidad:** el protocolo MCP define un formato estandarizado de herramientas (*tools*) con esquemas de entrada y salida validados mediante *Zod*, lo que facilita añadir nuevas funcionalidades –como el módulo de análisis PERVAL como nuevas herramientas del mismo servidor, sin modificar la arquitectura general.
4. **Despliegue independiente:** al tratarse de dos componentes independientes (extensión estática y servidor MCP), pueden desplegarse por separado en este caso, como dos servicios de *Render* (`tfm-bpmn-frontend` y `tfm-bpmn-mcp-server`) y escalarse o sustituirse de forma independiente.

4.3.4. Tecnologías para el desarrollo del *frontend* y del servidor

Por último, se analizaron las tecnologías concretas a emplear tanto en la extensión cliente como en el servidor MCP:

- **Empaquetado del cliente:** dado que el ejemplo `modeler` de *bpmn-js-examples* ya utiliza *Webpack* como herramienta de empaquetado, se mantuvo esta elección para minimizar los cambios necesarios sobre la estructura del proyecto y aprovechar la configuración existente para la carga de estilos, iconos y módulos de *bpmn-js*.
- **Interfaz del panel conversacional:** se valoró el uso de un *framework* de componentes (p. ej. *React* o *Vue*) frente a la integración del panel directamente con *JavaScript* y *jQuery* sobre el DOM existente. Dado que el ejemplo base de *bpmn-js-examples* no utiliza ningún *framework* de este tipo, y que el panel conversacional es un componente relativamente sencillo (lista de mensajes, campo de entrada, indicadores de estado), se optó por **JavaScript y jQuery**, evitando así introducir una dependencia adicional de gran tamaño y manteniendo la coherencia con el resto del ejemplo.
- **Servidor MCP:** se eligió **Node.js con Express** para implementar el servidor MCP, dado que es el entorno de ejecución natural para el SDK oficial de *Model Context Protocol* [14], y porque permite compartir el mismo lenguaje (*JavaScript*) entre cliente y servidor. La validación de los esquemas de entrada y salida de cada herramienta MCP se realiza mediante **Zod**, que se integra de forma nativa con el SDK de MCP.
- **Internacionalización:** para dar soporte a usuarios de habla española e inglesa, la interfaz y los *prompts* enviados al LLM se parametrizan según el idioma seleccionado (*es/en*), de modo que tanto los textos de la interfaz como las respuestas del LLM (incluidas las justificaciones del análisis PERVAL) se generen en el idioma elegido por el usuario.
- **Pruebas:** se incorporó **Playwright** para la realización de pruebas *end-to-end* sobre la extensión, permitiendo automatizar la verificación de los flujos principales (generación de un modelo a partir de una descripción, edición conversacional, análisis PERVAL) en un navegador real.
- **Despliegue:** se seleccionó **Render** como plataforma de despliegue, por ofrecer un nivel gratuito adecuado para un proyecto académico y por permitir definir, mediante un único archivo `render.yaml`, los dos servicios necesarios: un sitio estático para la extensión (`tfm-bpmn-frontend`) y un servicio *web* de tipo Node.js para el servidor MCP (`tfm-bpmn-mcp-server`).

Como resultado del análisis realizado en las secciones anteriores, se seleccionaron las alternativas tecnológicas que mejor satisfacen los requisitos funcionales y no funcionales identificados para el desarrollo de la solución propuesta. En todos los casos, la elección se fundamentó en criterios relacionados con la interoperabilidad, la extensibilidad, la mantenibilidad y la adecuación a los objetivos del presente TFM. La Tabla 4.1 resume las principales decisiones de diseño adoptadas, las alternativas consideradas durante el análisis y los criterios que justificaron cada elección. Sobre esta base, en la siguiente sección se presenta la solución propuesta.

Las decisiones sintetizadas en la Tabla 4.1 constituyen la base tecnológica sobre la que se construye la solución desarrollada en este trabajo. A partir de ellas se derivan los requisitos funcionales y no funcionales que se presentan en la siguiente sección, los cuales concretan el comportamiento esperado del sistema y las características de calidad que deberá satisfacer.

4.3. Identificación y análisis de soluciones posibles

Cuadro 4.1: Resumen de las principales decisiones de diseño adoptadas.

Decisión de diseño	Alternativas consideradas	Criterio principal de evaluación	Decisión adoptada
Editor de modelado BPMN	draw.io, Camunda Modeler, bpmn.io	Capacidad de extensión, edición interactiva, acceso al modelo BPMN y disponibilidad de API.	bpmn.io
Proveedor de LLM	API de OpenAI (ChatGPT), API de Google (Gemini), GitHub Models	Disponibilidad de plan gratuito, acceso a múltiples modelos y API unificada compatible con el formato OpenAI.	GitHub Models (modelo principal gpt-4.1 con mecanismo de <i>fallback</i> automático)
Arquitectura de integración	Llamadas directas del cliente al proveedor de LLM; servidor MCP intermedio (Node.js/Express)	Seguridad de credenciales, centralización de la lógica de construcción de <i>prompts</i> y reparación de XML.	Servidor MCP (Node.js + Express)
Comunicación con el LLM	SDK nativo de cada proveedor (OpenAI SDK, Google AI SDK...); API REST compatible con formato OpenAI	Unificación de llamadas multi-modelo y compatibilidad directa con GitHub Models sin dependencias adicionales.	API REST compatible con OpenAI (punto de acceso único para todos los modelos de GitHub Models)
Tecnologías de desarrollo	React/Vue + Python/Flask; JavaScript + Node.js/Express; otras combinaciones	Coherencia con el ejemplo base de <i>bpmn-js</i> , lenguaje compartido cliente-servidor e integración nativa con el MCP SDK.	JavaScript: cliente (<i>bpmn-js</i> , jQuery, Webpack); servidor (Node.js, Express, Zod)

4.4. Solución propuesta

A partir del modelado funcional y de las decisiones tecnológicas descritas anteriormente, se concreta la solución propuesta mediante la especificación de los requisitos funcionales (tabla 4.2) y no funcionales (tabla 4.3) del sistema, y mediante un conjunto de quince historias de usuario (US01-US15) que detallan el trabajo realizado para satisfacer dichos requisitos.

4.4.1. Requisitos funcionales

La tabla 4.2 recoge los requisitos funcionales (RF) identificados para el sistema.

Código	Descripción
RF-01	El sistema debe permitir al usuario describir un proceso de negocio en lenguaje natural y generar, a partir de dicha descripción, un modelo BPMN 2.0 válido.
RF-02	El sistema debe permitir al usuario solicitar modificaciones sobre un modelo BPMN existente mediante instrucciones en lenguaje natural, conservando el contexto de la conversación.
RF-03	El sistema debe visualizar el modelo BPMN generado o modificado en el lienzo de <i>bpmn.io</i> .
RF-04	El sistema debe permitir exportar el modelo BPMN resultante en formato XML.
RF-05	El sistema debe validar y, en caso de errores menores, reparar automáticamente el XML BPMN devuelto por el LLM antes de cargarlo en el editor.
RF-06	El sistema debe seleccionar automáticamente un modelo de respaldo cuando el modelo principal no esté disponible o devuelva un error.
RF-07	El sistema debe permitir analizar el valor percibido (PERVAL) de las tareas o entregables de un modelo BPMN, según las dimensiones emocional, social, de calidad/rendimiento y de precio [12].
RF-08	El sistema debe permitir seleccionar el ámbito (tareas o entregables) y el actor sobre el que se realiza el análisis PERVAL.
RF-09	El sistema debe presentar visualmente los resultados del análisis PERVAL (resumen por dimensión y valor general), acompañados de una justificación textual.
RF-10	El sistema debe permitir seleccionar el idioma de la interfaz y de las respuestas generadas (español/inglés).
RF-11	El sistema debe informar al usuario del estado de cada petición (cargando, completada, error) en todo momento.

Cuadro 4.2: Requisitos funcionales del sistema.

4.4.2. Requisitos no funcionales

La tabla 4.3 recoge los requisitos no funcionales (RNF) identificados para el sistema.

Código	Descripción
RNF-01	Usabilidad: la interacción con el sistema debe realizarse mediante un panel conversacional integrado en el propio editor de <i>bpmn.io</i> , sin requerir conocimientos previos de la notación BPMN ni de los proveedores de LLM.
RNF-02	Seguridad: las claves de API de los proveedores de LLM deben almacenarse únicamente como variables de entorno del servidor MCP, sin exponerse en ningún momento al cliente.
RNF-03	Tiempo de respuesta: el sistema debe proporcionar realimentación visual inmediata (indicadores de carga) ante cualquier petición al LLM, dado que los tiempos de respuesta de estos servicios pueden oscilar entre varios segundos y, en modelos complejos, más de un minuto.
RNF-04	Extensibilidad: el servidor MCP debe organizarse como un conjunto de herramientas (<i>tools</i>) independientes con esquemas de entrada/salida validados mediante <i>Zod</i> , de modo que puedan añadirse nuevas funcionalidades sin modificar la arquitectura general.
RNF-05	Portabilidad: el sistema debe poder desplegarse en una plataforma <i>cloud</i> (<i>Render</i>) mediante un único fichero de configuración (<i>render.yaml</i>), y ser ejecutable en cualquier entorno compatible con Node.js.
RNF-06	Mantenibilidad: el código debe organizarse de forma modular, separando claramente la extensión cliente, el servidor MCP, la lógica de llamada a los LLM (<i>llm.js</i>) y cada herramienta MCP (<i>tools/</i>).
RNF-07	Conformidad: los modelos BPMN generados o modificados deben generarse en un formato compatible con BPMN 2.0 [10] y susceptible de ser importado por herramientas compatibles.
RNF-08	Privacidad: el sistema debe minimizar los datos enviados a los proveedores de LLM (principio de minimización de datos del RGPD [13]) y no debe almacenar de forma persistente datos personales de los usuarios.

Cuadro 4.3: Requisitos no funcionales del sistema.

4.4.3. Priorización de requisitos. MoSCoW

Con el objetivo de establecer un orden claro de implementación y gestionar los recursos disponibles dentro de los límites de un proyecto académico, se ha aplicado la metodología MoSCoW para priorizar los requisitos identificados en las secciones anteriores. Esta metodología clasifica los requisitos en cuatro categorías según su importancia crítica para el éxito del proyecto:

- **Must have** (debe tener): funcionalidades absolutamente esenciales cuya ausencia impediría que el sistema cumpla su propósito fundamental.
- **Should have** (debería tener): funcionalidades importantes que aportan valor significativo pero cuya ausencia no impide el funcionamiento básico del sistema.
- **Could have** (podría tener): funcionalidades deseables que pueden implementar-

Análisis del problema

se si los recursos lo permiten, sin comprometer los objetivos principales.

- **Won't have** (no tendrá): funcionalidades descartadas explícitamente para la versión actual del proyecto, ya sea por limitaciones de tiempo, recursos o decisiones estratégicas de desarrollo.

Must have

Se consideran imprescindibles aquellos requisitos sin los cuales el sistema no puede cumplir su propósito fundamental: la generación y el análisis iterativo de modelos BPMN mediante LLM.

La generación de modelos BPMN a partir de descripciones en lenguaje natural (RF-01) constituye el núcleo de la propuesta; sin esta funcionalidad, la herramienta pierde su razón de ser. Íntimamente ligada a ella, la modificación iterativa del modelo mediante instrucciones conversacionales (RF-02) es igualmente esencial, ya que materializa el paradigma de *vibe modeling* sobre el que se fundamenta el proyecto e impide que el sistema sea un mero generador estático de diagramas. La visualización del modelo resultante en el lienzo de *bpmn.io* (RF-03) es también imprescindible: sin ella el usuario no puede validar el resultado generado ni interactuar con el editor.

La validación y reparación automática del XML BPMN (RF-05) es un requisito crítico, dado que los LLMs producen con frecuencia XML malformado o truncado; sin este mecanismo, los modelos generados serían inutilizables en la mayoría de los casos. El análisis del valor percibido según la clasificación PERVAL (RF-07) constituye el segundo objetivo central del proyecto académico; su ausencia eliminaría la componente de análisis orientado al cliente que diferencia esta herramienta de otros asistentes de modelado basados en LLM. La presentación visual de los resultados del análisis con justificación textual (RF-09) es igualmente esencial: sin ella el análisis PERVAL no aporta valor interpretable al usuario.

En cuanto a los requisitos no funcionales, la integración del panel conversacional directamente en el entorno de *bpmn.io* sin requerir conocimientos previos de la notación BPMN (RNF-01) es el requisito de usabilidad fundamental que materializa el concepto de *vibe modeling*. El almacenamiento de las claves de API exclusivamente como variables de entorno del servidor MCP (RNF-02) es crítico desde el punto de vista de la seguridad: exponer dichas credenciales en el código del cliente pondría en riesgo las cuentas de los usuarios y comprometería la viabilidad de la herramienta en producción.

Should have

Esta categoría engloba requisitos que aportan valor significativo a la utilidad, la resiliencia y la calidad del sistema, sin ser estrictamente imprescindibles para el funcionamiento básico.

La exportación del modelo BPMN en formato XML (RF-04) incrementa considerablemente la utilidad práctica de la herramienta, al permitir al usuario conservar y reutilizar los modelos fuera del entorno de *bpmn.io*. El mecanismo de selección automática de modelo de respaldo (RF-06) es importante para garantizar la continuidad del servicio ante los límites de uso de la API de *GitHub Models*, especialmente frecuentes durante el desarrollo y las pruebas; sin él, el sistema fallaría con mayor frecuencia

ante picos de peticiones. La configuración del ámbito (tareas o entregables) y del actor sobre el que se realiza el análisis PERVAL (RF-08) es relevante para adaptar el análisis a distintos contextos de proceso, aunque podría implementarse con valores predeterminados en una primera versión. La información del estado de cada petición mediante indicadores visuales (RF-11) mejora notablemente la experiencia de usuario en interacciones con latencias elevadas, habituales en llamadas a LLMs.

Desde el punto de vista técnico, la organización del servidor MCP como conjunto de herramientas independientes con esquemas validados mediante *Zod* (RNF-04) facilita la incorporación de nuevas funcionalidades sin alterar la arquitectura general. La modularidad del código, con separación clara entre el cliente, el servidor MCP, el módulo de llamada al LLM y las herramientas individuales (RNF-06), facilita el mantenimiento y la evolución del sistema. La conformidad de los modelos generados con el estándar BPMN 2.0 (RNF-07) garantiza la interoperabilidad con otras herramientas de modelado, aunque está parcialmente cubierta por el mecanismo de validación de RF-05.

Could have

Los siguientes requisitos añaden valor pero pueden implementarse de forma diferida sin comprometer los objetivos principales del proyecto.

La selección del idioma de la interfaz y de las respuestas generadas (RF-10) amplía el alcance de la herramienta a usuarios de diferentes entornos lingüísticos. Sin embargo, el sistema es plenamente funcional en un único idioma, y su implementación puede postergarse hasta que las funcionalidades principales estén consolidadas. La realimentación visual inmediata ante cualquier petición al LLM —más allá del indicador de carga básico— (RNF-03) mejoraría la percepción de la capacidad de respuesta del sistema, pero no afecta a su comportamiento funcional. La portabilidad del sistema mediante un único fichero de configuración para su despliegue en *Render* (RNF-05) es deseable para las pruebas de aceptación TAM con usuarios reales, aunque durante el desarrollo basta con ejecutar el sistema en local. La gestión explícita de la minimización de datos personales conforme al RGPD (RNF-08) está parcialmente cubierta por el requisito de seguridad de las claves de API (RNF-02); los mecanismos adicionales de privacidad son deseables pero no bloquean el funcionamiento de la herramienta.

Won't have

Esta categoría incluye funcionalidades descartadas de forma explícita para la versión actual del proyecto, ya sea por limitaciones de tiempo y recursos, o por decisiones estratégicas orientadas a validar el concepto fundamental antes de ampliar el alcance.

El soporte para proveedores de LLM adicionales distintos de *GitHub Models* fue considerado durante el análisis de alternativas y descartado (véase §4.3.2): el plan gratuito de *GitHub Models* ofrece acceso a modelos GPT de alta calidad con límites suficientes para los objetivos del proyecto, y añadir proveedores adicionales incrementaría la complejidad del servidor MCP sin aportar valor diferencial en esta primera versión. La persistencia del historial de conversación entre sesiones fue igualmente descartada: el historial se gestiona en memoria del cliente durante la sesión activa, lo cual es suficiente para los ciclos de refinamiento iterativo previstos; una persistencia duradera

Análisis del problema

requeriría una base de datos, lo que excede el alcance del proyecto. Por último, la exportación del diagrama en formatos alternativos (imagen PNG/SVG, PDF) no forma parte de los objetivos del TFM y puede cubrirse mediante las funcionalidades nativas del propio *bpmn.io*.

Resumen por prioridad:

Must (8): RF-01, RF-02, RF-03, RF-05, RF-07, RF-09, RNF-01, RNF-02

Should (7): RF-04, RF-06, RF-08, RF-11, RNF-04, RNF-06, RNF-07

Could (4): RF-10, RNF-03, RNF-05, RNF-08

Won't (3 funcionalidades): integración con otros proveedores de LLM, persistencia del historial entre sesiones, exportación en formatos alternativos

Funcionales	Must	Should	Could	Won't
Generación BPMN (RF-01 a RF-06)	01, 02, 03, 05	04, 06	—	—
Análisis PERVAL (RF-07 a RF-09)	07, 09	08	—	—
Configuración/UX (RF-10 a RF-11)	—	11	10	—

Cuadro 4.4: Resumen de priorización MoSCoW de requisitos funcionales.

No funcionales	Must	Should	Could	Won't
Usabilidad (RNF-01)	01	—	—	—
Seguridad y privacidad (RNF-02, 08)	02	—	08	—
Rendimiento (RNF-03)	—	—	03	—
Arquitectura y calidad (RNF-04 a RNF-07)	—	04, 06, 07	05	—

Cuadro 4.5: Resumen de priorización MoSCoW de requisitos no funcionales.

4.4.4. Historias de usuario

Para concretar el desarrollo del sistema se ha definido un conjunto de quince historias de usuario (US01-US15), siguiendo una metodología ágil basada en *Scrum* [3]. Para cada historia de usuario se indica una breve descripción, una estimación del esfuerzo necesario (en horas) y el desglose en tareas concretas, también estimadas en horas. Estas historias de usuario constituyen la base de la planificación del trabajo por *sprints* presentada a continuación.

US01 – Selección tecnológica y planificación del proyecto

Descripción: como desarrollador, quiero analizar las alternativas tecnológicas disponibles para la generación de modelos BPMN mediante LLM y para el análisis PERVAL, para seleccionar el editor BPMN base, el o los proveedores de LLM, y la arquitectura de integración, y planificar el desarrollo del proyecto por *sprints*.

Estimación: 8h.

Tareas:

- Estudio de trabajos relacionados sobre generación de modelos BPMN mediante LLM (3h).
- Selección del editor BPMN, del proveedor de LLM y de la arquitectura de integración (3h).
- Definición del *backlog* inicial y planificación de *sprints* (2h).

US02 – Configuración del entorno de desarrollo

Descripción: como desarrollador, quiero configurar un entorno de desarrollo a partir del ejemplo `modeler` de `bpmn-js-examples`, para que disponga de un editor BPMN funcional sobre el que construir la extensión.

Estimación: 6h.

Tareas:

- Clonado y configuración inicial del proyecto `modeler` (2h).
- Configuración de *Webpack* y de las dependencias del proyecto (2h).
- Verificación del funcionamiento del editor BPMN base (2h).

US03 – Diseño e implementación del servidor MCP

Descripción: como desarrollador, quiero implementar un servidor *backend* que siga la especificación de *Model Context Protocol* [14], para que actúe como intermediario seguro entre la extensión cliente y los proveedores de LLM.

Estimación: 12h.

Tareas:

- Estudio de la especificación MCP y del *SDK* oficial (3h).
- Implementación del servidor Express con transporte HTTP y registro de herramientas (4h).
- Definición de la estructura de herramientas (*tools*) y de sus esquemas de entrada/salida con *Zod* (3h).
- Pruebas de conexión entre la extensión cliente y el servidor MCP (2h).

US04 – Integración con *GitHub Models*

Descripción: como desarrollador, quiero integrar el servidor MCP con la API de *GitHub Models*, para poder invocar distintos modelos de LLM (`gpt-4.1`, `gpt-4o`, etc.) para la generación de modelos BPMN y el análisis PERVAL.

Estimación: 10h.

Tareas:

- Configuración de credenciales (`GITHUB_TOKEN`) y definición del catálogo de modelos disponibles (2h).
- Implementación de la función de llamada al LLM (`callOpenAI`) (4h).

Análisis del problema

- Implementación del mecanismo de selección de modelo de respaldo ante errores (2h).
- Pruebas de integración con distintos modelos (2h).

US05 – Generación de modelos BPMN a partir de lenguaje natural

Descripción: como usuario, quiero describir un proceso de negocio en lenguaje natural y obtener automáticamente un modelo BPMN 2.0 que lo represente, para poder partir de un primer borrador del modelo sin tener que construirlo manualmente.

Estimación: 14h.

Tareas:

- Diseño del *prompt* de generación, incluyendo el catálogo completo de elementos BPMN soportados (4h).
- Implementación de la herramienta MCP de generación de modelos BPMN (4h).
- Implementación del *parsing* robusto de la respuesta del LLM (`parseLLMJson`) (2h).
- Carga del XML generado en el editor *bpmn-js* (2h).
- Pruebas con descripciones de proceso de distinta complejidad (2h).

US06 – Edición conversacional iterativa del modelo

Descripción: como usuario, quiero poder solicitar modificaciones sobre el modelo BPMN actual mediante mensajes en lenguaje natural, para poder refinar iterativamente el modelo a través de una conversación con el sistema.

Estimación: 12h.

Tareas:

- Diseño del panel conversacional (historial de mensajes y campo de entrada) (3h).
- Implementación de la herramienta MCP de edición, que recibe el XML actual y la instrucción del usuario (4h).
- Gestión del historial de la conversación en el cliente (3h).
- Pruebas de iteraciones sucesivas de modificación sobre un mismo modelo (2h).

US07 – Validación y reparación automática del XML BPMN

Descripción: como usuario, quiero que el sistema detecte y corrija automáticamente los errores habituales en el XML BPMN devuelto por el LLM (etiquetas sin cerrar, XML truncado), para que el modelo se cargue correctamente en el editor incluso cuando la respuesta del LLM no es perfecta.

Estimación: 10h.

Tareas:

- Análisis de los errores más habituales en las respuestas del LLM (truncamientos, comillas sin cerrar, etiquetas incompletas) (3h).
- Implementación de la función de limpieza y reparación de XML (`cleanXML`) (4h).

- Implementación de la detección automática de descripciones completas frente a incrementales (2h).
- Pruebas con respuestas malformadas del LLM (1h).

US08 – Diseño de la herramienta de análisis PERVAL

Descripción: como desarrollador, quiero diseñar una herramienta MCP capaz de analizar el valor percibido por el cliente de las tareas o entregables de un modelo BPMN, según la clasificación de Sweeney y Soutar [12], para que el usuario pueda obtener información sobre el valor que aporta cada elemento del proceso.

Estimación: 10h.

Tareas:

- Estudio de las dimensiones de valor percibido (emocional, social, calidad/rendimiento, precio) (2h).
- Diseño del esquema de entrada/salida de la herramienta `analyzePerval` (3h).
- Diseño del *prompt* de análisis PERVAL, incluyendo el ámbito (tareas/entregables) y el actor (3h).
- Pruebas con modelos BPMN de ejemplo (2h).

US09 – Implementación de la herramienta `analyzePerval`

Descripción: como desarrollador, quiero implementar en el servidor MCP la herramienta de análisis PERVAL diseñada en US08, para que, dado un modelo BPMN y un conjunto de tareas, el sistema devuelva la clasificación por dimensiones, el resumen agregado y el valor general.

Estimación: 10h.

Tareas:

- Implementación de la herramienta MCP `analyzePerval` con su esquema *Zod* (3h).
- Extracción de las tareas y entregables a partir del XML BPMN (3h).
- Cálculo del resumen por dimensión y del valor general a partir de la respuesta del LLM (2h).
- Pruebas de la herramienta con distintos ámbitos y actores (2h).

US10 – Visualización de los resultados del análisis PERVAL

Descripción: como usuario, quiero visualizar de forma clara los resultados del análisis PERVAL (resumen por dimensión, valor general y justificación de cada tarea), para poder interpretar fácilmente el valor percibido de cada elemento del proceso.

Estimación: 10h.

Tareas:

- Diseño de la interfaz de selección de ámbito (tareas/entregables) y actor (2h).
- Implementación de la visualización del resumen por dimensión y del valor general (4h).

Análisis del problema

- Resaltado sobre el diagrama BPMN de los elementos analizados (2h).
- Pruebas de usabilidad de la visualización (2h).

US11 – Diseño e implementación de la interfaz del panel conversacional

Descripción: como usuario, quiero disponer de un panel conversacional integrado en el editor con un diseño visual coherente y agradable, para que la interacción con el sistema resulte intuitiva y profesional.

Estimación: 10h.

Tareas:

- Diseño de la paleta de colores (azul/verde) y selección de tipografía (*Inter*) (2h).
- Implementación de los estilos del panel conversacional, adaptados a distintos tamaños de pantalla (4h).
- Implementación de los indicadores de estado de carga y de error (2h).
- Pruebas de la interfaz en distintos tamaños de pantalla (2h).

US12 – Internacionalización de la interfaz y de los *prompts*

Descripción: como usuario, quiero poder utilizar el sistema en español o en inglés, tanto en la interfaz como en las respuestas generadas por el LLM, para poder trabajar en el idioma de su preferencia.

Estimación: 8h.

Tareas:

- Extracción de los textos de la interfaz a ficheros de idioma (es/en) (3h).
- Parametrización de los *prompts* de generación, edición y análisis PERVAL según el idioma seleccionado (3h).
- Pruebas del sistema completo en ambos idiomas (2h).

US13 – Pruebas *end-to-end* con *Playwright*

Descripción: como desarrollador, quiero disponer de un conjunto de pruebas automatizadas *end-to-end*, para poder verificar que los flujos principales del sistema (generación, edición conversacional y análisis PERVAL) funcionan correctamente en un navegador real.

Estimación: 8h.

Tareas:

- Configuración de *Playwright* sobre el proyecto (2h).
- Implementación de pruebas de generación y edición conversacional de modelos BPMN (3h).
- Implementación de pruebas de la herramienta `analyzePerval` (2h).
- Integración de las pruebas en el flujo de desarrollo (1h).

US14 – Despliegue en *Render* y documentación

Descripción: como desarrollador, quiero desplegar la extensión y el servidor MCP en *Render*, para que el sistema sea accesible públicamente y pueda ser utilizado y evaluado fuera del entorno de desarrollo local.

Estimación: 7h.

Tareas:

- Configuración del fichero `render.yaml` con los dos servicios (`tfm-bpmn-frontend` y `tfm-bpmn-mcp-server`) (2h).
- Despliegue y pruebas de funcionamiento del servidor MCP y de la extensión en *Render* (3h).
- Documentación de las variables de entorno necesarias y del procedimiento de puesta en marcha (2h).

US15 – Validación de la solución con usuarios mediante TAM

Descripción: como desarrollador, quiero validar la solución desarrollada mediante una evaluación con un grupo de aproximadamente quince usuarios, empleando el modelo de aceptación de la tecnología (*Technology Acceptance Model*, TAM), para poder valorar la utilidad percibida, la facilidad de uso percibida y la intención de uso del sistema.

Estimación: 10h.

Tareas:

- Diseño del cuestionario TAM (utilidad percibida, facilidad de uso percibida e intención de uso) adaptado al sistema desarrollado (2h).
- Selección y convocatoria de un grupo de aproximadamente quince participantes (2h).
- Realización de las sesiones de prueba, en las que los participantes utilizan el sistema desplegado y completan el cuestionario (4h).
- Análisis de los resultados del cuestionario y elaboración de conclusiones sobre la aceptación de la solución (2h).

La estimación total del conjunto de historias de usuario asciende a **145 horas**, que se distribuyen en ocho *sprints* semanales tal como se detalla a continuación.

4.5. Plan de trabajo

El desarrollo del sistema se ha planificado siguiendo una metodología ágil basada en *Scrum* [3], organizando el trabajo en **ocho sprints semanales** (Sprint 0 a Sprint 7), desde el 26/05/2026 hasta el 20/07/2026. Cada *sprint* agrupa una o varias de las historias de usuario definidas en la sección 4.4.4, de modo que la suma de las estimaciones de las historias de usuario de cada *sprint* determina su carga de trabajo prevista. De forma paralela al desarrollo de cada *sprint*, se redacta la documentación correspondiente de la memoria del TFM, de modo que, una vez finalizado el Sprint 7,

Análisis del problema

el trabajo restante se centra en completar y revisar las secciones de la memoria que no hayan podido finalizarse durante el desarrollo.

Conviene señalar que la planificación mediante sprints constituye exclusivamente la estrategia adoptada para organizar el desarrollo incremental del artefacto software. La evaluación científica de la propuesta se aborda de forma independiente mediante un estudio empírico de aceptación basado en el TAM, descrito en los capítulos posteriores. De este modo, Scrum se emplea como metodología de desarrollo del software, mientras que la validación de la investigación se fundamenta en un procedimiento empírico específico.

La tabla 4.6 resume la planificación de los ocho *sprints*, indicando para cada uno su rango de fechas, las historias de usuario asignadas y la carga de trabajo estimada.

Sprint	Fechas	Historias de usuario	Carga (h)
Sprint 0	26/05/2026 – 01/06/2026	US01 (Selección tecnológica y planificación), US02 (Configuración del entorno de desarrollo)	14
Sprint 1	02/06/2026 – 08/06/2026	US03 (Servidor MCP), US04 (Integración con <i>GitHub Models</i>)	22
Sprint 2	09/06/2026 – 15/06/2026	US05 (Generación de modelos BPMN a partir de lenguaje natural)	14
Sprint 3	16/06/2026 – 22/06/2026	US06 (Edición conversacional iterativa), US07 (Validación y reparación de XML)	22
Sprint 4	23/06/2026 – 29/06/2026	US08 (Diseño de la herramienta PERVAL), US09 (Implementación de <i>analyzePerval</i>)	20
Sprint 5	30/06/2026 – 06/07/2026	US10 (Visualización de resultados PERVAL), US11 (Interfaz del panel conversacional)	20
Sprint 6	07/07/2026 – 13/07/2026	US12 (Internacionalización), US13 (Pruebas <i>end-to-end</i>)	16
Sprint 7	14/07/2026 – 20/07/2026	US14 (Despliegue en <i>Render</i> y documentación), US15 (Validación con usuarios mediante TAM)	17

Cuadro 4.6: Planificación del trabajo por *sprints*.

Como puede observarse, la carga de trabajo prevista se sitúa en torno a las 18 horas semanales de media (145 horas en total repartidas en 8 *sprints*), con dos *sprints* de menor carga (Sprint 0 y Sprint 2) dedicados, respectivamente, a la planificación inicial del proyecto y a la implementación de la funcionalidad central de generación de modelos BPMN, que por su complejidad se ha mantenido como única historia de usuario del *sprint*. Los *sprints* 1, 3, 4 y 5 concentran la mayor parte del desarrollo de las dos funcionalidades principales del sistema –la generación y edición conversacional de modelos BPMN (*sprints* 1-3) y la herramienta de análisis PERVAL (*sprints* 4-5)–, mientras que los *sprints* 6 y 7 se dedican a pulir la interfaz, dar soporte a la internacionalización, completar las pruebas automatizadas, desplegar el sistema en producción y validar la solución con un grupo de usuarios mediante el modelo TAM.

Capítulo 5

Diseño de la solución

En este capítulo se presenta la arquitectura general del sistema desarrollado en este TFM, detallando la organización de sus componentes principales, los mecanismos de comunicación entre ellos y las decisiones de diseño adoptadas. Asimismo, se describe el modelo de datos empleado y se analizan los lenguajes, *frameworks* y herramientas utilizados en el desarrollo, evaluando su adecuación al contexto del proyecto.

5.1. Arquitectura del sistema

La arquitectura del software hace referencia a la organización estructural de un sistema, compuesta por los distintos componentes que lo integran, sus características observables externamente y las interacciones que existen entre ellos. Esta visión estructurada permite entender cómo se construye, se comunica y evoluciona el sistema a lo largo del tiempo.

La arquitectura presentada en este capítulo constituye el resultado directo de las decisiones de diseño analizadas y justificadas en el capítulo anterior (véase §4.3), donde se identificaron y compararon las principales alternativas tecnológicas. Cada elección estructural responde, por tanto, a un proceso sistemático de análisis y selección de alternativas, y no a decisiones arbitrarias.

La solución desarrollada en este TFM se articula en torno a dos componentes principales: la **extensión sobre bpmn.io**, que actúa como cliente MCP y proporciona la interfaz gráfica de modelado y el panel conversacional; y el **servidor MCP**, que actúa como intermediario entre la extensión y el proveedor de LLM, y que expone entre sus herramientas la capacidad de análisis PERVAL. La figura 5.1 muestra la interacción entre los distintos componentes del sistema.

Como se puede observar, el usuario interactúa con la extensión sobre bpmn.io tanto a través del editor gráfico (modificando directamente el diagrama BPMN) como a través del panel conversacional (expresando instrucciones en lenguaje natural). La extensión se comunica con el servidor MCP mediante peticiones HTTP, y este se encarga de invocar las herramientas adecuadas y gestionar las llamadas al LLM. El XML BPMN del modelo actúa como **estado compartido** entre el usuario y el LLM: ambos pueden leerlo y modificarlo, siendo bpmn.io el punto de renderización y persistencia en sesión. Esta bidireccionalidad es la que define el paradigma de *vibe modeling* guiado e híbrido implementado en este TFM.

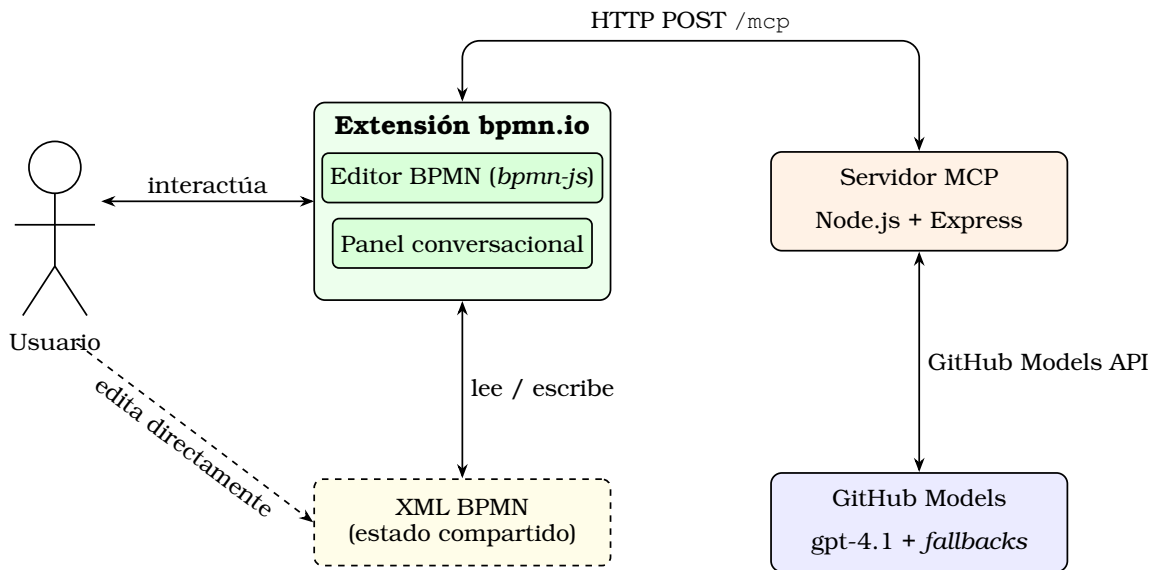


Figura 5.1: Interacción entre los diferentes componentes del sistema.

La adopción de una arquitectura modular y desacoplada favorece la mantenibilidad y la extensibilidad del sistema, ya que cada componente puede modificarse o sustituirse de forma independiente sin afectar al resto.

5.1.1. Justificación de la arquitectura adoptada

La arquitectura adoptada para este TFM responde a una serie de consideraciones técnicas, funcionales y de escalabilidad. A continuación, se justifican las principales decisiones arquitectónicas.

La primera y más determinante decisión ha sido adoptar la **arquitectura MCP** (*Model Context Protocol*) como patrón estructural del sistema. Como se describe en el capítulo de Fundamentos, el protocolo MCP define una separación explícita en tres roles: el *host* o cliente (la extensión sobre bpmn.io), que desea acceder a las capacidades del LLM; el servidor MCP, que expone un conjunto de herramientas con una interfaz estandarizada; y el LLM, que las ejecuta. Esta separación aporta ventajas que justifican su elección frente a una integración directa entre la interfaz de usuario y la API del LLM:

- **Estandarización e interoperabilidad:** el protocolo MCP define un contrato estable entre cliente y servidor independiente del proveedor de LLM. Cambiar de proveedor (p. ej., de GitHub Models a otro) o de modelo no requiere modificar el código de la extensión, únicamente la configuración del servidor.
- **Seguridad:** las credenciales de acceso a la API del LLM residen exclusivamente en el servidor MCP, nunca en el código JavaScript que se ejecuta en el navegador del usuario y que podría ser inspeccionado.
- **Centralización de la lógica de herramientas:** el servidor MCP concentra la definición, validación y ejecución de las nueve herramientas del sistema. Añadir o modificar una herramienta no requiere cambios en la extensión del cliente.
- **Capacidad de extensión:** la arquitectura MCP permite incorporar nuevas he-

herramientas o fuentes de datos (p. ej., acceso a repositorios de modelos BPMN externos) de forma incremental, sin alterar la arquitectura base del sistema.

En segundo lugar, se ha optado por **bpmn.io** como base de la herramienta de modelado. Las dos alternativas barajadas fueron bpmn.io y Draw.io. Draw.io es un editor de diagramas de propósito general, extensible y de código abierto, que incluye soporte para notación BPMN; sin embargo, no garantiza la conformidad estricta con el estándar BPMN 2.0, no expone una API de extensión orientada específicamente al modelado de procesos y su arquitectura no facilita la integración programática del panel conversacional ni la manipulación del XML BPMN desde código externo. bpmn.io, en cambio, está diseñado específicamente para BPMN 2.0, ejecuta íntegramente en el navegador sin instalación adicional, ofrece una arquitectura modular y extensible mediante *plugins* que permite integrar el panel conversacional y la comunicación con el servidor MCP directamente en la interfaz de edición, y es ampliamente adoptada tanto en entornos académicos como profesionales.

En tercer lugar, el servidor MCP se ha implementado en **modo sin estado** (*stateless*): cada petición crea una nueva instancia del servidor MCP y su transporte, que se destruyen al finalizar la respuesta. Este enfoque simplifica la gestión del ciclo de vida de las sesiones para un prototipo en el que la persistencia entre peticiones no es un requisito.

Por último, se ha seleccionado **GitHub Models** como proveedor de LLM por su plan gratuito, que permite un número elevado de peticiones durante el desarrollo y las pruebas, y por la disponibilidad de múltiples modelos a través de una única API compatible con el formato OpenAI. El modelo principal es gpt-4.1, con *fallback* automático hacia gpt-4o, gpt-4o-mini, gpt-4.1-mini, gpt-4.1-nano y Phi-4, garantizando la resiliencia del sistema ante la indisponibilidad puntual de modelos.

5.1.2. Arquitectura detallada por componente

A continuación se describe la estructura y responsabilidades de cada uno de los dos componentes principales del sistema.

5.1.2.1. Extensión sobre bpmn.io (cliente MCP)

La extensión sobre bpmn.io constituye el punto de entrada del usuario al sistema. Se ha desarrollado como una capa adicional sobre la librería de código abierto *bpmn-js*, que proporciona las capacidades nativas de edición y renderizado de diagramas BPMN 2.0. A esta capa base se ha añadido un **panel conversacional** que permite al usuario interactuar con el LLM mediante instrucciones en lenguaje natural, siguiendo la estrategia de prompting iterativo con validación humana en el bucle (*Iterative prompting with human-in-the-loop validation*) descrita en el capítulo de Fundamentos.

Desde el punto de vista del protocolo MCP, la extensión actúa como **cliente MCP** o *host*: es la aplicación que desea hacer uso de las capacidades del LLM y que envía las solicitudes al servidor MCP. La comunicación se realiza mediante peticiones HTTP POST al *endpoint* `/mcp` del servidor, siguiendo el transporte *Streamable HTTP* definido por el SDK de MCP.

El elemento central para el paradigma de *vibe modeling* guiado e híbrido es el **XML BPMN como estado compartido**. La librería bpmn-js mantiene en memoria el XML

del diagrama actual. Este XML puede ser leído y modificado tanto por el usuario —a través de la edición directa del diagrama en la interfaz gráfica— como por el LLM —a través de las herramientas del servidor MCP que generan o modifican el XML y lo devuelven a la extensión para su renderización—. Esta característica bidireccional diferencia la propuesta de la mayoría de los trabajos existentes en la literatura, en los que el usuario se limita a proporcionar instrucciones textuales sin posibilidad de edición directa del artefacto.

La figura 5.2 ilustra la distribución funcional de la extensión en cuatro capas.

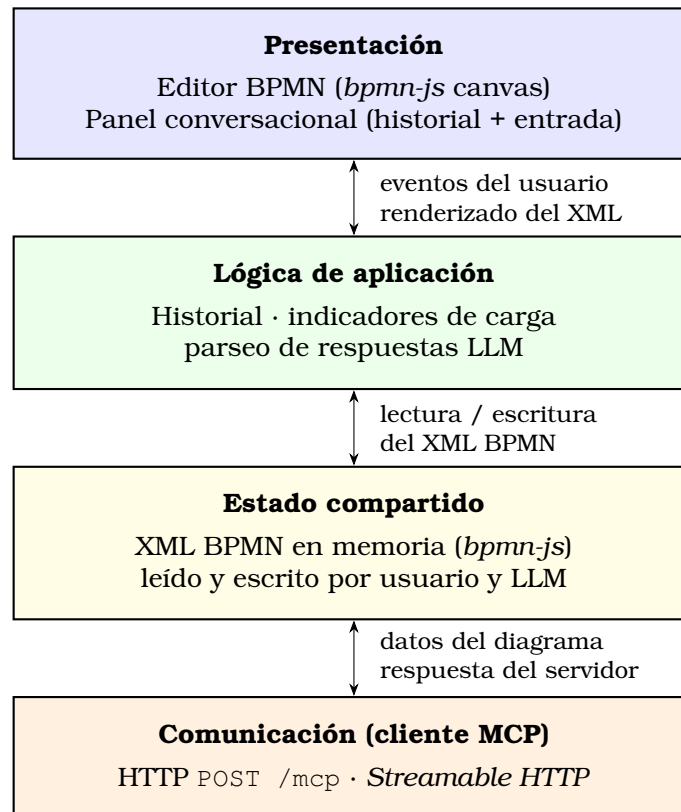


Figura 5.2: Estructura por capas de la extensión sobre bpmn.io.

5.1.2.2. Servidor MCP

El servidor MCP es el componente intermediario del sistema. Está implementado como un servicio **Node.js** con el framework **Express v5** y desplegado en la plataforma en la nube *Render.com*, lo que garantiza su disponibilidad continua con acceso HTTPS.

El servidor expone un único *endpoint* (POST /mcp) y gestiona las peticiones del cliente siguiendo el protocolo MCP en modo *stateless*: por cada petición recibida, se crea una nueva instancia del servidor MCP (*McpServer*) y su transporte (*StreamableHTTPServerTransport*) se conectan, se procesa la petición y ambos se destruyen al finalizar la respuesta. Este diseño elimina la necesidad de gestionar ciclos de vida de sesiones entre peticiones.

El servidor registra **nueve herramientas** MCP, cada una con nombre, descripción y esquema de entrada/salida definido mediante la librería *Zod*. La tabla 5.1 resume las herramientas disponibles y su función en el proceso de modelado.

Herramienta	Función
identify_structure	Analiza la descripción en lenguaje natural del proceso e identifica la organización principal, sus departamentos y los actores externos; devuelve el objeto <code>structure</code> y el XML BPMN inicial (<i>pools/lanes</i> sin pasos).
suggest_flow_start	Propone el evento o eventos de inicio del proceso a partir de la descripción y los departamentos identificados.
suggest_next_step	Propone el siguiente elemento del proceso (tarea, <i>gateway</i> o evento) dado el estado actual del diagrama.
suggest_gateway_branches	Sugiere las ramas de un <i>gateway</i> de decisión y sus condiciones de guarda.
suggest_branch_step	Propone los pasos internos de una rama de un <i>gateway</i> .
modify_structure	Modifica la estructura del proceso (actores externos, departamentos) antes de la generación de pasos.
render_diagram	Genera el XML BPMN 2.0 completo y validado a partir del estado acumulado del diagrama (<code>structure + diagramState</code>).
refine_diagram	Aplica una modificación libre expresada en lenguaje natural sobre el XML BPMN existente y devuelve el XML modificado.
analyze_perval	Analiza el diagrama BPMN según el modelo PERVAL (Sweeney y Soutar, 2001), devolviendo las dimensiones de valor percibido por tarea y un resumen global.

Cuadro 5.1: Herramientas MCP registradas en el servidor.

Cada herramienta realiza una o varias llamadas a la API de GitHub Models con *prompts* de sistema y usuario diseñados específicamente para la tarea correspondiente. El mecanismo de *fallback* automático garantiza la resiliencia del sistema ante errores de cuota o indisponibilidad de modelos individuales.

La figura 5.3 muestra la organización interna del servidor MCP en cuatro capas.

5.1.2.2.1. Herramienta de análisis PERVAL (`analyze_perval`). Entre las nueve herramientas registradas en el servidor destaca `analyze_perval`, que implementa el análisis de valor percibido según el modelo PERVAL (Sweeney y Soutar, 2001). No se trata de un componente independiente, sino de una herramienta más dentro del servidor MCP, cuya lógica reside en el mismo servicio Node.js junto al resto de herramientas de generación y refinamiento de diagramas.

La herramienta recibe como entrada el XML BPMN del diagrama, la lista de tareas o entregas a analizar (extraídas previamente del XML por la extensión), el nombre del actor externo cuyo punto de vista se adopta, el alcance del análisis (entregas al cliente o todas las tareas del proceso) y el idioma de los textos generados. A partir de esta información el LLM aplica el marco PERVAL y devuelve, en formato JSON estructurado, las dimensiones de valor para cada elemento analizado (*Quality, Price, Emotional, Social*), junto con un resumen global por dimensión y una valoración

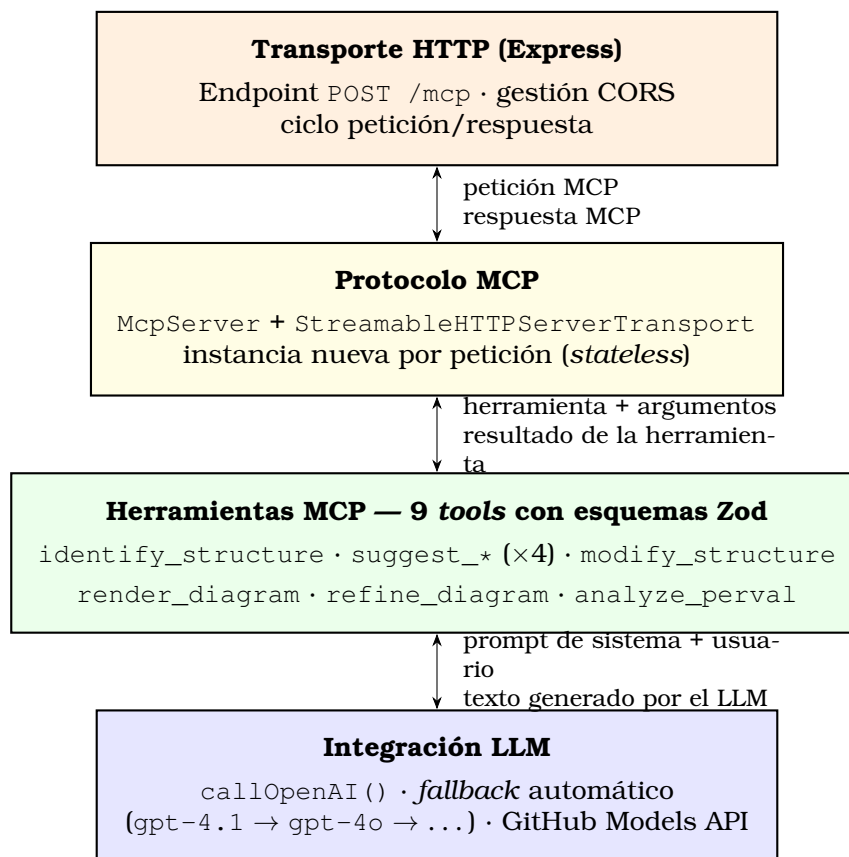


Figura 5.3: Estructura interna del servidor MCP.

general del proceso.

5.2. Diseño detallado

En esta sección se aborda el modelo de datos empleado en el sistema y el flujo de comunicación entre componentes durante las principales operaciones de la herramienta.

5.2.1. Modelo de datos

A diferencia de los sistemas basados en bases de datos relacionales, la herramienta desarrollada en este TFM no emplea persistencia estructurada entre sesiones. El **artefacto central del sistema es el XML BPMN 2.0**, que actúa como estado compartido y transitorio durante la sesión de modelado en el navegador. Este artefacto es gestionado por la librería `bpmn-js` en memoria y transmitido entre los componentes según sea necesario.

Junto al XML BPMN, el sistema maneja internamente los siguientes objetos de datos:

- **Objeto structure:** representa la estructura organizativa del proceso. Contiene los actores externos del proceso (`poolExterno`, array de objetos con nombre y rol), la organización principal con sus departamentos (`poolPrincipal.nombre` y `poolPrincipal.lanes`) y un resumen descriptivo del proceso (`resumen`). Es

el resultado de la herramienta `identify_structure` y el punto de partida para la generación iterativa del diagrama.

- **Objeto `diagramState`:** representa el estado acumulado del diagrama durante la generación paso a paso. Está compuesto por un array de pasos (`steps`), donde cada paso describe un elemento del proceso (tarea, evento o *gateway*) con su posición en el flujo y su departamento. Este objeto, junto con el objeto `structure`, es utilizado por la herramienta `render_diagram` para generar el XML BPMN 2.0 completo.
- **Resultado PERVAL:** estructura JSON devuelta por la herramienta `analyze_perval`. Está compuesta por un array de objetos de análisis por tarea (con campos `nombre`, `dimensiones`, `valor` y `justificacion`), un resumen de las cuatro dimensiones PERVAL (`Quality`, `Price`, `Emotional`, `Social`) y una valoración general del proceso (`valorGeneral`).

La ausencia de persistencia en base de datos es una decisión de diseño coherente con la naturaleza del prototipo y con el modelo de interacción adoptado: el XML BPMN que el usuario ve en el editor es siempre el estado actualizado, independientemente de si la última modificación la realizó el usuario directamente o el LLM a través de una herramienta.

5.2.2. Flujo de comunicación

En esta subsección se describe el flujo de comunicación entre los componentes durante las dos operaciones principales de la herramienta: la generación y refinamiento iterativo de modelos BPMN, y el análisis PERVAL.

5.2.2.1. Generación y refinamiento iterativo de modelos BPMN

La generación de un nuevo modelo BPMN sigue un flujo de comunicación estructurado en múltiples fases, que implementa la estrategia de prompting iterativo con validación humana en el bucle descrita en el capítulo de Fundamentos. El flujo completo se describe a continuación.

El proceso comienza cuando el usuario introduce en el panel conversacional una descripción en lenguaje natural del proceso de negocio a modelar. La extensión envía esta descripción al servidor MCP, que invoca la herramienta `identify_structure`. El LLM analiza la descripción e identifica la organización principal, sus departamentos y los actores externos, devolviendo un objeto `structure` y un XML BPMN inicial con la estructura de *pools* y *lanes*. Este XML se renderiza en el editor `bpmn.io` y se presenta al usuario para su validación o corrección.

Una vez confirmada la estructura, la extensión guía al usuario a través de la definición paso a paso del proceso, invocando sucesivamente las herramientas `suggest_flow_start`, `suggest_next_step`, `suggest_gateway_branches` y `suggest_branch_step` según el tipo de elemento propuesto en cada iteración. En cada paso, el usuario puede aceptar la propuesta del LLM, modificarla o rechazarla antes de continuar con el siguiente elemento. Cuando se han definido todos los pasos del proceso, se invoca la herramienta `render_diagram`, que genera el XML BPMN 2.0 completo y lo transmite a la extensión para su renderización en el editor.

Si el usuario desea realizar modificaciones sobre el diagrama ya generado, puede hacerlo mediante dos mecanismos complementarios que caracterizan el paradigma de *vibe modeling* guiado e híbrido:

1. Mediante una **instrucción en lenguaje natural** a través del panel conversacional, que invoca la herramienta `refine_diagram`. Esta herramienta recibe el XML actual del diagrama junto con la instrucción de modificación, genera el XML modificado mediante el LLM aplicando el cambio solicitado y lo devuelve a la extensión para su renderización.
2. Mediante **edición directa del diagrama** en la interfaz gráfica de bpmn.io, sin necesidad de instrucciones textuales. Las modificaciones realizadas por el usuario quedan reflejadas directamente en el XML del modelo gestionado por bpmn-js, que actúa como estado compartido. En la siguiente interacción conversacional, el servidor MCP recibirá el XML actualizado, garantizando la coherencia entre ambas formas de modificación.

La figura 5.4 ilustra las fases del proceso de generación y refinamiento iterativo de modelos BPMN.

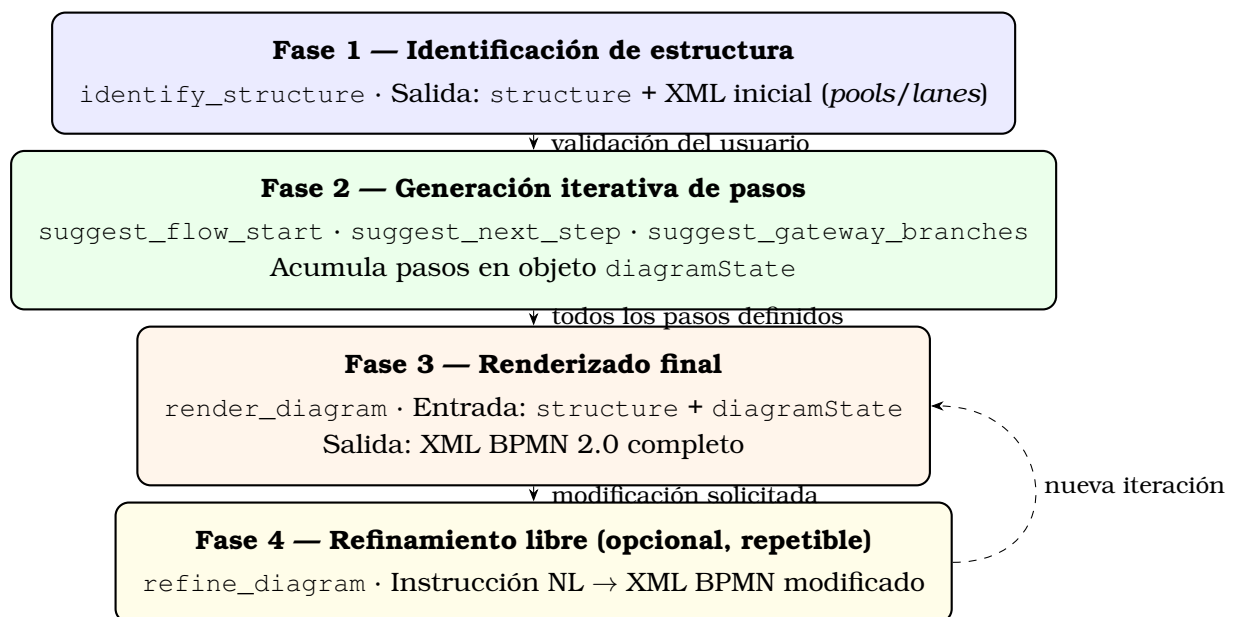


Figura 5.4: Fases del flujo de generación y refinamiento iterativo de modelos BPMN. La fase 4 puede ejecutarse tantas veces como el usuario lo requiera.

5.2.2.2. Análisis PERVAL

El análisis PERVAL puede iniciarse en cualquier momento sobre el diagrama BPMN disponible en el editor. El flujo comienza cuando el usuario solicita el análisis a través del panel conversacional. La extensión lee el XML BPMN actual desde bpmn-js y extrae la lista de tareas o entregas que se desean analizar. A continuación, envía al servidor MCP una solicitud a la herramienta `analyze_perval`, incluyendo el XML, la lista de elementos a analizar, el nombre del actor externo cuyo punto de vista se adopta, el alcance del análisis y el idioma de los textos de salida.

El servidor MCP construye el *prompt* de sistema específico para PERVAL y lo envía al

LLM junto con la información del proceso. El LLM analiza el proceso según el modelo PERVAL (Sweeney y Soutar, 2001) y devuelve un objeto JSON estructurado con las dimensiones de valor percibido para cada elemento analizado. El servidor transmite este resultado a la extensión, que lo presenta al usuario en el panel conversacional de forma estructurada.

La figura 5.5 resume el flujo de la operación de análisis PERVAL.

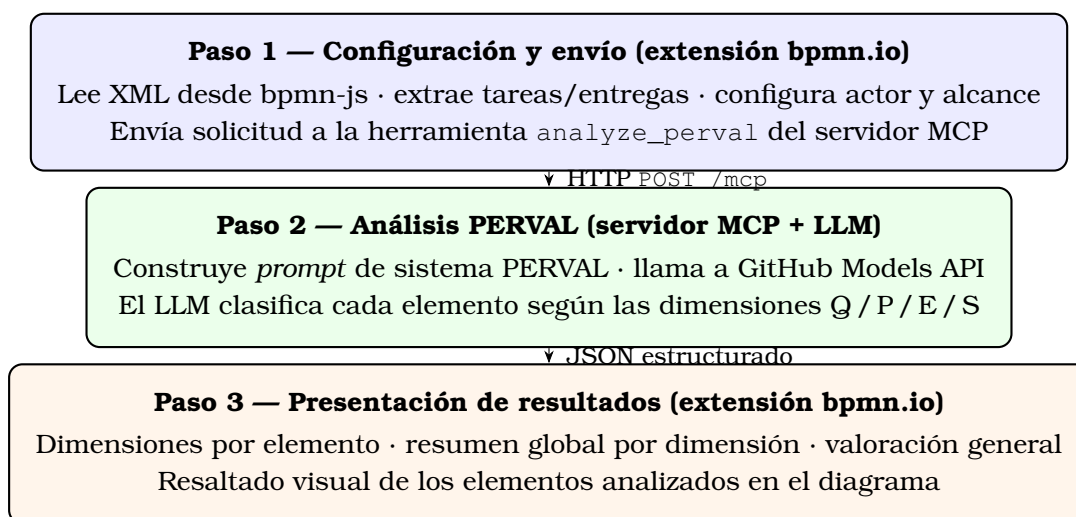


Figura 5.5: Flujo de la operación de análisis PERVAL.

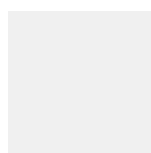
5.3. Tecnología utilizada

En el desarrollo del proyecto se han empleado distintas tecnologías orientadas al diseño, la programación, el control de versiones y el despliegue del sistema. A continuación, se describen las principales decisiones tecnológicas adoptadas en cada uno de estos ámbitos.

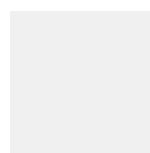
5.3.1. Diseño

Para la elaboración de los diagramas del proyecto se ha utilizado la herramienta **Draw.io**, una aplicación web de código abierto que permite diseñar y editar diagramas directamente desde el navegador, sin necesidad de instalar software adicional ni adquirir licencias. Draw.io ofrece compatibilidad con múltiples plataformas y soporte para diagramas UML, de arquitectura, de flujo y de red, entre otros. Su uso en este TFM ha permitido elaborar los diagramas de actores, de contexto, de casos de uso y de arquitectura presentes en la memoria.

Para el diseño de los bocetos de la interfaz de usuario se ha empleado **Figma**, una plataforma de diseño colaborativo orientada al desarrollo de interfaces y prototipos interactivos. Figma ha permitido elaborar de forma ágil los bocetos iniciales del panel conversacional y de la distribución general de la interfaz, facilitando la clarificación de ideas y la planificación del desarrollo antes de iniciar la implementación.



Draw.io



Figma

5.3.2. Programación

A continuación se expone la justificación de las tecnologías de programación seleccionadas para cada componente del sistema, atendiendo a criterios de adecuación al contexto, compatibilidad, robustez y madurez de las herramientas.

5.3.2.1. Lenguajes y tecnologías utilizados en cada componente

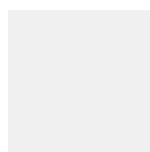
Dado que el sistema está compuesto por dos módulos software principales —la extensión sobre bpmn.io y el servidor MCP—, en esta subsección se describe la elección de lenguajes de programación y tecnologías para cada uno de ellos, junto con la justificación de dichas decisiones.

5.3.2.1.1. Extensión sobre bpmn.io. La extensión sobre bpmn.io ha sido desarrollada en **JavaScript** con módulos ES (*ECMAScript Modules*), el estándar moderno de organización y reutilización de código en el ecosistema web. JavaScript es el único lenguaje de programación soportado de forma nativa por los navegadores web, lo que lo convierte en la opción obligatoria para el desarrollo de la extensión, dado que bpmn.io es una herramienta de modelado basada en navegador.

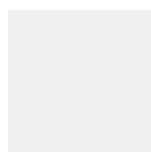
La librería **bpmn-js** (versión 18.12.0) proporciona el núcleo de edición y renderizado de diagramas BPMN 2.0. Se trata de una librería de código abierto mantenida activamente por la organización bpmn.io, con una arquitectura modular y extensible mediante *plugins*. La librería **diagram-js** (versión 15.1.0), sobre la que se construye bpmn-js, proporciona las capacidades base de manipulación de elementos de diagrama.

La librería **jQuery** (versión 4.0.0) se ha utilizado para simplificar la manipulación del DOM en el panel conversacional y la gestión de eventos de la interfaz añadida por la extensión. Su uso resulta adecuado para el alcance de esta extensión, en la que no se requiere una arquitectura de componentes reactiva de mayor complejidad.

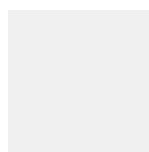
El empaquetado de la extensión para su ejecución en el navegador se realiza mediante **Webpack 5**, que transpila y agrupa los módulos JavaScript, los estilos CSS y los recursos estáticos en un *bundle* listo para ser servido por el servidor de desarrollo o desplegado en producción.



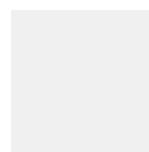
JavaScript



bpmn-js



jQuery



Webpack

5.3.2.1.2. Servidor MCP. El servidor MCP ha sido implementado en **JavaScript** (ES modules) ejecutándose sobre **Node.js**, el entorno de ejecución de JavaScript en el

Diseño de la solución

lado del servidor. La elección de JavaScript para el servidor permite compartir código y convenciones con la extensión del cliente, reduciendo la complejidad del proyecto y facilitando el mantenimiento.

El framework **Express** (versión 5.2.1) se ha utilizado para implementar el servidor HTTP. Express es uno de los frameworks web más consolidados del ecosistema Node.js, con una curva de aprendizaje reducida y un ecosistema de *middleware* amplio. Su flexibilidad lo hace adecuado para implementar el *endpoint* único del servidor MCP (POST /mcp) con un mínimo de configuración.

La librería **@modelcontextprotocol/sdk** (versión 1.29.0) proporciona la implementación del protocolo MCP para Node.js, incluyendo el servidor (McpServer), el transporte HTTP (StreamableHTTPServerTransport) y las utilidades de registro y gestión de herramientas. Su uso garantiza el cumplimiento del estándar MCP y la interoperabilidad con cualquier cliente MCP compatible.

Para la definición de los esquemas de entrada y salida de las herramientas MCP se ha utilizado la librería **Zod** (versión 4.4.3), que permite definir esquemas de validación de datos de forma declarativa en JavaScript. Cada herramienta expone su `inputSchema` definido mediante Zod, garantizando que los datos recibidos del cliente tienen la estructura y los tipos esperados antes de ser procesados.

La gestión de las variables de entorno —incluyendo el token de acceso a la API de GitHub Models— se realiza mediante **dotenv** (versión 17.4.2), que permite configurar el servidor a través de un fichero `.env` local sin exponer credenciales en el código fuente.



5.3.2.2. Frameworks y bibliotecas

La tabla 5.2 resume los principales *frameworks* y bibliotecas utilizados en el proyecto, indicando el componente en el que se emplean y la versión utilizada.

Tecnología	Componente	Versión
bpmn-js	Extensión bpmn.io	18.12.0
diagram-js	Extensión bpmn.io	15.1.0
jQuery	Extensión bpmn.io	4.0.0
Webpack	Extensión bpmn.io	5.x
Express	Servidor MCP	5.2.1
@modelcontextprotocol/sdk	Servidor MCP	1.29.0
Zod	Servidor MCP	4.4.3
dotenv	Servidor MCP	17.4.2
cors	Servidor MCP	2.8.6

Cuadro 5.2: Principales *frameworks* y bibliotecas utilizados en el proyecto.

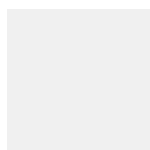
5.3.2.3. Entornos de desarrollo

El desarrollo del proyecto se ha llevado a cabo principalmente con **Visual Studio Code**, un editor de código ligero, extensible y ampliamente adoptado en el ecosistema JavaScript/Node.js. Visual Studio Code ofrece soporte nativo para JavaScript, integración con Git, *debugging* integrado y un ecosistema de extensiones que facilita el desarrollo y la inspección del código.

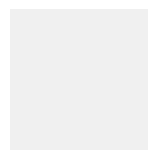
La gestión de dependencias del proyecto se realiza mediante **npm** (*Node Package Manager*), el gestor de paquetes estándar del ecosistema Node.js, que permite instalar, actualizar y gestionar las dependencias del proyecto de forma declarativa a través del fichero `package.json`.

5.3.3. Control de versiones

El control de versiones del proyecto se ha gestionado mediante **Git**, el sistema de control de versiones distribuido más utilizado en la actualidad. Git permite registrar el historial de cambios del código y trabajar con ramas de desarrollo independientes. El repositorio remoto se ha alojado en **GitHub**, plataforma de alojamiento de repositorios Git que proporciona además herramientas de seguimiento de tareas y gestión de proyectos. El repositorio se ha organizado siguiendo la estructura de *sprints* de la metodología ágil adoptada, con *commits* frecuentes que documentan el progreso incremental del desarrollo.



Git

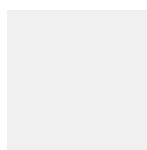


GitHub

5.3.4. Despliegue

El servidor MCP se ha desplegado en la plataforma en la nube **Render.com**, que ofrece servicios de alojamiento de aplicaciones Node.js con despliegue continuo desde repositorios de GitHub, reinicio automático ante fallos y acceso HTTPS mediante certificado SSL gestionado por la propia plataforma. Esta elección permite que el servidor MCP esté disponible de forma continua y accesible desde cualquier instancia del editor bpmn.io, sin necesidad de infraestructura propia.

La extensión sobre bpmn.io se ejecuta íntegramente en el navegador del usuario y no requiere despliegue en un servidor web propio. Durante el desarrollo, se sirve localmente mediante el servidor de desarrollo de Webpack (`webpack-dev-server`), que permite la recarga automática de los cambios en el navegador sin necesidad de una compilación manual.



Render.com

Capítulo 6

Desarrollo e implementación

Dado que el desarrollo del proyecto se ha llevado a cabo siguiendo una metodología ágil con un enfoque iterativo e incremental, organizando el trabajo en *sprints* semanales de acuerdo con la planificación definida en el capítulo de análisis (véase §4.5), el sistema ha experimentado una evolución progresiva a lo largo del tiempo. En cada iteración se ha partido del resultado de la anterior, incorporando nuevas funcionalidades y refinando las ya implementadas, con la participación activa del autor como desarrollador y usuario del sistema.

6.1. Evolución del proyecto por *sprints*

En los siguientes apartados se describen, para cada iteración, las tareas realizadas, las historias de usuario completadas y los principales hitos técnicos alcanzados.

6.1.1. *Sprint 0* (26/05/2026 – 01/06/2026)

El Sprint 0 estuvo dedicado a la selección tecnológica, la planificación del proyecto y la configuración del entorno de desarrollo, abarcando las historias de usuario US01 y US02.

En el marco de la US01, se llevó a cabo un análisis exhaustivo de las alternativas disponibles para abordar los objetivos del TFM. Respecto al editor BPMN, se estudiaron las opciones de *draw.io* y *bpmn.io*: mientras que *draw.io* ofrece soporte general para diagramas BPMN, no proporciona una API que permita la manipulación programática del modelo ni la integración de un panel conversacional propio, por lo que se descartó en favor de *bpmn.io* (*bpmn-js*), que sí ofrece estas capacidades a través de su arquitectura modular de *plugins*. En cuanto al proveedor de LLM, se compararon el acceso directo a las API de OpenAI y de Google con el servicio GitHub Models, optándose finalmente por este último por su plan gratuito y por la posibilidad de acceder a múltiples modelos (*gpt-4.1*, *gpt-4o*, *Phi-4*, entre otros) a través de una única API compatible con el formato OpenAI. Asimismo, se definió la arquitectura de integración basada en el protocolo MCP, que permite desacoplar la extensión del cliente de la lógica de llamada al LLM y almacenar las credenciales de forma segura en el servidor. Finalmente, se estableció el *backlog* inicial de historias de usuario y se planificó la distribución del trabajo en los ocho *sprints* del proyecto.

La US02 se centró en la configuración del entorno de desarrollo. Se clonó el repositorio `bpmn-js-examples` y se tomó como punto de partida el ejemplo `modeler`, que proporciona un editor BPMN 2.0 funcional en el navegador con soporte de importación y exportación de XML. Se configuró el sistema de empaquetado mediante **Webpack 5**, se verificó que todas las dependencias de `bpmn-js` se resolvieran correctamente y se comprobó el funcionamiento básico del editor. Al finalizar el sprint, se disponía de un entorno de desarrollo operativo sobre el que construir la extensión.

6.1.2. Sprint 1 (02/06/2026 – 08/06/2026)

El Sprint 1 abordó el diseño e implementación del servidor MCP y su integración con la API de GitHub Models, correspondiendo a las historias US03 y US04.

La US03 comenzó con el estudio de la especificación del *Model Context Protocol* y del SDK oficial `@modelcontextprotocol/sdk` para Node.js. A continuación, se implementó el servidor utilizando el framework **Express** (versión 5.2.1), que expone un único *endpoint* `POST /mcp`. El servidor crea, por cada petición recibida, una instancia del `McpServer` y del transporte `StreamableHTTPServerTransport`, los conecta, procesa la petición y los destruye al finalizar la respuesta, siguiendo el modelo *stateless* descrito en el diseño. Se definió la estructura base de registro de herramientas MCP, especificando para cada una su nombre, descripción y esquemas de entrada y salida mediante la librería **Zod**. Al finalizar esta historia se realizaron pruebas de conexión entre una extensión de prueba en el navegador y el servidor, verificando que el protocolo MCP funcionaba correctamente en el entorno local.

La US04 añadió la integración con la API de GitHub Models. Se implementó la función `callOpenAI`, encargada de construir la solicitud HTTP a la API de GitHub Models con el *prompt* de sistema y de usuario correspondiente, gestionar la respuesta y devolver el texto generado. Se configuró el acceso mediante la variable de entorno `GITHUB_TOKEN`, gestionada con la librería **dotenv** para evitar su exposición en el código fuente. Se implementó también el mecanismo de *fallback* automático: si el modelo principal (`gpt-4.1`) devuelve un error de cuota o de disponibilidad, la función intenta sucesivamente con `gpt-4o`, `gpt-4o-mini`, `gpt-4.1-mini`, `gpt-4.1-nano` y `Phi-4`, lanzando un error únicamente si todos fallan. Este mecanismo resultó especialmente útil durante el desarrollo, cuando los límites de peticiones del plan gratuito se alcanzaban con frecuencia.

6.1.3. Sprint 2 (09/06/2026 – 15/06/2026)

El Sprint 2 se dedicó íntegramente a la implementación de la generación de modelos BPMN a partir de descripciones en lenguaje natural (US05), que constituye la funcionalidad central del sistema.

El principal reto de esta historia fue el diseño de los *prompts* de las herramientas de generación iterativa. Para cada una de las seis herramientas del flujo de generación (`identify_structure`, `suggest_flow_start`, `suggest_next_step`, `suggest_gateway_branches`, `suggest_branch_step` y `render_diagram`), se elaboró un *prompt* de sistema que especifica con precisión el formato de la respuesta esperada (objeto JSON o XML BPMN 2.0 válido), los elementos BPMN soportados y las convenciones de nomenclatura y estructura del diagrama. Se implementó la función `parseLLMJson`, que extrae y deserializa el objeto JSON de la respuesta del LLM incluso cuando esta incluye

texto adicional o bloques de código Markdown, lo que resultó necesario dado que los modelos de lenguaje no siempre devuelven JSON puro.

En el lado de la extensión, se implementó la lógica de carga del XML BPMN generado en el editor *bpmn-js*: al recibir el XML del servidor MCP, la extensión lo importa mediante la API de la librería (`bpmnViewer.importXML`), actualiza la vista del editor y centra automáticamente el lienzo en el contenido del diagrama. Se realizaron pruebas con descripciones de proceso de diferente complejidad (procesos lineales, procesos con *gateways* y múltiples participantes) para verificar la calidad de los modelos generados y ajustar los *prompts* en consecuencia. Al finalizar el sprint se disponía de un sistema capaz de generar un modelo BPMN 2.0 completo a partir de una descripción en lenguaje natural.

6.1.4. *Sprint 3* (16/06/2026 – 22/06/2026)

En el Sprint 3 se implementaron la edición conversacional iterativa del modelo (US06) y la validación y reparación automática del XML BPMN (US07).

La US06 incorporó al sistema la posibilidad de realizar modificaciones sobre el diagrama generado mediante instrucciones en lenguaje natural. Se implementó la herramienta MCP `refine_diagram`, que recibe el XML BPMN actual del diagrama junto con la instrucción de modificación expresada por el usuario y devuelve el XML modificado. El *prompt* de esta herramienta fue especialmente cuidado para garantizar que el LLM preservara los elementos no afectados por la modificación y que el XML devuelto fuera válido. En el lado de la extensión, se diseñó e implementó el panel conversacional: un área lateral que muestra el historial de mensajes intercambiados entre el usuario y el sistema, con un campo de entrada de texto y un botón de envío. Se implementó la gestión del historial de la conversación en el cliente, que se transmite al servidor en cada petición para mantener el contexto de la interacción.

La US07 abordó un problema detectado durante el Sprint 2: los LLM generan con frecuencia XML BPMN malformado o truncado, especialmente cuando el diagrama es complejo y la respuesta supera los límites de tokens del modelo. Se analizaron los errores más habituales (etiquetas sin cerrar, comentarios XML incrustados, texto adicional antes o después del bloque XML, comillas sin escapar) y se implementó la función `cleanXML`, que aplica una secuencia de transformaciones para sanear la respuesta antes de intentar cargarla en *bpmn-js*. Esta función resultó crítica para la fiabilidad del sistema: sin ella, una proporción significativa de las respuestas del LLM no podía cargarse en el editor.

6.1.5. *Sprint 4* (23/06/2026 – 29/06/2026)

El Sprint 4 se centró en el diseño e implementación de la herramienta de análisis PERVAL, abarcando las historias US08 y US09.

La US08 se dedicó al diseño de la herramienta. Se realizó un estudio detallado del modelo PERVAL de Sweeney y Soutar [12], prestando especial atención a la operacionalización de las cuatro dimensiones de valor percibido (*Quality*, *Price*, *Emotional* y *Social*) en el contexto de tareas y entregas de procesos de negocio. A partir de este estudio se diseñó el esquema de entrada de la herramienta: el XML BPMN del diagrama, la lista de tareas o entregas a analizar, el nombre del actor externo cuyo punto de vista se adopta, el alcance del análisis (entregas al cliente externo o todas las tareas

del proceso) y el idioma de los textos generados. Se diseñó asimismo el *prompt* de análisis PERVAL, que instruye al LLM a clasificar cada elemento del proceso según las cuatro dimensiones de valor y a devolver un objeto JSON estructurado con las clasificaciones individuales, un resumen por dimensión y una valoración general del proceso.

La US09 completó la implementación de la herramienta. Se registró `analyze_perval` en el servidor MCP con su esquema Zod correspondiente, se implementó la lógica de extracción de las tareas y entregas directamente a partir del XML BPMN recibido y se verificó el cálculo del resumen agregado por dimensión a partir de las clasificaciones individuales devueltas por el LLM. Se realizaron pruebas con modelos BPMN de diferente complejidad y con distintos actores externos para validar la calidad y consistencia de los resultados del análisis.

6.1.6. Sprint 5 (30/06/2026 – 06/07/2026)

El Sprint 5 abordó la visualización de los resultados del análisis PERVAL (US10) y el diseño e implementación de la interfaz del panel conversacional (US11).

En el marco de la US10, se implementó en la extensión la interfaz de configuración del análisis PERVAL: un formulario que permite al usuario seleccionar el alcance del análisis (tareas o entregas al cliente externo), introducir el nombre del actor externo cuyo punto de vista se desea adoptar y lanzar el análisis. Una vez recibida la respuesta del servidor, la extensión presenta los resultados de forma estructurada en el panel conversacional: para cada elemento analizado se muestran las cuatro dimensiones PERVAL con su clasificación y la justificación textual generada por el LLM, y se añade un resumen global por dimensión junto con la valoración general del proceso. Se implementó también el resaltado visual sobre el diagrama BPMN de los elementos analizados, de modo que el usuario pueda identificar fácilmente qué tareas o entregas han sido evaluadas.

La US11 se centró en el diseño visual del panel conversacional. Se definió una paleta de colores basada en tonos azul y verde para diferenciar los mensajes del usuario y las respuestas del sistema, y se seleccionó la fuente *Inter* por su legibilidad en pantallas de alta densidad. Se implementaron los estilos del panel adaptados a diferentes tamaños de pantalla, se añadieron indicadores de estado de carga (animación de puntos durante el procesamiento de peticiones) y mensajes de error con el formato adecuado para cada tipo de fallo (error de conexión con el servidor, error de cuota del LLM, XML inválido). Se realizaron pruebas de la interfaz en distintas resoluciones de pantalla y se ajustaron los estilos para garantizar una experiencia de usuario coherente.

6.1.7. Sprint 6 (07/07/2026 – 13/07/2026)

El Sprint 6 se destinó a la internacionalización de la interfaz y los *prompts* (US12) y a la implementación de las pruebas *end-to-end* con *Playwright* (US13).

La US12 implementó el soporte multilingüe del sistema. Se extrajeron todos los textos de la interfaz a ficheros de idioma (`i18n/es.js` y `i18n/en.js`), cubriendo los mensajes del panel conversacional, los textos de los formularios, los indicadores de estado y los mensajes de error. El idioma activo se selecciona mediante un selector en la interfaz y se almacena en el estado de la extensión. Los *prompts* de todas las herramientas

MCP se parametrizaron según el idioma activo, de modo que las respuestas del LLM —incluidas las justificaciones del análisis PERVAL— se generan directamente en el idioma seleccionado, sin necesidad de traducción posterior. Se verificó el funcionamiento completo del sistema en ambos idiomas mediante pruebas manuales de los flujos principales.

La US13 configuró *Playwright* sobre el proyecto e implementó un conjunto de pruebas *end-to-end* que verifican el funcionamiento de los flujos principales del sistema en un navegador real (Chromium). Las pruebas cubren la generación de un modelo BPMN a partir de una descripción en lenguaje natural, la modificación conversacional del modelo generado y la ejecución del análisis PERVAL sobre el diagrama resultante. Dado que las pruebas realizan llamadas reales a la API de GitHub Models, los tiempos de ejecución son variables y pueden superar el minuto en modelos de mayor latencia; por ello, se configuraron *timeouts* generosos y se añadieron verificaciones intermedias del estado de la interfaz que permiten distinguir entre un sistema lento y uno que ha fallado.

6.1.8. Sprint 7 (14/07/2026 – 20/07/2026)

El Sprint 7 cerró el desarrollo del proyecto con el despliegue en producción (US14) y la validación de la solución con usuarios mediante el modelo TAM (US15).

La US14 configuró el despliegue continuo en la plataforma *Render.com*. Se elaboró el fichero `render.yaml`, que define dos servicios: un sitio estático (`tfm-bpmn-frontend`) para servir la extensión empaquetada por Webpack, y un servicio web Node.js (`tfm-bpmn-mcp-server`) para el servidor MCP. Ambos servicios se despliegan automáticamente al hacer *push* al repositorio de GitHub, con reinicio automático ante fallos y acceso HTTPS mediante certificados gestionados por la propia plataforma. Se realizaron pruebas de integración entre la extensión desplegada en *Render* y el servidor MCP, verificando el correcto funcionamiento del sistema en el entorno de producción y documentando las variables de entorno necesarias para futuras configuraciones.

La US15 completó la validación de la solución con un grupo de aproximadamente quince participantes, siguiendo el modelo de aceptación de la tecnología (*Technology Acceptance Model*, TAM) [?]. Se diseñó un cuestionario que mide tres constructos: la utilidad percibida (PU), la facilidad de uso percibida (PEOU) y la intención de uso (IU), adaptando los ítems estándar del TAM al contexto específico de la herramienta. Los participantes utilizaron la versión desplegada del sistema para generar y editar modelos BPMN a partir de descripciones de proceso propuestas, y completaron el cuestionario al finalizar la sesión. Los resultados obtenidos se presentan y analizan en el capítulo de Resultados.

Los aspectos técnicos más relevantes de la implementación —la lógica de llamada al LLM, los *prompts* de generación y análisis PERVAL, el renderizador programático del XML BPMN y las funciones de robustez— se describen en detalle en la sección 6.2.

6.2. Implementación de los componentes técnicos clave

Los apartados anteriores han narrado la evolución del sistema iteración a iteración. Esta sección profundiza en los componentes de implementación de mayor relevancia técnica. Para cada *prompt*, se muestra el texto exacto en lenguaje natural que recibe

6.2. Implementación de los componentes técnicos clave

el LLM, con [corchetes] para las partes dinámicas que se inyectan en tiempo de ejecución a partir del estado de la conversación. Para los componentes algorítmicos (función de llamada al LLM, renderizador de XML y funciones de robustez) se muestra el código fuente completo.

6.2.1. La función `callOpenAI` y el mecanismo de *fallback*

Toda la comunicación con el LLM se canaliza a través de `callOpenAI`, implementada en `mcp-server/src/llm.js`. La función construye una petición POST al *endpoint* de inferencia de GitHub Models, transmitiendo un *prompt* de sistema, un historial de mensajes y los parámetros de generación: temperatura 0,3 y límite de 8.000 *tokens* de salida. El mecanismo de *fallback* opera en dos niveles: rotación automática entre los seis modelos de la lista `LLM_MODELS` ante errores de cuota o disponibilidad, y reintento global con espera adaptativa cuando todos los modelos devuelven error 429 simultáneamente. Cada llamada individual dispone de un *timeout* de 30 segundos implementado con `AbortController`.

```
// mcp-server/src/llm.js
export const LLM_MODELS = ['gpt-4.1', 'gpt-4o', 'gpt-4o-mini', 'gpt-4.1-mini', 'gpt-4.1-nano', 'Phi-4'];
let _llmModelIdx = 0; // recuerda el último modelo que funcionó en esta sesión

export async function callOpenAI(apiKey, systemPrompt, messages) {
  let lastErr = null;

  for (let globalRetry = 0; globalRetry < 3; globalRetry++) {
    let minWaitMs = 0;
    let allRateLimited = true;

    for (let k = 0; k < LLM_MODELS.length; k++) {
      const idx = (_llmModelIdx + k) % LLM_MODELS.length;
      const model = LLM_MODELS[idx];
      let response;
      const controller = new AbortController();
      const timer = setTimeout(() => controller.abort(), 30000); // 30s por modelo
      try {
        response = await fetch('https://models.inference.ai.azure.com/chat/completions', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${apiKey}` },
          body: JSON.stringify({
            model,
            messages: [{ role: 'system', content: systemPrompt }, ...messages],
            temperature: 0.3,
            max_tokens: 8000
          }),
          signal: controller.signal
        });
      } catch(e) { clearTimeout(timer); lastErr = e; allRateLimited = false;
      continue; }
      clearTimeout(timer);

      if (response.ok) {
        _llmModelIdx = idx;
        const data = await response.json();
        return data.choices[0].message.content.trim();
      }
    }
  }
}
```



```
    }

    const err = await response.json().catch(() => ({}));
    const msg = err.error?.message || `Error ${response.status} en la API`;
    console.warn(`Modelo "${model}" no disponible (${msg}), probando con el
siguiente...`);
    lastErr = new Error(msg);

    if (response.status === 429) {
      const waitMatch = msg.match(/wait (\d+) second/i);
      if (waitMatch) minWaitMs = Math.max(minWaitMs, parseInt(waitMatch[1]) *
1000);
    } else {
      allRateLimited = false;
    }
  }

  if (allRateLimited && globalRetry < 2) {
    const wait = minWaitMs > 0 ? Math.min(minWaitMs, 60000) : 15000;
    console.warn(`Todos los modelos en rate-limit. Reintentando en ${wait / 1000}
s...`);
    await new Promise(r => setTimeout(r, wait));
  } else if (!allRateLimited) {
    break;
  }
}

throw new Error(`Ningún modelo disponible (cuota agotada o modelos retirados).
Espera unas horas o usa otro token. Último error: ${lastErr?.message || '?'}`);
}
```

6.2.2. Prompts del flujo de generación iterativa

El flujo de generación descompone la tarea en seis llamadas al LLM más un *prompt* auxiliar. En los bloques siguientes se reproduce el texto exacto en lenguaje natural que recibe el modelo en cada llamada. Las partes entre [corchetes] son valores dinámicos inyectados en tiempo de ejecución desde el estado de la conversación; el resto es texto estático del *prompt*.

Nota: todos los *prompts* incluyen, cuando la interfaz está en modo inglés, una directiva de idioma final («*IMPORTANT — LANGUAGE: Generate all human-readable content in English...*») que garantiza que las respuestas del LLM se generen en el idioma activo. En modo español dicha directiva es una cadena vacía y no aparece en el *prompt*.

6.2.2.1. buildStructureSystemPrompt — herramienta identify_structure

Primer paso del flujo: identifica los participantes del proceso y los devuelve en JSON puro. Exige JSON sin texto adicional, distingue entre actores con rol *cliente* (receptor del valor) y *colaborador* (participante sin recibir el resultado final), y prohíbe inventar departamentos no mencionados.

Eres un experto en modelado de procesos BPMN 2.0 para e-commerce.
Analiza la descripción de un proceso de negocio e identifica los participantes
y departamentos.

RESPONDE ÚNICAMENTE con JSON válido (sin texto adicional, sin bloques de código):

6.2. Implementación de los componentes técnicos clave

```
{
  "poolExterno": [{"nombre": "nombre del canal o cliente externo", "rol": "cliente"}],
  "poolPrincipal": {
    "nombre": "nombre de la empresa u organización principal",
    "lanes": ["Departamento1", "Departamento2"]
  },
  "resumen": "Una frase resumiendo los participantes"
}

REGLAS:
- poolExterno: canales o actores EXTERNOS (clientes, marketplace, pasarela de pago, transportista, proveedor...). Puede ser [].
- "rol" de cada poolExterno: "cliente" si ese actor es quien RECIBE EL VALOR/ resultado del proceso (normalmente quien inicia el proceso o a quien se dirige el resultado final, p.ej. el comprador/cliente final); "colaborador" para el resto de actores externos que participan pero NO son destinatarios del valor (pasarela de pago, transportista, proveedor, marketplace...).
- Si poolExterno no está vacío, debe haber AL MENOS un actor con "rol":"cliente".
- poolPrincipal.lanes: SOLO los departamentos/áreas internas mencionados EXPLÍCITAMENTE en la descripción (Ventas, Logística, Atención al Cliente, Finanzas...). Máximo 5.
  Si la descripción NO menciona departamentos ni áreas internas, devuelve [] (lista vacía) -- NO inventes departamentos: la organización se dibujará como una única piscina sin calles.
- Nombres concisos: 1-3 palabras
```

6.2.2.2. buildModifyStructurePrompt — herramienta modify_structure

Permite al usuario ajustar la estructura identificada (añadir o renombrar departamentos, cambiar actores) antes de continuar con la definición del flujo. Recibe el JSON de la estructura actual y la instrucción de modificación.

```
Estructura actual:
[JSON de la estructura actual, por ejemplo:]
{
  "poolExterno": [{"nombre": "Cliente", "rol": "cliente"},
                  {"nombre": "Transportista", "rol": "colaborador"}],
  "poolPrincipal": {"nombre": "TiendaOnline",
                    "lanes": ["Ventas", "Logística"]},
  "resumen": "Proceso de venta online entre el cliente y la tienda"
}

Modificación: "[instrucción del usuario, p.ej.: añade el departamento Finanzas]"

Devuelve ÚNICAMENTE el JSON modificado (mismo formato, conservando el campo "rol" de cada poolExterno --"cliente" o "colaborador"-- y ajustándolo solo si la modificación lo requiere).
```

6.2.2.3. buildFlowStartPrompt — herramienta suggest_flow_start

Identifica el punto o puntos de inicio del proceso. Permite varios inicios en paralelo (poco frecuente) e indica si cada inicio está disparado por un mensaje de un actor

Desarrollo e implementación

externo (trigger: "message") o es autónomo (trigger: "none").

```
Eres un experto en modelado BPMN 2.0 para e-commerce.
Departamentos internos: [Ventas, Logística, Atención al Cliente]
Actores externos: [Cliente, Pasarela de pago, Transportista]

Descripción del proceso:
[descripción del proceso introducida por el usuario]

Identifica el/los punto(s) de INICIO del proceso (puede haber varios inicios en
paralelo si el proceso arranca por más de un motivo).
Para cada inicio indica:
- "lane": el departamento donde ocurre (uno de los departamentos listados)
- "nombre": nombre breve del evento de inicio
- "trigger": "message" si lo dispara la recepción de un mensaje de un actor externo
  ,
  o "none" en caso contrario
- "from": si trigger es "message", el nombre de uno de los actores externos; si no,
  null

REGLAS:
- Si no hay actores externos, usa siempre "trigger":"none" y "from":null.
- Normalmente hay un único inicio; usa varios solo si la descripción lo indica
  explícitamente.

RESPONDE SOLO con JSON (sin texto adicional):
{"inicios":[{"lane":"[primer departamento]","nombre":"Inicio","trigger":"none","
  from":null}]}
```

6.2.2.4. buildNextStepPrompt — herramienta suggest_next_step

El *prompt* más extenso del sistema. En cada llamada recibe el inventario cronológico de pasos ya confirmados y propone el siguiente elemento del flujo principal. Incluye reglas detalladas de clasificación de tipo BPMN para evitar el uso del tipo genérico task, que era el comportamiento por defecto de los modelos sin esta guía. Los marcadores «OBLIGATORIO» y «NUNCA» responden a observaciones empíricas: formulaciones más suaves eran ignoradas con frecuencia.

```
Eres un experto en modelado BPMN 2.0 para e-commerce.
Departamentos internos: [Ventas, Logística, Atención al Cliente]
Actores externos: [Cliente, Transportista]

Descripción completa del proceso:
[descripción del proceso]

Pasos confirmados hasta ahora (en orden cronológico):
[historial acumulado de pasos, p.ej.:]
1. [Ventas] Tarea de usuario: Registrar pedido
2. [Logística] Gateway exclusivo: Verificar disponibilidad de stock
  - Caso "Hay stock": Preparar envío -> Generar etiqueta [converge]
  - Caso "Sin stock": Notificar al cliente [termina el proceso]

¿Cuál es el SIGUIENTE paso del proceso, según la descripción? Indica:
- "tipo": uno de userTask | serviceTask | sendTask | exclusiveGateway |
  parallelGateway |
  inclusiveGateway | timerEvent | intermediateCatchEvent | intermediateThrowEvent |
  compensationEvent | endMessageEvent | errorEndEvent | task
```

6.2. Implementación de los componentes técnicos clave

```
- "nombre": nombre breve del paso
- "lane": departamento que lo realiza (uno de los departamentos listados)
- "actorExterno": SOLO si tipo es "sendTask", "intermediateThrowEvent" o "
  endMessageEvent"
  y va dirigido a un actor externo, su nombre; si no, null
- "esFinal": true si DESPUÉS de este paso el proceso TERMINA

=== TIPO DE ELEMENTO -- REGLAS OBLIGATORIAS (aplica en orden) ===

> GATEWAYS -- modela SIEMPRE la estructura real del proceso:
- "exclusiveGateway" (XOR): cuando el proceso llega a una CONDICIÓN O DECISIÓN
  que lo
  bifurca en caminos MUTUAMENTE EXCLUYENTES (solo se sigue UNO). Señales: "si
  ...",
  "en caso de...", "dependiendo de...", "cuando...", "según si...".
  NUNCA ignores una bifurcación real.
- "parallelGateway" (AND): cuando varias actividades ocurren EN PARALELO o
  SIMULTÁNEAMENTE y TODAS deben completarse. Señales: "al mismo tiempo", "en
  paralelo",
  "simultáneamente", "mientras tanto", "a la vez".
  OBLIGATORIO cuando el proceso divide en ramas concurrentes.
- "inclusiveGateway" (OR): cuando se pueden seguir UNO O VARIOS caminos según
  condiciones no excluyentes. Señales: "según lo que corresponda", "los que
  apliquen",
  "uno o más de...".
Si hay una bifurcación o paralelismo en este punto -> pon el gateway AHORA.

> TAREAS -- clasifica según quién ejecuta la acción:
- "userTask": OBLIGATORIO cuando la acción la realiza una PERSONA (empleado,
  agente,
  operario...). Incluye: revisar, aprobar, gestionar, tramitar, atender, validar
  manualmente, seleccionar, contactar, introducir datos, tomar una decisión,
  rellenar
  formulario, inspeccionar físicamente.
  -> Si una persona interviene, SIEMPRE "userTask". NUNCA uses "task" para
  acciones
  humanas.
- "serviceTask": OBLIGATORIO cuando la acción la realiza el SISTEMA de forma
  automática,
  sin intervención humana. Incluye: consultar API, actualizar base de datos,
  generar
  documento, procesar pago automáticamente, enviar petición a sistema externo,
  calcular,
  transformar datos, integrar sistemas, lanzar webhook, sincronizar inventario.
  -> Si es un sistema automático, SIEMPRE "serviceTask". NUNCA uses "task" para
  tareas
  del sistema.
- "sendTask": OBLIGATORIO para ENVIAR un mensaje, notificación, email, SMS,
  confirmación, factura o aviso a un actor EXTERNO cuando el proceso continúa
  después.
  Rellena "actorExterno".
- "task": SOLO como último recurso si la acción es genuinamente ambigua (ni
  claramente
  humana ni claramente automática). En la mayoría de procesos de e-commerce no
  debería
  aparecer.

> EVENTOS INTERMEDIOS -- insértalos cuando el proceso necesita esperar o revertir:
- "timerEvent": OBLIGATORIO cuando el proceso debe ESPERAR un plazo de tiempo
  antes de
  continuar. Señales: "esperar X días/horas", "al cabo de...", "tras N días
```

```
laborables",
"en un plazo de...", "después de...", "pasado el periodo de...",
"antes del vencimiento". -> Siempre que haya una espera temporal explícita o
implícita, modélala con timerEvent.
- "compensationEvent": cuando hay que DESHACER o REVERTIR una acción anterior.
- "intermediateCatchEvent": espera de un mensaje entrante de un actor externo.
- "intermediateThrowEvent": lanza un mensaje a un actor externo sin terminar el
proceso.

> EVENTOS DE FIN especiales:
- "endMessageEvent": el proceso TERMINA ENVIANDO un mensaje/notificación final a
un
actor externo (confirmación de pedido, factura, ticket...). "esFinal" debe ser
true.
- "errorEndEvent": el proceso TERMINA con error irrecuperable (pago rechazado
definitivamente, fraude detectado, cancelación sin solución...). "esFinal":
true.

REGLAS ADICIONALES:
- Si no hay actores externos, "actorExterno" debe ser siempre null.
- PROHIBIDO repetir un paso ya confirmado. Si no quedan pasos nuevos, devuelve el ú
ltimo
paso real con "esFinal": true.
- "nombre" DEBE ser descriptivo y específico. PROHIBIDO: "Siguiente paso", "Tarea".

RESPONDE SOLO con JSON (sin texto adicional):
{"siguiente":{"tipo":"userTask","nombre":"...", "lane":"[primer departamento]","
actorExterno":null},
"esFinal":false}
```

6.2.2.5. buildGatewayBranchesPrompt — herramienta suggest_gateway_branches

Cuando el paso propuesto es una puerta lógica, se lanza esta llamada adicional para que el LLM nombre los casos que salen de dicha puerta. Distingue los tres tipos de *gateway* y fija entre 2 y 5 ramas con nombres breves.

```
Eres un experto en modelado BPMN 2.0 para e-commerce.
Departamentos internos: [Ventas, Logística, ...]
Actores externos: [Cliente, ...]

Descripción completa del proceso:
[descripción del proceso]

El proceso ha llegado a una puerta [según el tipo de gateway:]
- EXCLUSIVA (XOR): solo se sigue UNO de los casos según la condición
- INCLUSIVA (OR): se siguen UNO O VARIOS de los casos según las condiciones
- PARALELA (AND): se siguen TODOS los caminos a la vez
llamada "[nombre del gateway]".

Según la descripción, ¿qué casos/ramas salen de esta puerta?
- Mínimo 2 y máximo 5 casos.
- Nombres breves de 1-4 palabras (p.ej. "Hay stock" / "Sin stock", o
"Pago aceptado" / "Pago rechazado").

RESPONDE SOLO con JSON (sin texto adicional):
{"casos":["Caso 1","Caso 2"]}
```

6.2. Implementación de los componentes técnicos clave

6.2.2.6. buildBranchStepPrompt — herramienta suggest_branch_step

Análogo a buildNextStepPrompt pero opera dentro del contexto de un caso específico de una puerta. Prohíbe *gateways* anidados dentro de una rama y exige indicar si el paso es el último de la rama (`esFinalRama`) y si ese final implica la terminación del proceso completo (`terminaProceso`).

```
Eres un experto en modelado BPMN 2.0 para e-commerce.
Departamentos internos: [Ventas, Logística, ...]
Actores externos: [Cliente, ...]

Descripción completa del proceso:
[descripción del proceso]

El proceso llegó a la puerta "[nombre del gateway]" (exclusiva/XOR).
Estamos definiendo los pasos del caso/rama: "[nombre del caso]".

Pasos confirmados de ESTA rama hasta ahora:
[historial de pasos de la rama, p.ej.:]
1. [Logística] Tarea de usuario: Preparar envío
(o bien: "(ninguno todavía)" si la rama acaba de comenzar)

¿Cuál es el SIGUIENTE paso de esta rama, según la descripción? Indica:
- "tipo": uno de userTask | serviceTask | sendTask | timerEvent |
  intermediateCatchEvent | intermediateThrowEvent | compensationEvent |
  endMessageEvent | errorEndEvent | task
  (NO se permiten gateways dentro de una rama)
- "nombre": nombre breve del paso
- "lane": departamento que lo realiza (uno de los departamentos listados)
- "actorExterno": SOLO si tipo es "sendTask", "intermediateThrowEvent" o "
  endMessageEvent"
  y va dirigido a un actor externo, su nombre; si no, null
- "esFinalRama": true si este es el ÚLTIMO paso de esta rama
- "terminaProceso": SOLO si esFinalRama es true. true si el proceso COMPLETO
  termina aquí;
  false si converge.

=== TIPO DE ELEMENTO -- REGLAS OBLIGATORIAS (aplica en orden) ===

> TAREAS -- clasifica según quién ejecuta:
- "userTask": OBLIGATORIO cuando la acción la realiza una PERSONA (revisar,
  aprobar,
  gestionar, tramitar, atender, validar manualmente, contactar, rellenar,
  inspeccionar,
  tomar una decisión). -> NUNCA uses "task" para acciones que realiza una persona
.
- "serviceTask": OBLIGATORIO cuando la acción la realiza el SISTEMA automá
ticamente
  (consultar API, actualizar BBDD, generar documento, procesar automáticamente,
  sincronizar, calcular, integrar sistemas, lanzar webhook).
  -> NUNCA uses "task" para acciones automáticas del sistema.
- "sendTask": OBLIGATORIO para ENVIAR mensaje/notificación/email a un actor
EXTERNO
  cuando la rama continúa. Rellena siempre "actorExterno".
- "task": SOLO si la acción es genuinamente ambigua (ni claramente humana ni
  automática).

> EVENTOS INTERMEDIOS:
- "timerEvent": OBLIGATORIO cuando la rama debe ESPERAR un plazo de tiempo (
  esperar X
```

Desarrollo e implementación

```
días, al cabo de N horas, tras el periodo de..., antes del vencimiento...)).
- "compensationEvent": deshacer/revertir una acción anterior de esta rama.
- "intermediateCatchEvent": esperar un mensaje de un actor externo.
- "intermediateThrowEvent": lanzar un mensaje a un actor externo sin terminar la
  rama.

> EVENTOS DE FIN de rama (implican esFinalRama=true y terminaProceso=true):
- "endMessageEvent": esta rama TERMINA enviando un mensaje/notificación final a
  un actor
  externo.
- "errorEndEvent": esta rama TERMINA con error irrecuperable (pago rechazado,
  fraude,
  cancelación sin solución...)).

MUY IMPORTANTE -- ENVÍOS a actores externos:
- NUNCA uses "task" para enviar algo a un actor externo. Usa "sendTask" si la rama
  continúa, o "endMessageEvent" si termina con ese envío. Rellena siempre "
  actorExterno".

REGLAS:
- Si no hay actores externos, "actorExterno" debe ser siempre null.
- PROHIBIDO repetir un paso ya confirmado de esta rama. Si la descripción no
  contiene
  ningún paso NUEVO para esta rama, devuelve el último paso real con "esFinalRama":
  true.
- El "nombre" del paso DEBE ser descriptivo y específico. PROHIBIDO usar nombres
  genéricos
  como "Siguiente paso", "Next step", "Tarea", "Actividad", "Paso" o similares.

RESPONDE SOLO con JSON (sin texto adicional):
{"siguiente":{"tipo":"userTask","nombre":"...", "lane":"[primer departamento]","
  actorExterno":null},
  "esFinalRama":false,"terminaProceso":false}
```

6.2.2.7. bpmnJsonExpertPrompt — *prompt* de sistema auxiliar

Prompt de sistema breve reutilizado en las llamadas auxiliares rápidas (identificación de casos del *gateway*, inicio del proceso) donde no se necesita el contexto completo de clasificación de tipos BPMN.

Eres experto en modelado BPMN. Responde SOLO con JSON.

6.2.3. Renderizado programático del XML BPMN

La herramienta `render_diagram` no llama al LLM para producir el XML final. Delega en `renderDiagramFromState()`, implementada en `mcp-server/src/diagramRenderer.js`, que construye el XML BPMN 2.0 de forma completamente programática a partir del estado del diagrama acumulado. Esta decisión elimina las alucinaciones de XML que aparecían cuando se pedía al LLM el artefacto completo (identificadores duplicados, `sequenceFlow` rotos, coordenadas incoherentes). Los pasos principales se distribuyen con un espaciado horizontal fijo de 160 píxeles (DX); las ramas de una puerta lógica se desplazan verticalmente con un separador de 110 píxeles (BRANCH_GAP); la altura de cada calle se ajusta dinámicamente al número máximo de ramas del proceso.

6.2. Implementación de los componentes técnicos clave

```
// mcp-server/src/diagramRenderer.js
export function renderDiagramFromState(structure, state) {
  const ext = structure.poolExterno || [];
  const lanes = structure.poolPrincipal.lanes || [];
  const N = lanes.length;
  const EH = 160, EG = 20;
  const X0 = 150, DX = 160;
  const BRANCH_GAP = 110; // > altura de tarea (80) para que las ramas no se
    solapen

  const steps = state.steps || [];
  const starts = steps.filter(s => s.tipo === 'startEvent');
  const ends = steps.filter(s => s.tipo === 'endEvent' || s.tipo === '
    errorEndEvent');
  const mains = steps.filter(s => s.tipo !== 'startEvent' && s.tipo !== 'endEvent'
    && s.tipo !== 'errorEndEvent');

  // Altura de lane: si hay gateways con ramas, dejar sitio para el abanico
    vertical
  const maxBranches = Math.max(1, ...mains.filter(m => m.branches?.length)
    .map(m => m.branches.length));
  const LH = Math.max(180, maxBranches * BRANCH_GAP + 110);

  // Posiciones X: recorrido secuencial. Las ramas de un gateway se dibujan en
  // paralelo (mismo rango X, desplazamiento vertical _yOffset por rama) y
  // convergen, si procede, en el gateway de unión (join).
  starts.forEach(s => { s._x = X0; s._yOffset = 0; });
  let cursorX = X0 + DX;
  const branchExtras = [];
  mains.forEach(step => {
    step._yOffset = 0;
    if ((step.tipo === 'exclusiveGateway' || step.tipo === 'parallelGateway'
      || step.tipo === 'inclusiveGateway') && step.branches?.length) {
      step._x = cursorX; cursorX += DX;
      let maxLen = 0;
      step.branches.forEach((branch, bi) => {
        const yOff = (bi - (step.branches.length - 1) / 2) * BRANCH_GAP;
        (branch.steps || []).forEach((bstep, j) => {
          bstep._x = cursorX + j * DX; bstep._yOffset = yOff;
          branchExtras.push(bstep);
        });
        maxLen = Math.max(maxLen, (branch.steps || []).length);
      });
      cursorX += Math.max(maxLen, 1) * DX;
      if (step.join) {
        step.join._x = cursorX; step.join._yOffset = 0;
        branchExtras.push(step.join);
        cursorX += DX;
      }
    } else {
      step._x = cursorX; cursorX += DX;
    }
  });
  const endX = cursorX;
  ends.forEach(s => { s._x = endX; s._yOffset = 0; });

  const ordered = [...starts, ...mains, ...branchExtras, ...ends];

  const PW = Math.max(1200, endX + 300);
```



```
const extY = i => 30 + i * (EH + EG);
const extCY = i => extY(i) + EH / 2;
const mainY = ext.length > 0 ? 30 + ext.length * (EH + EG) : 30;
const mainH = Math.max(1, N) * LH;
const laneCY = j => N > 0 ? mainY + j * LH + LH / 2 : mainY + mainH / 2;

// -- Collaboration
let col = '';
ext.forEach((p, i) => {
  col += `    <participant id="Part_Ext${i+1}" name="${p.nombre}" processRef="
  Proc_Ext${i+1}" />\n`;
});
col += `    <participant id="Part_Main" name="${structure.poolPrincipal.nombre}"
  processRef="Proc_Main" />\n`;
ordered.forEach(s => {
  if (s.participantIdx !== undefined) {
    col += `    <messageFlow id="MF_${s.id}" name="${s.nombre}" sourceRef="${s.id}
    " targetRef="Part_Ext${s.participantIdx+1}" />\n`;
  }
  if (s.tipo === 'startEvent' && s.messageTrigger && s.fromParticipantIdx !==
  undefined) {
    col += `    <messageFlow id="MF_${s.id}_in" sourceRef="Part_Ext${s.
    fromParticipantIdx+1}" targetRef="${s.id}" />\n`;
  }
});

// -- External processes
let proc = '';
ext.forEach((p, i) => { proc += `    <process id="Proc_Ext${i+1}" isExecutable="
  false" />\n`; });

// -- Lane sets
let laneSetXml = '';
lanes.forEach((lane, j) => {
  const refs = ordered.filter(t => t.laneIdx === j)
    .map(t => `    <flowNodeRef>${t.id}</flowNodeRef>`).join('\n');
  laneSetXml += `    <lane id="Lane${j+1}" name="${lane}">\n${refs ? refs + '\n'
  : ''}    </lane>\n`;
});

// -- Task / gateway / event elements
const taskXml = ordered.map(t => {
  const n = (t.nombre || '').replace(/&/g, '&amp;').replace(/</g, '&lt;');
  .replace(/>/g, '&gt;').replace(/"/g, '&quot;');
  if (t.tipo === 'startEvent')
    return `    <startEvent id="${t.id}" name="${n}">${t.messageTrigger
    ? `<messageEventDefinition id="MED_${t.id}" />` : ''}</startEvent>`;
  if (t.tipo === 'endEvent')
    return `    <endEvent id="${t.id}" name="${n}"><messageEventDefinition id="MED_${
    t.id}" /></endEvent>`;
  if (t.tipo === 'exclusiveGateway') return `    <exclusiveGateway id="${t.id}"
  name="${n}" />`;
  if (t.tipo === 'parallelGateway') return `    <parallelGateway id="${t.id}"
  name="${n}" />`;
  if (t.tipo === 'inclusiveGateway') return `    <inclusiveGateway id="${t.id}"
  name="${n}" />`;
  if (t.tipo === 'intermediateCatchEvent')
    return `    <intermediateCatchEvent id="${t.id}" name="${n}"><
  messageEventDefinition id="MED_${t.id}" /></intermediateCatchEvent>`;
});
```

6.2. Implementación de los componentes técnicos clave

```
if (t.tipo === 'intermediateThrowEvent')
  return `    <intermediateThrowEvent id="${t.id}" name="${n}"><
messageEventDefinition id="MED_${t.id}"/></intermediateThrowEvent>`;
if (t.tipo === 'compensationEvent')
  return `    <intermediateThrowEvent id="${t.id}" name="${n}"><
compensateEventDefinition id="CED_${t.id}"/></intermediateThrowEvent>`;
if (t.tipo === 'timerEvent')
  return `    <intermediateCatchEvent id="${t.id}" name="${n}"><
timerEventDefinition id="TED_${t.id}"/></intermediateCatchEvent>`;
if (t.tipo === 'sendTask')    return `    <sendTask    id="${t.id}" name="${n}
"/>`;
if (t.tipo === 'userTask')    return `    <userTask    id="${t.id}" name="${n}
"/>`;
if (t.tipo === 'serviceTask') return `    <serviceTask id="${t.id}" name="${n}
"/>`;
if (t.tipo === 'errorEndEvent')
  return `    <endEvent id="${t.id}" name="${n}"><errorEventDefinition id="
EED_${t.id}"/></endEvent>`;
return `    <task id="${t.id}" name="${n}">`;
}).join('\n');

// -- Sequence flows
const escName = s => (s || '').replace(/&/g, '&amp;').replace(/</g, '&lt;');
                                .replace(/>/g, '&gt;').replace(/"/g, '&quot;');

const chain = [];
if (mains.length > 0) {
  starts.forEach(s => chain.push([s.id, mains[0].id]));
  mains.forEach((m, i) => {
    const next = mains[i+1];
    if ((m.tipo === 'exclusiveGateway' || m.tipo === 'parallelGateway'
      || m.tipo === 'inclusiveGateway') && m.branches?.length) {
      m.branches.forEach(branch => {
        const bsteps = branch.steps || [];
        const flowName = m.tipo !== 'parallelGateway' ? branch.nombre : '';
        if (bsteps.length > 0) {
          chain.push([m.id, bsteps[0].id, flowName]);
          for (let j = 0; j < bsteps.length - 1; j++) chain.push([bsteps[j].id,
bsteps[j+1].id]);
          if (!branch.endsHere && m.join) chain.push([bsteps[bsteps.length-1].id,
m.join.id]);
        } else if (m.join) {
          chain.push([m.id, m.join.id, flowName]);
        }
      });
    }
    if (m.join) {
      if (next) chain.push([m.join.id, next.id]);
      else ends.forEach(e => chain.push([m.join.id, e.id]));
    }
    } else {
      if (next) chain.push([m.id, next.id]);
      else ends.forEach(e => chain.push([m.id, e.id]));
    }
  });
} else {
  starts.forEach(s => ends.forEach(e => chain.push([s.id, e.id])));
}
const seqXml = chain.map(([src, tgt, name], i) =>
  `    <sequenceFlow id="SF_${i}" sourceRef="${src}" targetRef="${tgt}"${name ? `
name="${escName(name)}" ` : ''}/>`
).join('\n');
```

```
const laneSetBlock = N > 0 ? `      <laneSet id="LaneSet_1">\n${laneSetXml}    </
  laneSet>\n` : ``;
proc += `\n <process id="Proc_Main" isExecutable="false">
${laneSetBlock}${taskXml ? taskXml+'\n' : ``}${seqXml ? seqXml+'\n' : ``} </
  process>`;

// -- DI shapes
let shapes = `` , edges = ``;

ext.forEach((p, i) => {
  shapes += `      <bpmndi:BPMNShape id="Shape_Part_Ext${i+1}" bpmnElement="
  Part_Ext${i+1}" isHorizontal="true">
    <dc:Bounds x="30" y="${extY(i)}" width="${PW}" height="${EH}" />
  </bpmndi:BPMNShape>\n`;
});
shapes += `      <bpmndi:BPMNShape id="Shape_Part_Main" bpmnElement="Part_Main"
  isHorizontal="true">
    <dc:Bounds x="30" y="${mainY}" width="${PW}" height="${mainH}" />
  </bpmndi:BPMNShape>\n`;
lanes.forEach((_, j) => {
  shapes += `      <bpmndi:BPMNShape id="Shape_Lane${j+1}" bpmnElement="Lane${j
+1}" isHorizontal="true">
    <dc:Bounds x="60" y="${mainY+j*LH}" width="${PW-30}" height="${LH}" />
  </bpmndi:BPMNShape>\n`;
});

const isGwTipo = t => t === 'exclusiveGateway' || t === 'parallelGateway' || t
  === 'inclusiveGateway';
const isEvTipo = t => t === 'intermediateCatchEvent' || t === '
  intermediateThrowEvent'
  || t === 'compensationEvent' || t === 'timerEvent'
  || t === 'startEvent' || t === 'endEvent' || t === '
  errorEndEvent';
const shapeW = t => isGwTipo(t.tipo) ? 50 : isEvTipo(t.tipo) ? 36 : 100;
const shapeH = t => isGwTipo(t.tipo) ? 50 : isEvTipo(t.tipo) ? 36 : 80;
const elemCY = t => laneCY(t.laneIdx) + (t._yOffset || 0);

ordered.forEach(t => {
  const w = shapeW(t), h = shapeH(t);
  shapes += `      <bpmndi:BPMNShape id="Shape_${t.id}" bpmnElement="${t.id}"${
isGwTipo(t.tipo) ? ' isMarkerVisible="true"' : ``}>
    <dc:Bounds x="${t._x-w/2}" y="${elemCY(t)-h/2}" width="${w}" height="${h
}" />
  </bpmndi:BPMNShape>\n`;
});

// -- DI edges (sequence flows)
const elemOf = id => ordered.find(o => o.id === id);
chain.forEach(([src, tgt], i) => {
  const s = elemOf(src), t = elemOf(tgt);
  if (!s || !t) return;
  const sy = elemCY(s), ty = elemCY(t);
  let wps;
  if (Math.abs(sy - ty) < 1) {
    wps = [[s._x + shapeW(s) / 2, sy], [t._x - shapeW(t) / 2, ty]];
  } else if (isGwTipo(s.tipo)) {
    const gy = sy + (ty > sy ? shapeH(s) / 2 : -shapeH(s) / 2);
    wps = [[s._x, gy], [s._x, ty], [t._x - shapeW(t) / 2, ty]];
  } else if (isGwTipo(t.tipo)) {
    const gy = ty + (sy > ty ? shapeH(t) / 2 : -shapeH(t) / 2);
    wps = [[s._x + shapeW(s) / 2, sy], [t._x, sy], [t._x, gy]];
  }
});
```

6.2. Implementación de los componentes técnicos clave

```
    } else {
      const sx = s._x + shapeW(s) / 2, tx = t._x - shapeW(t) / 2;
      wps = [[sx, sy], [(sx + tx) / 2, sy], [(sx + tx) / 2, ty], [tx, ty]];
    }
    edges += `      <bpmndi:BPMNEdge id="Edge_SF_${i}" bpmnElement="SF_${i}">
    ${wps.map(([x, y]) => `      <di:waypoint x="${x}" y="${y}" />`).join('\n')}
      </bpmndi:BPMNEdge>\n`;
  });

  // -- DI edges (message flows): salientes
  ordered.filter(t => t.participantIdx !== undefined).forEach(t => {
    const srcY = elemCY(t) - shapeH(t) / 2;
    edges += `      <bpmndi:BPMNEdge id="Edge_MF_${t.id}" bpmnElement="MF_${t.id}">
      <di:waypoint x="${t._x}" y="${srcY}" />
      <di:waypoint x="${t._x}" y="${extCY(t.participantIdx)}" />
      </bpmndi:BPMNEdge>\n`;
  });

  // -- DI edges (message flows): entrantes
  starts.filter(s => s.messageTrigger && s.fromParticipantIdx !== undefined).
    forEach(s => {
      const tgtY = elemCY(s) - shapeH(s) / 2;
      edges += `      <bpmndi:BPMNEdge id="Edge_MF_${s.id}_in" bpmnElement="MF_${s.id}_in">
        <di:waypoint x="${s._x}" y="${extCY(s.fromParticipantIdx)}" />
        <di:waypoint x="${s._x}" y="${tgtY}" />
        </bpmndi:BPMNEdge>\n`;
    });

  return `<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL"
  xmlns:bpmndi="http://www.omg.org/spec/BPMN/20100524/DI"
  xmlns:dc="http://www.omg.org/spec/DD/20100524/DC"
  xmlns:di="http://www.omg.org/spec/DD/20100524/DI"
  id="Definitions_1" targetNamespace="http://bpmn.io/schema/bpmn">
  <collaboration id="Collab_1">
    ${col} </collaboration>
    ${proc}
    <bpmndi:BPMNDiagram id="BPMNDiagram_1">
      <bpmndi:BPMNPlane id="BPMNPlane_1" bpmnElement="Collab_1">
        ${shapes}${edges} </bpmndi:BPMNPlane>
      </bpmndi:BPMNDiagram>
    </definitions>`;
}

export function buildStructureBPMN(structure) {
  return renderDiagramFromState(structure, { steps: [] });
}
```

6.2.4. Prompts de refinamiento conversacional

La herramienta `refine_diagram` permite modificar el diagrama mediante instrucciones en lenguaje natural. Opera en dos partes: el *prompt* de sistema (`buildRefinementSystemPrompt`) fija las reglas de modificación del XML, y el mensaje de usuario (`buildRefinementMessage()`) inyecta en cada petición el inventario de elementos actuales, el XML BPMN completo y la instrucción del usuario. Este contexto enriquecido es necesario porque el LLM necesita conocer los identificadores exactos de los `sequenceFlow` para reconectar el flujo tras una modificación.

6.2.4.1. buildRefinementSystemPrompt

Eres un experto BPMN 2.0. Tu tarea es modificar un diagrama BPMN existente según las instrucciones del usuario.

REGLAS GENERALES:

- Devuelve SOLO XML válido completo (<?xml ... </definitions>), sin texto ni markdown.
- Mantén TODOS los pools, lanes y la estructura de piscinas existente.
- Mantén los messageFlows hacia actores externos que sigan siendo válidos.
- Modifica, añade o elimina SOLO lo que el usuario pida; no alteres el resto del diagrama.
- Si el usuario pide eliminar un elemento, elimínalo del proceso, de los sequenceFlows (reconecta los flujos para que el diagrama siga siendo válido) y de la sección BPMNDiagram.
- Si el usuario pide añadir un elemento, insértalo en el punto correcto del flujo, con los sequenceFlows adecuados, y añádele su BPMNShape en la sección de diagrama con coordenadas razonables (entre el elemento anterior y el siguiente).

TIPOS DE ELEMENTO -- usa el correcto según la acción:

- userTask: acción realizada por una PERSONA (revisar, aprobar, gestionar, atender ...).
- serviceTask: acción automática del SISTEMA (consultar API, procesar, generar, calcular).
- sendTask: ENVIAR mensaje/notificación a un actor EXTERNO. Requiere messageFlow.
- exclusiveGateway: decisión con caminos mutuamente excluyentes (XOR).
- parallelGateway: actividades que ocurren en paralelo (AND).
- inclusiveGateway: uno o varios caminos según condiciones (OR).
- timerEvent (intermediateCatchEvent+timerEventDefinition): espera temporal.
- endEvent con errorEventDefinition: fin con error irrecuperable.
- task: SOLO si la acción es genuinamente ambigua.

COHERENCIA:

- Cada elemento debe tener al menos un sequenceFlow entrante y uno saliente (excepto startEvent y endEvent).
- Los nombres de los elementos deben ser descriptivos y coherentes con el proceso.
- Si el usuario pide algo que no tiene sentido en el contexto del proceso, adáptalo razonablemente al contexto o interpreta la intención más lógica.
- Asegúrate de que las coordenadas (Bounds) de elementos nuevos no se solapen con los existentes.

6.2.4.2. buildRefinementMessage

=== ESTRUCTURA DEL DIAGRAMA ===

Organización: [nombre de la empresa, p.ej. TiendaOnline]
Departamentos (lanes): [Ventas, Logística, Atención al Cliente]
Actores externos: [Cliente, Transportista]

=== ELEMENTOS ACTUALES DEL DIAGRAMA ===

[inventario numerado de elementos del diagrama, p.ej.:]
1. [Ventas] Tarea de usuario: "Registrar pedido"
2. [Logística] Gateway exclusivo: "Verificar stock"
- Caso "Hay stock": Tarea de usuario: "Preparar envío" [Logística] -> Tarea de servicio: "Generar etiqueta" [Logística] [converge]

6.2. Implementación de los componentes técnicos clave

```
- Caso "Sin stock": Tarea de usuario: "Notificar al cliente" [Ventas] [termina
]
3. [Logística] Tarea de servicio: "Actualizar inventario"
4. [Logística] Envío de mensaje: "Confirmar envío" -> Cliente

=== XML BPMN ACTUAL ===
[XML BPMN completo del diagrama en su estado actual]

=== INSTRUCCIÓN DEL USUARIO ===
"[instrucción de modificación del usuario, p.ej.: añade un evento de espera
de 24 horas antes de la preparación del envío]"

Aplica la modificación y devuelve el XML COMPLETO (desde <?xml hasta </definitions
>)
con la sección bpmndi:BPMNDiagram.
```

6.2.5. Prompt de análisis PERVAL

La herramienta `analyze_perval` guía al LLM en la clasificación de los elementos del proceso según las cuatro dimensiones de valor percibido del modelo de Sweeney y Soutar [12]. El *prompt* de sistema parametriza el actor de referencia y el alcance del análisis (entregas solo evalúa los mensajes hacia el cliente externo; sin alcance definido, evalúa todas las tareas). El mensaje de usuario transmite la lista numerada de elementos y el XML BPMN completo.

6.2.5.1. `buildPervalSystemPrompt`

Eres un experto en análisis de valor percibido PERVAL (Sweeney y Soutar, 2001) aplicado a e-commerce.

[SI el proceso tiene entregas/mensajes explícitos hacia el cliente externo:]
El valor percibido se calcula sobre LO QUE LA EMPRESA DEVUELVE/ENVÍA AL CLIENTE (mensajes, confirmaciones, notificaciones, entregables...), identificado a partir de los flujos de mensaje hacia el participante externo del cliente. NO evalúes tareas puramente internas que no generan ninguna comunicación o entrega hacia el cliente.

[SI NO hay entregas explícitas identificadas:]
No se han identificado comunicaciones o entregas explícitas hacia un participante externo cliente, así que se analizan todas las tareas del proceso.

[SI se especifica un actor de referencia "[nombre del actor]":]
FOCO: analiza SOLO los elementos que tienen impacto directo o indirecto en la experiencia de "[nombre del actor]". Ignora los elementos puramente internos sin relación con este actor.

[SI no se especifica actor concreto:]
Analiza todos los elementos indicados.

DIMENSIONES PERVAL:

- Quality (Calidad): fiabilidad, seguridad, rendimiento, satisfacción del servicio
- Price (Precio): ahorro económico, transparencia de costes, relación calidad-precio
- Emotional (Emocional): conveniencia, facilidad, personalización, bienestar, tranquilidad
- Social (Social): reconocimiento, accesibilidad, flexibilidad de entrega/pago

Elementos sin impacto en el actor -> clasificarlos como "Interno".

NIVEL DE RENDIMIENTO -- para cada tarea, evalúa cómo de bien el proceso entrega valor percibido al actor y asigna un nivel:

- "muy_bueno": la tarea entrega un valor percibido excelente al actor
- "bueno": la tarea entrega un buen valor percibido
- "regular": el valor percibido es neutro o mejorable
- "malo": el valor percibido es bajo o deficiente
- "muy_malo": la tarea genera una percepción muy negativa

RESPONDE ÚNICAMENTE con JSON:

```
{
  "tareas": [{ "nombre": "...", "dimensiones": ["Quality"], "nivel": "bueno",
               "valor": "...", "justificacion": "..."}],
  "resumen": { "Quality": "...", "Price": "...", "Emotional": "...", "Social": "..."},
  "valorGeneral": "..."
}
```

6.2.5.2. buildPervalUserMessage

```
[SI scope = 'entregas':]
ENTREGAS/COMUNICACIONES DE LA EMPRESA AL CLIENTE:
[SI se especifica actor:] Actor a analizar: [nombre del actor]
1. Envío de mensaje: Confirmación de pedido al cliente
2. Envío de mensaje: Notificación de envío al cliente
3. Envío de mensaje: Entrega completada -- mensaje de seguimiento

[SI scope != 'entregas' (todas las tareas):]
TAREAS DEL PROCESO:
1. Registrar pedido
2. Verificar disponibilidad de stock
3. Preparar envío
4. Generar etiqueta de envío

XML BPMN:
[XML BPMN completo del diagrama]
```

6.2.6. Robustez ante respuestas imprecisas del LLM

Incluso con *prompts* que exigen respuestas en formato estricto, los LLMs producen con cierta frecuencia salidas que envuelven el contenido en bloques de código Mark-down, añaden texto explicativo o truncan la respuesta al alcanzar el límite de *tokens*. Las dos funciones siguientes, implementadas en `mcp-server/src/llm.js`, gestionan estos casos.

6.2.6.1. parseLLMJson

Extrae y deserializa el primer objeto JSON completo de la respuesta del LLM. Su autómata de estados cuenta llaves de apertura y cierre distinguiendo correctamente el contenido de las cadenas (incluidas secuencias de escape) del delimitador estructural, lo que lo hace más robusto que una expresión regular codiciosa que busca el primer `{` y el último `}`.

```
export function parseLLMJson(raw) {
```

6.2. Implementación de los componentes técnicos clave

```
const text = (raw || '').replace(/``(?:json)?/gi, '').trim();
try { return JSON.parse(text); } catch (e) { /* hay texto extra: se busca el
  objeto */ }
const start = text.indexOf('{');
if (start === -1) throw new Error('La respuesta no contiene JSON');
let depth = 0, inStr = false, esc = false;
for (let i = start; i < text.length; i++) {
  const ch = text[i];
  if (esc) { esc = false; continue; }
  if (ch === '\\') { esc = inStr; continue; }
  if (ch === '"') { inStr = !inStr; continue; }
  if (inStr) continue;
  if (ch === '{') depth++;
  else if (ch === '}') {
    depth--;
    if (depth === 0) return JSON.parse(text.slice(start, i + 1));
  }
}
return JSON.parse(text.slice(start)); // JSON truncado: que el error sea
  descriptivo
}
```

6.2.6.2. cleanXML

Sanea el XML BPMN generado por el LLM antes de intentar cargarlo en *bpmn-js*. Aplica un *pipeline* de cuatro transformaciones: (1) eliminación de bloques Markdown, (2) extracción del bloque XML, (3) reparación de comillas de cierre omitidas y (4) reparación de truncaciones añadiendo los cierres de etiqueta necesarios.

```
export function cleanXML(raw) {
  let xml = raw.replace(/``xml/gi, '').replace(/``bpmn/gi, '').replace(/``/g,
    '').trim();
  const idx = xml.indexOf('<?xml');
  if (idx > -1) xml = xml.substring(idx);
  if (!xml.startsWith('<?xml')) {
    const d = xml.indexOf('<definitions');
    if (d > -1) xml = xml.substring(d);
  }

  // -- Reparación 1: comilla de cierre olvidada antes de />
  // El LLM a veces genera: height="36/> en lugar de height="36"/>
  xml = xml.replace(/="(^[^">\n\r]*)\s*\>/g, '="$1"/>');

  // -- Reparación 2: truncaciones (XML cortado antes de </definitions>)
  if (!xml.includes('</definitions>')) {
    const lastGt = xml.lastIndexOf('>');
    if (lastGt > 0) xml = xml.substring(0, lastGt + 1);

    const needsPlane = xml.includes('<bpmndi:BPMNPlane') && !xml.includes('</bpmndi:
      BPMNPlane>');
    const needsDiag = xml.includes('<bpmndi:BPMNDiagram') && !xml.includes('</
      bpmndi:BPMNDiagram>');
    if (needsPlane) xml += '\n    </bpmndi:BPMNPlane>';
    if (needsDiag) xml += '\n  </bpmndi:BPMNDiagram>';
    xml += '\n</definitions>';
  }

  if (!xml.includes('<bpmndi:BPMNDiagram') && !xml.includes('<BPMNDiagram'))
```



```
    throw new Error(t('errNoDiagramSection'));
    return xml;
}
```

6.2.7. Funciones auxiliares

Las dos funciones siguientes, implementadas en `mcp-server/src/helpers.js`, son utilizadas internamente por los *prompts* de generación para construir el resumen cronológico de pasos y para normalizar la denominación de los departamentos cuando la organización no tiene calles definidas.

6.2.7.1. effectiveLanes

Devuelve la lista de departamentos efectivos para construir el contexto de los *prompts*. Si la organización no tiene *lanes* (piscina única), usa el nombre de la organización como única «calle» lógica con índice 0, de modo que las funciones de *prompt* nunca reciben una lista vacía.

```
export function effectiveLanes(structure) {
  const l = structure?.poolPrincipal?.lanes || [];
  return l.length > 0 ? l : [structure?.poolPrincipal?.nombre || t('defaultOrgName')];
}
```

6.2.7.2. elementLabel

Traduce el identificador de tipo BPMN (`userTask`, `exclusiveGateway`, etc.) a la etiqueta legible en el idioma activo de la interfaz, usando el sistema de internacionalización `i18n`. Es utilizado por `buildNextStepPrompt` y `buildBranchStepPrompt` para construir el resumen de pasos confirmados que se inyecta como contexto en cada llamada al LLM.

```
export function elementLabel(tipo) {
  const keys = {
    task: 'elTask',
    userTask: 'elUserTask',
    serviceTask: 'elServiceTask',
    sendTask: 'elSendTask',
    intermediateCatchEvent: 'elIntermediateCatch',
    intermediateThrowEvent: 'elIntermediateThrow',
    compensationEvent: 'elCompensation',
    timerEvent: 'elTimer',
    endMessageEvent: 'elEndMessage',
    errorEndEvent: 'elErrorEnd',
    exclusiveGateway: 'elExclusiveGw',
    parallelGateway: 'elParallelGw',
    inclusiveGateway: 'elInclusiveGw',
    startEvent: 'elStart',
    endEvent: 'elEnd',
  };
  return t(keys[tipo] || 'elTask');
}
```


Capítulo 7

Ejecución y funcionamiento del sistema

Este capítulo presenta el sistema en funcionamiento mediante una demostración guiada de sus principales flujos de uso. Se parte de la interfaz general del sistema para, a continuación, recorrer el ciclo completo de trabajo: descripción del proceso en lenguaje natural (por texto o por voz), generación iterativa del diagrama BPMN con validación humana en cada paso, edición conversacional del modelo resultante y análisis de valor percibido según el modelo PERVAL. Se ilustra también el soporte multilingüe español-inglés y el despliegue en producción en la plataforma *Render.com*.

7.1. Interfaz general del sistema

La interfaz del sistema integra en una única ventana de navegador el editor de diagramas BPMN y el panel conversacional que centraliza toda la interacción con el LLM. La figura 7.1 muestra el estado inicial de la aplicación con sus principales zonas identificadas.

El lienzo BPMN (2) es el editor *bpmn-js* estándar, con soporte de arrastrar y soltar, acercar/alejar y edición manual de propiedades. El panel conversacional (3) actúa como canal único de comunicación con el servidor MCP: todas las instrucciones de generación, refinamiento y análisis se introducen aquí, ya sea mediante texto o mediante el botón de voz (4). El selector de idioma (5) cambia simultáneamente los textos de la interfaz y la lengua en que el LLM genera las respuestas.

7.2. Generación iterativa de modelos BPMN

La generación de un diagrama BPMN parte de una descripción en lenguaje natural introducida por el usuario. El sistema ofrece dos modos de generación: el **modo guiado** (paso a paso), en el que el LLM propone cada elemento del flujo individualmente y el usuario confirma o corrige antes de continuar; y el **modo completo**, en el que, tras confirmar la estructura de participantes, el usuario únicamente especifica el evento de inicio y el sistema genera el diagrama íntegro de forma autónoma. Los pasos 1, 2 y 3 son comunes a ambos modos.

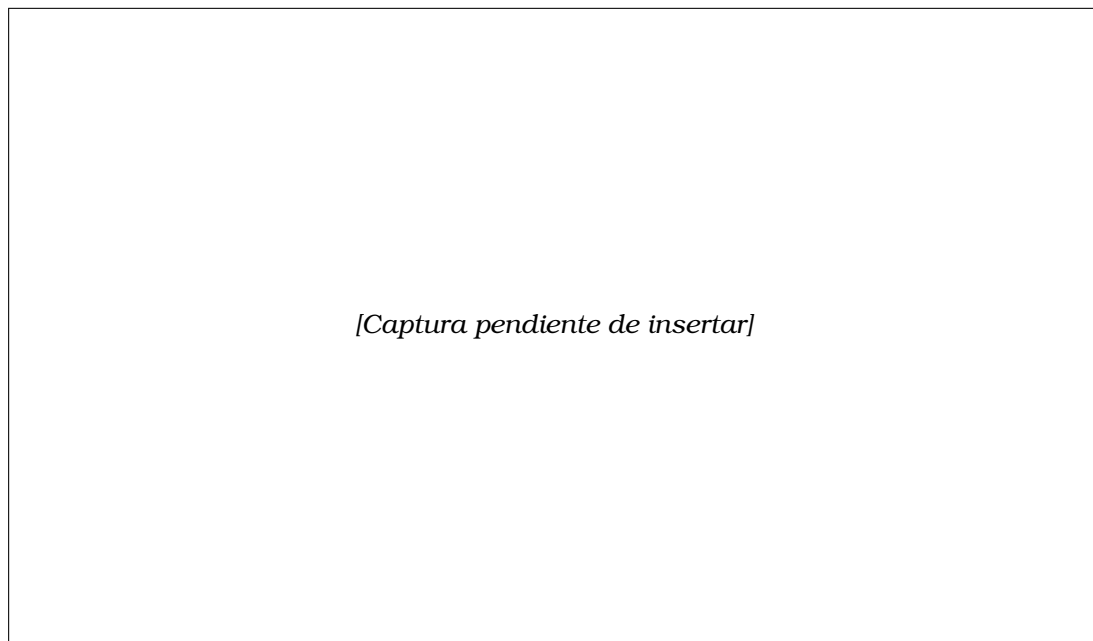


Figura 7.1: Vista general de la interfaz del sistema

7.2.1. Paso 1 — Descripción del proceso

El usuario introduce en el panel conversacional una descripción libre del proceso de negocio que desea modelar. La descripción puede ser breve (dos o tres frases) o extensa; el sistema utiliza su contenido íntegro como contexto para todas las llamadas posteriores al LLM. En la figura 7.2 se muestra un ejemplo con un proceso de compra en línea.

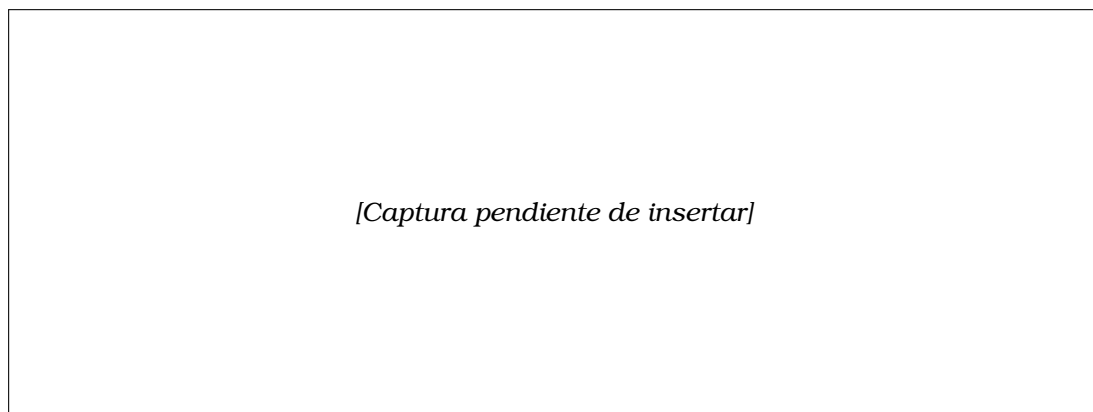


Figura 7.2: Introducción de la descripción del proceso en el panel conversacional. El usuario ha descrito un proceso de compra en línea con varios departamentos internos y actores externos.

7.2.2. Paso 2 — Identificación de participantes y estructura

Tras recibir la descripción, el sistema lanza la herramienta `identify_structure` y presenta al usuario los participantes identificados: el *pool* principal con sus departamentos (*lanes*) y los actores externos con su rol (`cliente` o `colaborador`). La

respuesta se muestra en el chat como una tarjeta estructurada con botones de confirmación y modificación (figura 7.3).

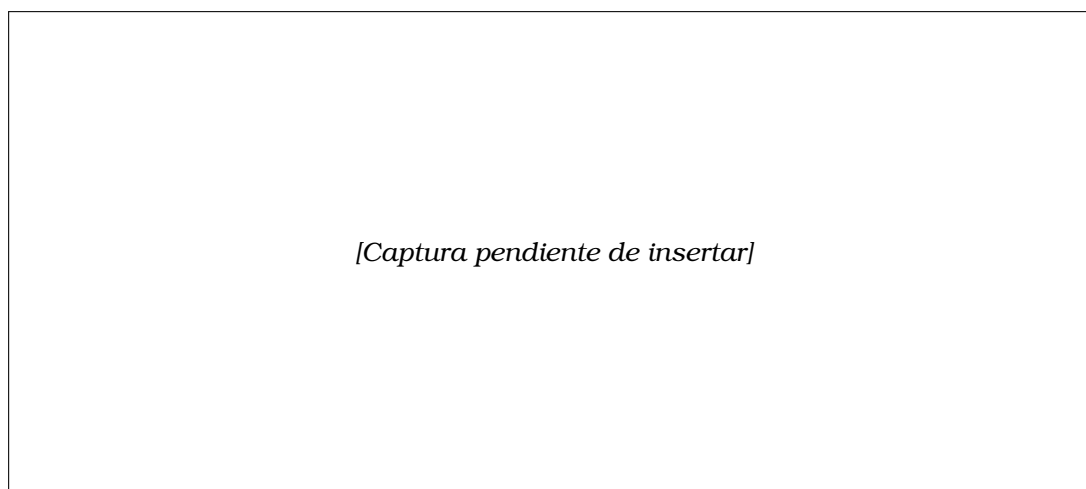


Figura 7.3: Resultado de la identificación de participantes.

Si el usuario no está satisfecho con la estructura propuesta puede introducir una corrección directamente en el chat y el sistema la integra antes de continuar.

7.2.3. Paso 3 — Identificación del inicio del proceso

Una vez confirmada la estructura de participantes, el sistema propone al usuario el punto de entrada del proceso: en qué departamento (lane) comienza y cuál es el primer evento o tarea de inicio. Esta información es necesaria tanto en el modo guiado como en el modo completo, ya que ancla el primer nodo del diagrama antes de inferir el resto del flujo. En la figura 7.4 se muestra un ejemplo de respuesta.

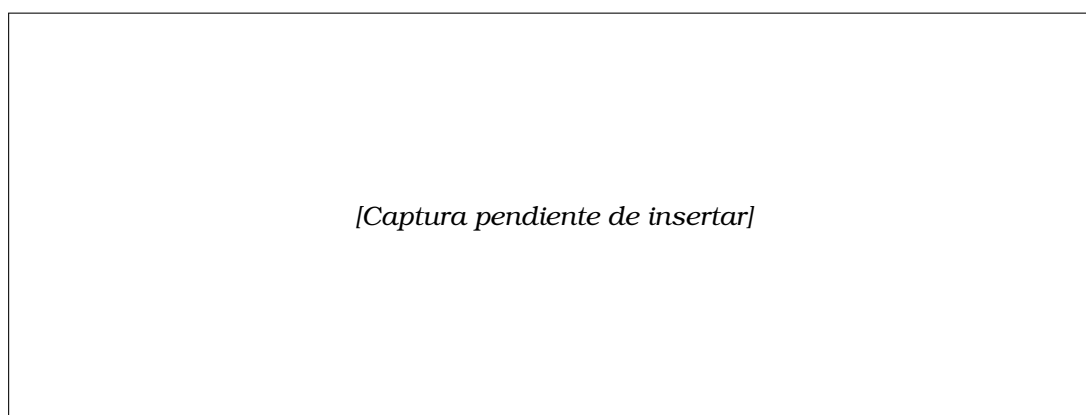


Figura 7.4: Identificación del inicio del proceso.

7.2.4. Paso 4 — Definición del flujo paso a paso

Una vez confirmada la estructura y el punto de inicio, el sistema propone cada elemento del flujo de forma secuencial: tareas de usuario, tareas de servicio, puertas lógicas y eventos. Cuando propone una puerta (*gateway*), solicita también los nombres de los casos que la bifurcan. En cada propuesta el usuario puede aceptar con

un botón, rechazar o reformular el elemento. La figura 7.5 ilustra el momento en que el sistema propone una puerta exclusiva con sus ramas.

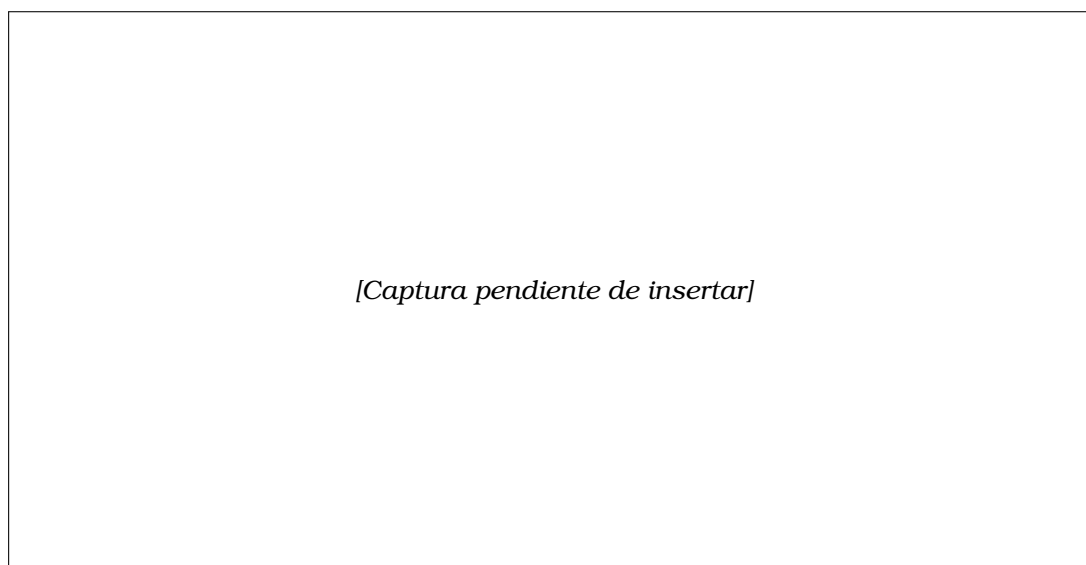


Figura 7.5: Generación paso a paso del flujo del proceso. El sistema propone una puerta exclusiva (XOR).

En cualquier momento durante la generación paso a paso, el usuario dispone de dos acciones adicionales. Por un lado, puede solicitar que el sistema genere automáticamente el resto del diagrama sin más preguntas, lo que equivale a pasar al modo completo a partir del punto en que se encuentra: el LLM toma los pasos ya confirmados como contexto e infiere los elementos restantes hasta completar el proceso. Por otro lado, el usuario puede ampliar o corregir la descripción original introduciendo información adicional en el chat —por ejemplo, añadir una condición de error no mencionada inicialmente o precisar el rol de un actor externo—, con lo que el sistema actualiza su contexto y continúa la generación teniendo en cuenta los nuevos datos.

7.2.5. Paso 5 — Diagrama BPMN generado

Cuando el sistema detecta que el proceso ha llegado a su fin, llama automáticamente a la herramienta `render_diagram`, que construye el XML BPMN 2.0 de forma completamente programática y lo carga en el editor `bpmn-js`. El diagrama aparece centrado en el lienzo y muestra piscinas, calles, tareas, puertas y flujos de secuencia con sus etiquetas (figura 7.6).

Desde este punto el usuario puede editar el diagrama directamente en el lienzo de `bpmn.io` o continuar la interacción mediante el panel conversacional para introducir refinamientos.

7.2.6. Alternativa: generación completa del diagrama

Como alternativa al modo guiado, el sistema permite generar el diagrama completo en una sola llamada al LLM. Tras identificar el inicio del proceso (paso 3), el usuario activa la generación completa; el LLM infiere entonces el flujo íntegro a partir de la descripción original y del nodo de inicio ya confirmado, y llama directamente a `render_diagram` sin requerir confirmaciones intermedias para cada elemento.

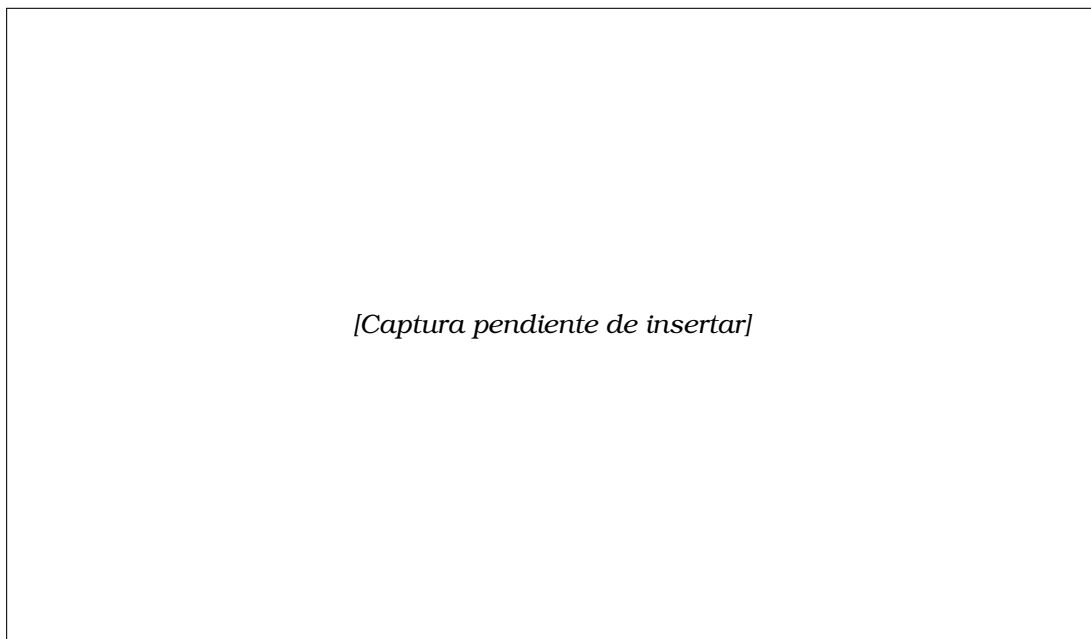


Figura 7.6: Diagrama BPMN 2.0 generado y cargado en el editor bpmn.io. Se aprecian las tres calles del *pool* principal (Ventas, Logística, Atención al Cliente), el *pool* externo del Cliente, los *gateways* con sus ramas y los flujos de mensaje entre participantes.

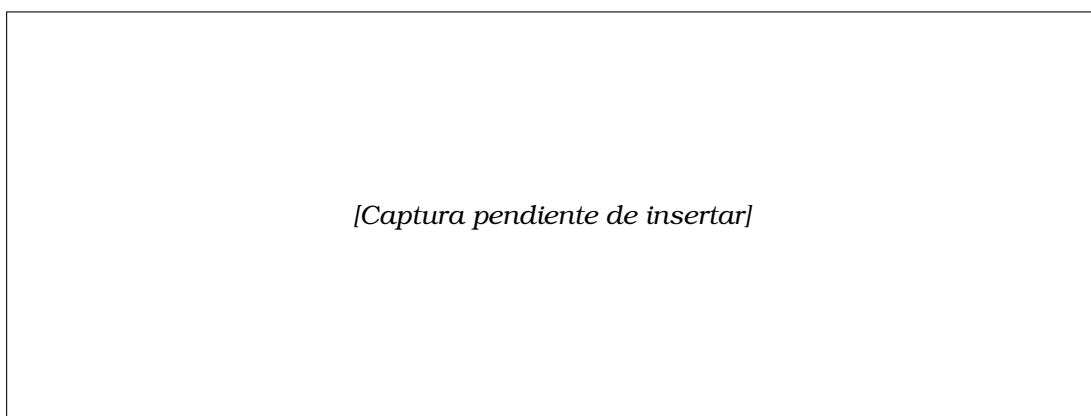


Figura 7.7: Modo de generación completa. Tras confirmar la estructura, el usuario activa la generación automática: el sistema solicita únicamente el evento de inicio y, a continuación, genera y renderiza el diagrama BPMN íntegro en una sola operación.

Este modo resulta especialmente útil cuando el usuario tiene una descripción detallada del proceso y desea obtener un borrador completo rápidamente, que posteriormente podrá refinar mediante instrucciones conversacionales (véase §7.3).

7.3. Edición conversacional del diagrama

La herramienta `refine_diagram` permite modificar el diagrama existente mediante instrucciones en lenguaje natural, sin necesidad de conocer la sintaxis XML de BPMN 2.0. El usuario describe el cambio deseado en el panel conversacional y el sistema devuelve el XML actualizado, que se recarga automáticamente en el editor.

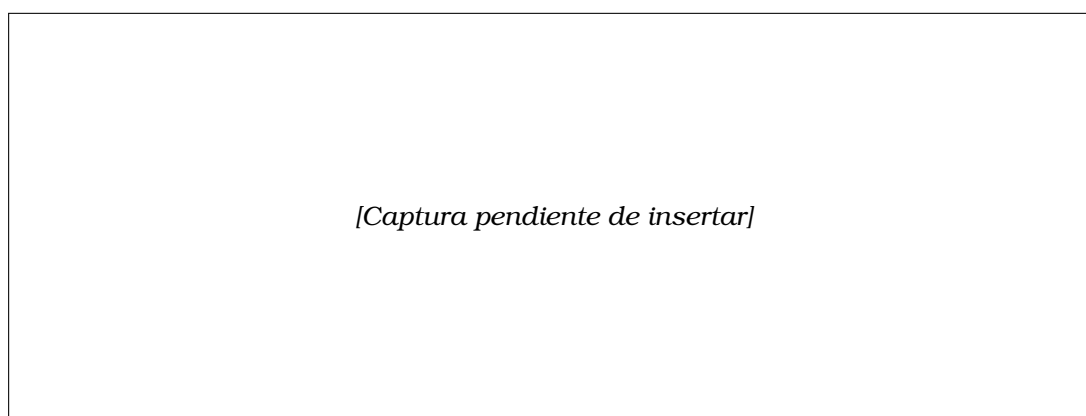


Figura 7.8: Instrucción de refinamiento introducida en el panel conversacional: el usuario solicita añadir un evento de espera entre la preparación del envío y la generación de la etiqueta. El historial de mensajes anterior sigue visible en el panel.

La figura 7.9 muestra el diagrama tras aplicar la modificación: el nuevo evento de temporizador aparece correctamente conectado en el flujo de la calle de Logística, entre las dos tareas indicadas.

7.4. Análisis de valor percibido PERVAL

Una vez generado el diagrama, el usuario puede lanzar el análisis de valor percibido integrado. La herramienta `analyze_perval` clasifica los elementos del proceso según las cuatro dimensiones del modelo PERVAL [12] (*Quality*, *Price*, *Emotional* y *Social*), con un nivel de rendimiento y una justificación textual para cada elemento.

7.4.1. Configuración del análisis

Antes de lanzar el análisis, el usuario selecciona el alcance (analizar solo las entregas al cliente externo, o todas las tareas del proceso) y puede especificar el nombre del actor externo cuyo punto de vista se adopta para la clasificación. El formulario de configuración se muestra en la figura 7.10.

7.4.2. Resultados del análisis

Los resultados se presentan en el panel conversacional en forma de tarjetas individuales por cada elemento analizado, con las dimensiones PERVAL aplicables, el nivel

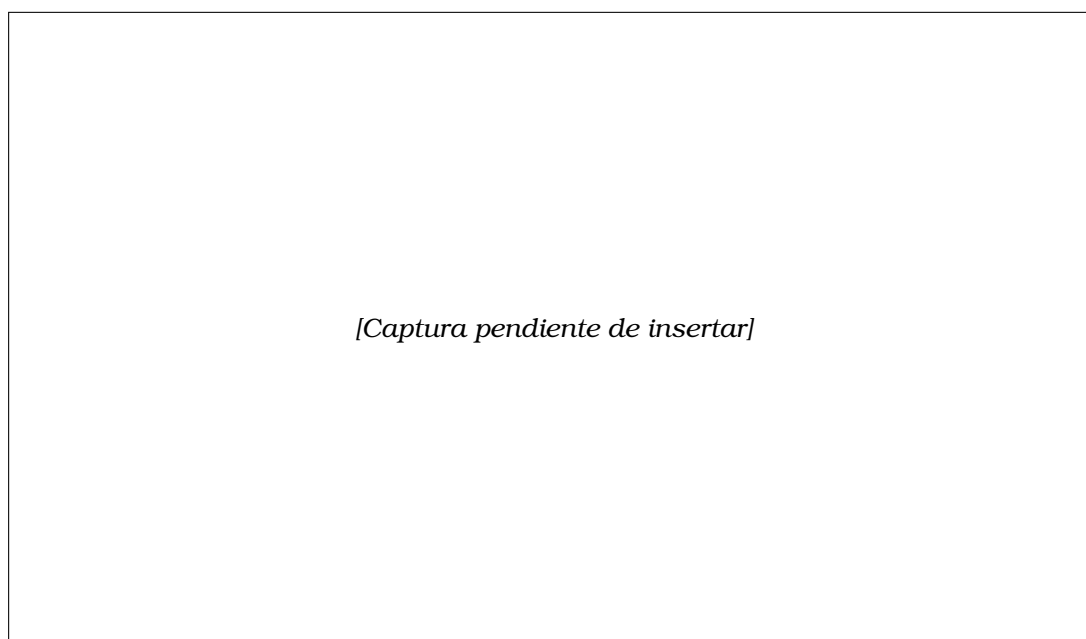


Figura 7.9: Diagrama BPMN actualizado tras la instrucción de refinamiento. Se ha insertado un evento de temporizador en la calle de Logística, con sus flujos de secuencia de entrada y salida correctamente reconectados.

[Captura pendiente de insertar]

Figura 7.10: Formulario de configuración del análisis PERVAL en el panel conversacional. El usuario ha seleccionado el alcance «Entregas al cliente externo» y ha especificado «Cliente» como actor de referencia.

de rendimiento codificado por color y la justificación textual generada por el LLM, seguidas de un resumen global por dimensión y una valoración general del proceso (figura 7.11).

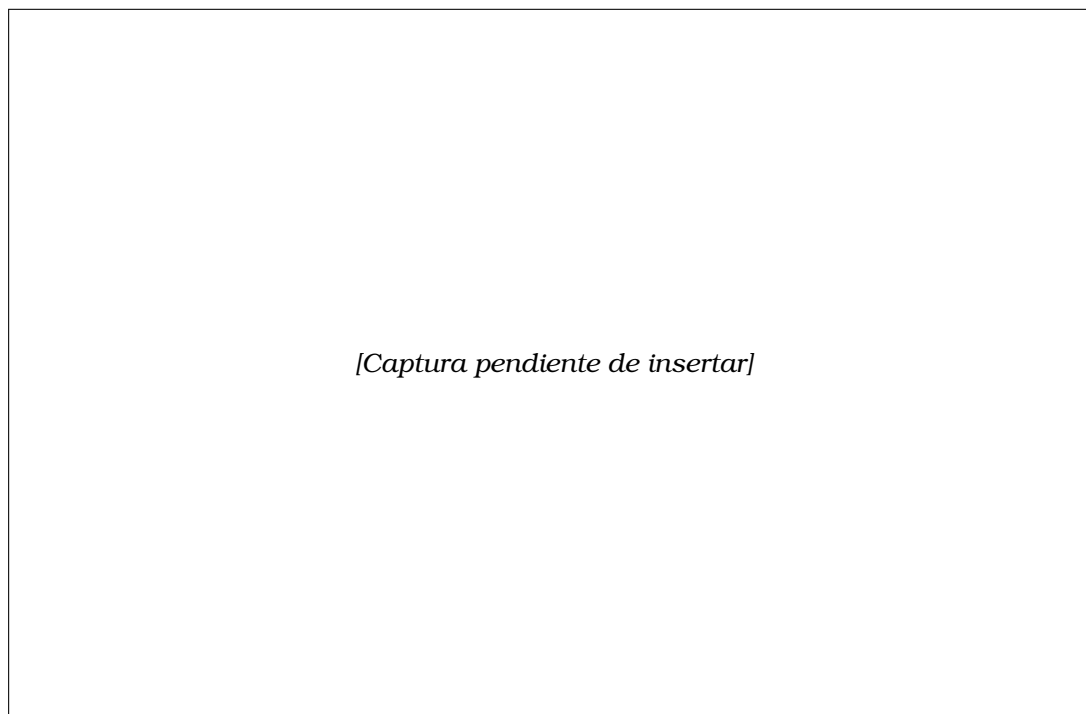


Figura 7.11: Resultados del análisis PERVAL en el panel conversacional. Cada elemento analizado muestra sus dimensiones PERVAL activas con el nivel de rendimiento codificado por color (muy bueno / bueno / regular / malo) y la justificación generada por el LLM. Al pie se presenta el resumen agregado por dimensión.

7.5. Entrada por voz

El sistema admite tanto texto como voz como modalidad de entrada. El botón de micrófono, situado junto al campo de texto del panel conversacional, activa la API de reconocimiento de voz del navegador (*Web Speech API*). Mientras el micrófono está activo, el botón muestra una animación de escucha y transcribe el audio en tiempo real al campo de texto. Al finalizar la locución, la transcripción se envía automáticamente como si el usuario hubiese escrito el mensaje.

Esta modalidad resulta especialmente útil para introducir descripciones largas de procesos o para dictar instrucciones de refinamiento sin necesidad de teclear.

7.6. Soporte multilingüe

El sistema soporta español e inglés de forma completa: al cambiar de idioma mediante el selector de la interfaz, se actualizan simultáneamente todos los textos de la interfaz, los mensajes del sistema en el panel conversacional y la directiva de idioma añadida a cada *prompt*, de modo que las respuestas del LLM —incluidas las justificaciones del análisis PERVAL— se generan directamente en la lengua seleccionada. La

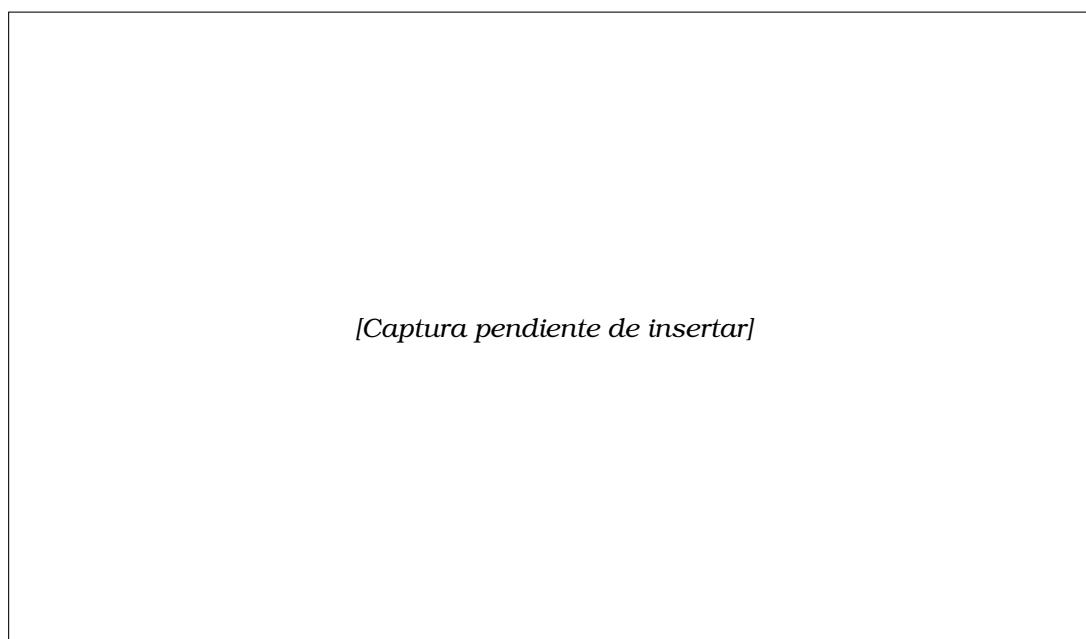


Figura 7.12: Diagrama BPMN con los elementos analizados resaltados visualmente. Los elementos con impacto positivo en el valor percibido aparecen con fondo verde; los que presentan margen de mejora, con fondo ámbar.

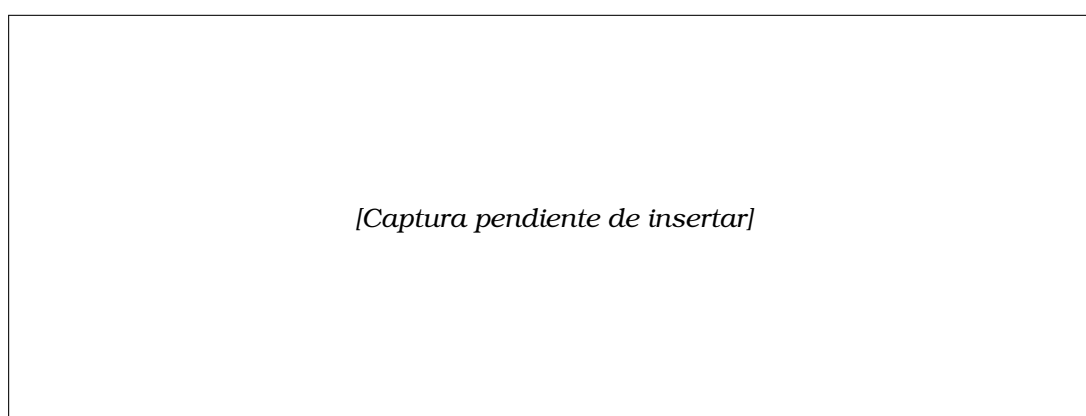


Figura 7.13: Entrada por voz activa. El botón de micrófono muestra la animación de escucha y la transcripción en tiempo real aparece en el campo de texto del panel conversacional.

figura 7.14 muestra la interfaz en modo inglés.

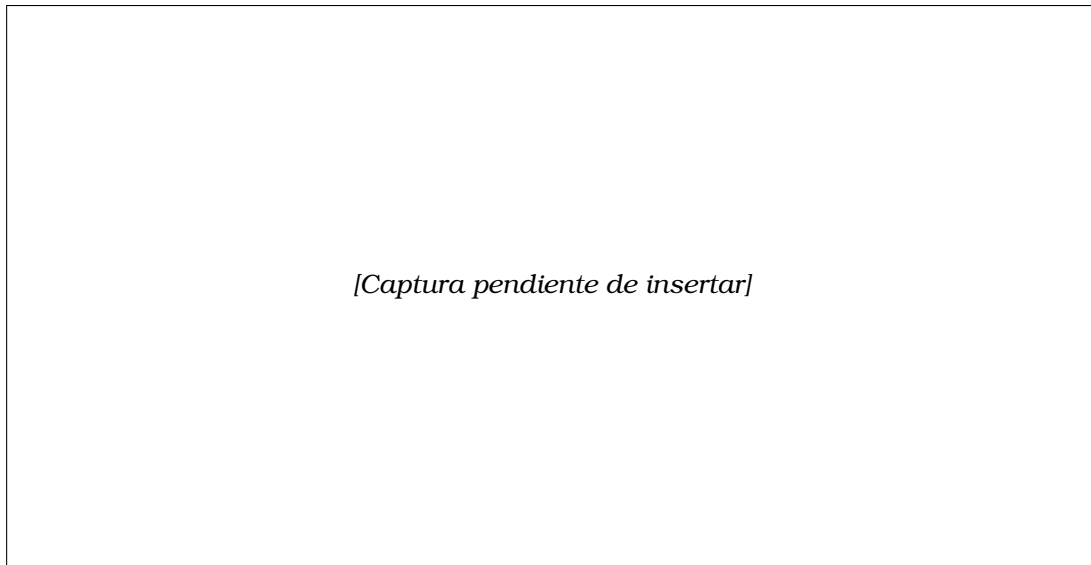


Figura 7.14: Interfaz del sistema en modo inglés. Los mensajes del panel conversacional, los botones de acción y los resultados del análisis PERVAL se muestran en inglés, al igual que los *prompts* enviados al LLM.

7.7. Despliegue en producción

El sistema está desplegado en la plataforma *Render.com* en dos servicios independientes definidos en el fichero `render.yaml` del repositorio. El primero (`tfm-bpmn-frontend`) es un sitio estático que sirve el *bundle* Webpack de la extensión. El segundo (`tfm-bpmn-mcp-server`) es un servicio web Node.js que ejecuta el servidor MCP. Ambos se actualizan automáticamente en cada *push* al repositorio de GitHub y están accesibles mediante HTTPS con certificados gestionados por la propia plataforma.

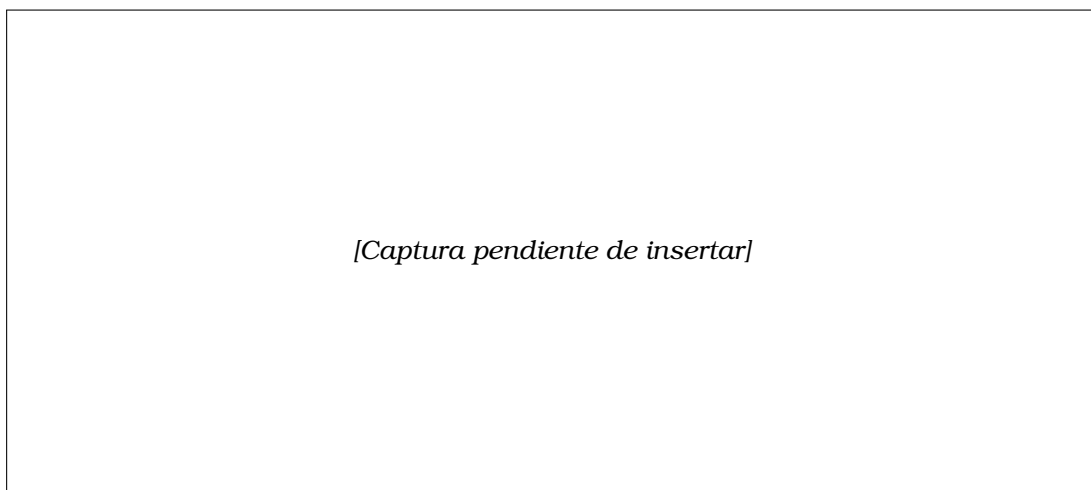


Figura 7.15: Panel de administración de *Render.com* mostrando los dos servicios desplegados: el sitio estático de la extensión y el servidor MCP Node.js, ambos en estado *Live* y con HTTPS activo.

Ejecución y funcionamiento del sistema

El sistema completo es accesible desde cualquier navegador moderno (Chrome, Firefox, Edge) sin necesidad de instalar ningún software adicional, en la dirección <https://tfm-bpmn-frontend.onrender.com/>.

Bibliografía

- [1] H. Kourani, A. Berti, D. Schuster y W. M. P. van der Aalst, "Evaluating Large Language Models on Business Process Modeling: Framework, Benchmark, and Self-Improvement Analysis," *Software and Systems Modeling*, 2025.
- [2] H. Kourani, A. Berti, D. Schuster y W. M. P. van der Aalst, "Process Modeling With Large Language Models," 2024.
- [3] N. D. Riano Nossa, "Estudio comparativo de metodologías tradicionales y ágiles aplicadas en la gestión de proyectos," Bachelor's thesis, Universidad Pontificia Bolivariana (UPB), Colombia, 2021.
- [4] M. Grohs, L. Abb, N. Elsayed y J.-R. Rehse, "Large Language Models can accomplish Business Process Management Tasks," en *BPM 2023 Workshops, Lecture Notes in Business Information Processing*, Springer, Cham, 2024. Disponible en: <https://arxiv.org/abs/2307.09923>.
- [5] L. F. Hörner, M. Möller y M. Reichert, "Automatically Generating BPMN 2.0 Process Models from Natural Language Process Descriptions: Challenges, Framework, Quality Assessment," *Business & Information Systems Engineering*, 2025. DOI: 10.1007/s12599-025-00983-x. Disponible en: <https://link.springer.com/article/10.1007/s12599-025-00983-x>.
- [6] N. Klievtsova, J.-V. Benzin, T. Kampik, J. Mangler y S. Rinderle-Ma, "Conversational Process Modeling: Can Generative AI Empower Domain Experts in Creating and Redesigning Process Models?," *arXiv preprint arXiv:2304.11065*, 2023. Disponible en: <https://arxiv.org/abs/2304.11065>.
- [7] J. Köpke y A. Safan, "Introducing the BPMN-Chatbot for Efficient LLM-Based Process Modeling," en *CEUR Workshop Proceedings*, vol. 3758, paper 15, BPM 2024 Forum, Cracovia, 2024. Disponible en: <https://ceur-ws.org/Vol-3758/paper-15.pdf>.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser y I. Polosukhin, "Attention Is All You Need," en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. Disponible en: <https://arxiv.org/abs/1706.03762>.
- [9] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., "Language Models are Few-Shot Learners," en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020. Disponible en: <https://arxiv.org/abs/2005.14165>.

-
- [10] Object Management Group (OMG), “Business Process Model and Notation (BPMN), Version 2.0,” enero de 2011. Disponible en: <https://www.omg.org/spec/BPMN/2.0/>.
- [11] M. Dumas, M. La Rosa, J. Mendling y H. A. Reijers, *Fundamentals of Business Process Management*, 2.^a ed., Springer, 2018.
- [12] J. C. Sweeney y G. N. Soutar, “Consumer Perceived Value: The Development of a Multiple Item Scale,” *Journal of Retailing*, vol. 77, n.º 2, pp. 203–220, 2001.
- [13] Parlamento Europeo y Consejo de la Unión Europea, “Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (RGPD),” 2016. Disponible en: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [14] Anthropic, “Model Context Protocol (MCP) Specification,” 2024. Disponible en: <https://modelcontextprotocol.io>.
- [15] A. Karpathy, “There’s a new kind of coding I call ‘vibe coding’...,” [Publicación en X (antes Twitter)], 2 de febrero de 2025. Disponible en: <https://x.com/karpathy/status/1886192184808149265>.
- [16] Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., et al. (2021). *On the Opportunities and Risks of Foundation Models*. Stanford Center for Research on Foundation Models (CRFM), Stanford University. Available at: <https://arxiv.org/abs/2108.07258>
- [17] Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., et al. (2022). Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research (TMLR)*.
- [18] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., et al. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), Article 248.
- [19] Feuerriegel, S., Hartmann, J., Janiesch, C., & Zschech, P. (2024). Generative AI. *Business & Information Systems Engineering*, 66(1), 111–126. <https://doi.org/10.1007/s12599-023-00834-7>
- [20] Weske, M. (2019). *Business Process Management: Concepts, Languages, Architectures* (3rd ed.). Berlin: Springer.
- [21] J. Mendling, H. A. Reijers y W. M. P. van der Aalst, “Seven Process Modeling Guidelines (7PMG),” *Information and Software Technology*, vol. 52, n.º 2, pp. 127–136, 2010.
- [22] Zeithaml, V. A. (1988). Consumer Perceptions of Price, Quality, and Value: A Means-End Model and Synthesis of Evidence. *Journal of Marketing*, 52(3), 2–22.

Anexo I

Este capítulo es opcional, y se escribirá de acuerdo con las indicaciones del Tutor.

Anexo II

Este capítulo es opcional, y se escribirá de acuerdo con las indicaciones del Tutor.